



(19) **United States**

(12) **Patent Application Publication**
Brodsky et al.

(10) **Pub. No.: US 2025/0286728 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **FLEXIBLE ACCESS CONTROL USING
EXPRESSIVE MONOTONE-POLICY
AGGREGATE SIGNATURES**

(71) Applicant: **NTT Research, Inc.**, Sunnyvale, CA
(US)

(72) Inventors: **Maya Farber Brodsky**, Tel Aviv (IL);
Arka Rai Choudhuri, Emeryville, CA
(US); **Abhishek Jain**, Baltimore, MD
(US); **Omer Paneth**, Tel Aviv (IL)

(73) Assignee: **NTT Research, Inc.**, Sunnyvale, CA
(US)

(21) Appl. No.: **19/076,922**

(22) Filed: **Mar. 11, 2025**

Related U.S. Application Data

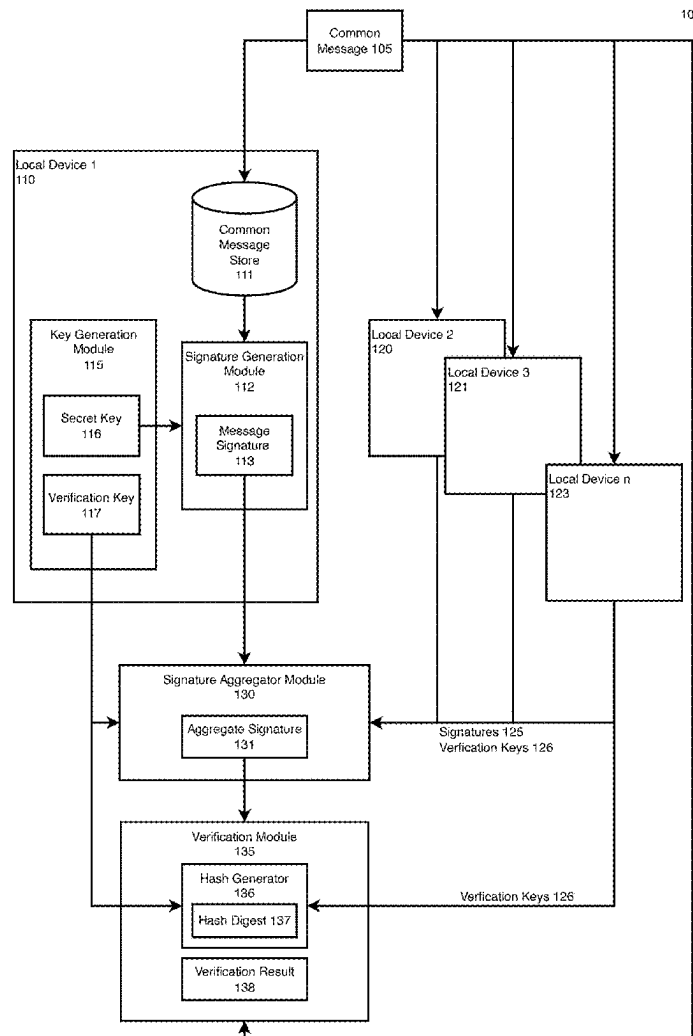
(60) Provisional application No. 63/563,916, filed on Mar.
11, 2024.

Publication Classification

(51) **Int. Cl.**
H04L 9/32 (2006.01)
H04L 9/08 (2006.01)
(52) **U.S. Cl.**
CPC **H04L 9/3247** (2013.01); **H04L 9/0825**
(2013.01); **H04L 9/3236** (2013.01)

(57) **ABSTRACT**

A method and system for generating and verifying a monotone policy aggregate signature of a message. The method involves storing a common message at a local storage device, generating a signature for the common message using a secret cryptographic key at a local signature generation module, and transmitting the user signature to an aggregator module. The aggregator module combines the user signature with other user signatures to generate an aggregate signature. The aggregate signature is then transmitted to a verification module for verification. The monotone policy aggregate signatures are constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies.



Setup(1^λ):

1. Generate $hk \leftarrow \text{Gen}_{\text{HT}}(1^\lambda)$.
2. Generate $(\text{crs}_{\text{BARG}}, \text{td}_{\text{BARG}}) \leftarrow \text{Setup}_{\text{BARG}}(1^\lambda)$.
3. Output $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.

KeyAgg($\text{crs}, \{vk_i\}_{i \in [k]}$):

1. Parse $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.
2. Output $\widehat{vk} = \text{Hash}_{\text{HT}}(hk, (vk_1, \dots, vk_k))$.

SigAgg($\text{crs}, f, \{vk_i, \sigma_i\}_{i \in [k]}, m$):

1. Parse $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.
2. Compute $\widehat{vk} = \text{Hash}_{\text{HT}}(hk, (vk_1, \dots, vk_k))$.
3. Let $z = (hk, \widehat{vk}, m)$, and define the Turing machine $M(z, j, w_j)$ which operates as follows:
 - Parse $z = (hk, \widehat{vk}, m)$.
 - Parse $w_j = (\rho_j, vk_j, \sigma_j)$.
 - Check that $\text{Verify}_{\text{HT}}(hk, \widehat{vk}, j, vk_j, \rho_j) = 1$.
 - Check that $\text{Verify}(vk_j, m, \sigma_j) = 1$.

Let $T = \text{poly}(\lambda)$ so that the above pseudocode terminates.

4. For every $j \in [k]$, construct a witness w_j such that $M(z, j, w_j) = 1$, using the Open_{HT} algorithm to produce the appropriate openings. Let $w = (w_1, \dots, w_k)$.
5. Output $\widehat{\sigma} = \mathcal{P}_{\text{BARG}}(\text{crs}_{\text{BARG}}, f, M, z, 1^T, w)$.

AggVerify($\text{crs}, f, \widehat{vk}, m, \widehat{\sigma}$):

1. Parse $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.
2. Define M, T as above and $z = (hk, \widehat{vk}, m)$, and let $x = (f, M, z, T)$.
3. Output $\mathcal{V}_{\text{BARG}}(\text{crs}_{\text{BARG}}, x, \widehat{\sigma})$.

Fig. 1

Setup(1^λ):

1. Generate $hk \leftarrow \text{Gen}_{\text{HT}}(1^\lambda)$.
2. Let g be a null function (we can use any arbitrary function).
3. Generate $(\text{crs}_{\text{BARG}}, \text{td}_{\text{BARG}}) \leftarrow \text{Setup}_{\text{BARG}}(1^\lambda, g)$.
4. Output $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.

KeyAgg($\text{crs}, \{vk_i\}_{i \in [k]}$):

1. Parse $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.
2. Output $\widehat{vk} = \text{Hash}_{\text{HT}}(hk, (vk_1, \dots, vk_k))$.

SigAgg($\text{crs}, f, \{vk_i, \sigma_i\}_{i \in [k]}, m$):

1. Parse $\text{crs} = (hk, \text{crs}_{\text{BARG}})$.
2. Compute $\widehat{vk} = \text{Hash}_{\text{HT}}(hk, (vk_1, \dots, vk_k))$.
3. Let $z = (hk, \widehat{vk}, m)$, and define the Turing machine $M(z, j, w_j)$ which operates as follows:
 - Parse $z = (hk, \widehat{vk}, m)$.
 - Parse $w_j = (\rho_j, vk_j, \sigma_j)$.
 - Check that $\text{Verify}_{\text{HT}}(hk, \widehat{vk}, j, vk_j, \rho_j) = 1$.
 - Check that $\text{Verify}(vk_j, m, \sigma_j) = 1$.
4. For every $j \in [k]$, construct a witness w_j for x , using the Open_{HT} algorithm to produce the appropriate openings. Let $w = (w_1, \dots, w_k)$.
5. Let $y = (vk_1, \dots, vk_k)$.
6. Output $\widehat{\sigma} = \mathcal{P}_{\text{BARG}}(\text{crs}_{\text{BARG}}, y, f, M, z, 1^T, w)$.

AggVerify($\text{crs}_V, f, \widehat{vk}, m, \widehat{\sigma}$):

1. Parse $\text{crs}_V = (hk, \text{crs}_{\text{BARG}})$.
2. Define M, T as above and $z = (hk, \widehat{vk}, m)$, and let $x = (f, M, z, T)$.
3. Output $\mathcal{V}_{\text{BARG}}(\text{crs}_{\text{BARG}}, \widehat{vk}, x, \widehat{\sigma})$.

Fig. 2

Query($1^\lambda, t$):

1. Generate $hk_{HT} \leftarrow \text{Gen}_{HT}(1^\lambda)$.
2. Generate $(crs_{seBARG}, td_{seBARG}) \leftarrow \text{Gen}_{seBARG}(1^\lambda, I)$, where $I = \{1, t\}$.
3. Output $dk = td_{seBARG}$, $vk = q = (hk_{HT}, crs_{seBARG})$.

Answer($D, 1^T, \Gamma, X, q$):

1. Parse $q = (hk_{HT}, crs_{seBARG})$.
2. Parse $D = (r_i)_{i \in [k]} \in \{0, 1\}^{k \times m}$ and $X = (x_i)_{i \in [k]} \in \{0, 1\}^{k \times n}$.
3. Let c_0 be the starting configuration, and for every $1 \leq i \leq k$ compute $c_i = \Gamma(c_{i-1}, x_i, r_i)$.
4. Compute $rt = \text{Hash}_{HT}(hk_{HT}, (c_0, \dots, c_k))$.
5. Define an instance $Y = (M, z, k, T')$ of BatchIndexTMSAT. The input z is defined as $z = (X, hk_{HT}, rt)$. The batch size is set to k . The Turing machine $M(z, j, w_j)$ is defined to operate as follows:
 - (a) Parse $z = (X, hk_{HT}, rt)$, and $X = (x_i)_{i \in [k]}$.
 - (b) Parse $w_j = (r_j, c_{j-1}, c_j, \rho_{j-1}, \rho_j)$, where $m' = |w_j|$.
 - (c) If $j = 1$, check that c_0 is the starting configuration. If $j = k$, check that c_k is the accepting configuration.
 - (d) Check that $\text{Verify}_{HT}(hk_{HT}, rt, j-1, c_{j-1}, \rho_{j-1}) = 1$ and $\text{Verify}_{HT}(hk_{HT}, rt, j, c_j, \rho_j) = 1$.
 - (e) Check that $\Gamma(c_{j-1}, x_j, r_j) = c_j$.

The description length of M is a constant. Finally, the time bound $T' = \text{poly}(n, k, w, T, N, S)$ is set so that the pseudocode above terminates.

6. For every $j \in [k]$, construct a witness (j, w_j) for Y , using the database $D = (r_i)_{i \in [k]}$, the computed configurations $c_0, \dots, c_k$, and the Open_{HT} algorithm to produce the appropriate openings.
7. Compute $\pi_{seBARG} = \mathcal{P}_{seBARG}(crs_{seBARG}, M, z, 1^{T'}, w_1, \dots, w_k)$.
8. Output $a = (rt, \pi_{seBARG})$.

Dec(dk, a):

1. Parse $dk = td_{seBARG}$ and $a = (rt, \pi_{seBARG})$.
2. Extract $w_1, w_t = \text{Extract}_{seBARG}(td_{seBARG}, \pi_{seBARG})$.
3. Parse $w_t = (r_t, c_{t-1}, c_t, \rho_{t-1}, \rho_t)$.
4. Output r_t .

Verify(vk, Γ, X, a):

1. Parse $vk = (hk_{HT}, crs_{seBARG})$ and $a = (rt, \pi_{seBARG})$.
2. Define $Y = (M, z, k, T')$ as in Answer.
3. Output $\mathcal{V}_{seBARG}(crs_{seBARG}, Y, \pi_{seBARG})$.

Fig. 3

Setup(1^λ):

1. For every $j \in [\lambda]$, sample $i_j \leftarrow [2^j]$ uniformly at random.
2. For every $j \in [\lambda]$, generate $(dk_j, vk_j, q_j) \leftarrow \text{Query}(1^\lambda, i_j)$.
3. Output $\text{crs} = (q_j, vk_j)_{j \in [\lambda]}$, $\text{td} = (i_j, dk_j)_{j \in [\lambda]}$.

$\mathcal{P}(\text{crs}, f, M, z, 1^T, w_1, \dots, w_k)$:

1. Parse $\text{crs} = (q_j, vk_j)_{j \in [\lambda]}$.
2. Let $(q, vk) = (q_j, vk_j)$ for $j = \lceil \log k \rceil$ (where recall that $f: \{0, 1\}^k \rightarrow \{0, 1\}$).
3. Let $\Gamma_{f,T}$ be the policy-checking constraint defined by Γ_f and T : given c_{t-1}, x_t, r_t as input, $\Gamma_{f,T}$ does the following:
 - (a) Parse $x_t = (M, z, t)$ and $r_t = w_t$.
 - (b) Let $b_t = 1$ if $M(z, t, w_t)$ accepts within T steps, and $b_t = 0$ otherwise.
 - (c) Output $c_t = \Gamma_f(c_{t-1}, b_t)$.
4. Let $D = (w_t)_{t \in [k]}$.
5. Output $\pi = \text{Answer}(D, \Gamma_{f,T}, (M, z), q)$.

$\mathcal{V}(\text{crs}, x, \pi)$:

1. Parse $\text{crs} = (q_j, vk_j)_{j \in [\lambda]}$, $x = (f, M, z, T)$ and $\pi = a$.
2. Let $(q, vk) = (q_j, vk_j)$ for $j = \lceil \log k \rceil$.
3. Let $\Gamma_{f,T}$ as in $\mathcal{P}$.
4. Output $\text{Verify}(vk, \Gamma_{f,T}, (M, z), a)$.

Extract(td, π):

1. Parse $\text{td} = (i_j, dk_j)_{j \in [\lambda]}$ and $\pi = a$.
2. Let $(i, dk) = (i_j, dk_j)$ for $j = \lceil \log k \rceil$.
3. Extract $w_t = \text{Dec}(dk, a)$.
4. Output (i, w_t) .

Fig. 4

$\text{Setup}(1^\lambda)$ does the following:

1. For every $j \in [\lambda]$, sample $i_j \leftarrow [2^j]$ uniformly at random.
2. For every $j \in [\lambda]$, generate $(\text{dk}_j, \text{vk}_j, q_j) \leftarrow \text{Query}(1^\lambda, i_j)$.
3. Output $\text{crs} = (q_j, \text{vk}_j)_{j \in [\lambda]}$, $\text{td} = (\text{dk}_j)_{j \in [\lambda]}$.

$\mathcal{P}(\text{crs}, f, M, z, 1^T, \{i_j, w_j\}_{j \in [t]})$ does the following:

1. Parse $\text{crs} = (q_j, \text{vk}_j)_{j \in [\lambda]}$.
2. Suppose that f is a weighted threshold function with k inputs, weights $\{\alpha_i\}_{i \in [k]}$ and threshold t . We define the policy-checking constraint $\Gamma_{f,T}$ which does the following given c_{j-1}, x_j, r_j as input:
 - (a) Parse the state c_{j-1} . The state is either $c_{j-1} = \perp$, or the starting state $c_{j-1} = c_0$, or a tuple $c_{j-1} = (i, c)$ where $i \in [k]$ is an index, and $c \in [\alpha_i]$ is a counter.
 - (b) Parse the instance $x_j = (M, z, j)$ and the row $r_j = (i', w_r)$.
 - (c) Define the next state c_j via one of the following cases:
 - i. If $c_{j-1} = c_0$ and $1 \leq i' \leq k$ and $M(z, i', w_r)$ accepts within T steps, set $c_j = (i', 1)$.
 - ii. If $1 \leq c < \alpha_i$ and $i = i'$, set $c_j = (i, c + 1)$.
 - iii. If $c = \alpha_i$ and $i < i' \leq k$ and $M(z, i', w_r)$ accepts within T steps, set $c_j = (i', 1)$.
 - iv. Otherwise, set $c_j = \perp$.
 - (d) If $j = t$ and $c_j \neq \perp$, output the accepting state. Otherwise, output c_j .
3. Let $(q, \text{vk}) = (q_j, \text{vk}_j)$ for $j = \lceil \log t \rceil$.
4. Let $D = (i_j, w_j)_{j \in [t]}$ (we assume i_j are sorted).
5. Output $\pi = \text{Answer}(D, \Gamma_{f,T}, (M, z), q)$.

$\mathcal{V}(\text{crs}, x, \pi)$ does the following:

1. Parse $\text{crs} = (q_j, \text{vk}_j)_{j \in [\lambda]}$, $x = (f, M, z, T)$ and $\pi = a$.
2. Let $(q, \text{vk}) = (q_j, \text{vk}_j)$ for $j = \lceil \log t \rceil$.
3. Let $\Gamma_{f,T}$ as in $\mathcal{P}$.
4. Output $\text{Verify}(\text{vk}, \Gamma_{f,T}, (M, z), a)$.

$\text{Extract}(\text{td}, \pi)$ does the following:

1. Parse $\text{td} = (\text{dk}_j)_{j \in [\lambda]}$ and $\pi = a$.
2. Let $\text{dk} = \text{dk}_j$ for $j = \lceil \log t \rceil$.
3. Output $(i_j, w_j) = \text{Dec}(\text{dk}, a)$.

Fig. 5

$\text{Gen}(1^\lambda, g)$ does the following:

1. Generate $(\text{hk}_{\text{PEHT}}, \text{vk}_{\text{PEHT}}, \text{td}_{\text{PEHT}}) \leftarrow \text{Gen}_{\text{PEHT}}(1^\lambda, g')$, for $g' \in G'_\lambda$ defined above.
2. For each $\beta \in \{1, 2\}$, generate $(\text{hk}_{\text{PEHT}}^{(\beta)}, \text{vk}_{\text{PEHT}}^{(\beta)}, \text{td}_{\text{PEHT}}^{(\beta)}) \leftarrow \text{Gen}_{\text{PEHT}}(1^\lambda, g''_\beta)$, where g''_β is an arbitrary function in G''_λ .
3. Generate $(\text{crs}_{\text{seBARG}}, \text{td}_{\text{seBARG}}) \leftarrow \text{Gen}_{\text{seBARG}}(1^\lambda, I)$ where $I \subseteq [N]$ is initialized to $\{1, 2, N\}$.
4. Let $\text{crs} = (\text{hk}_{\text{PEHT}}, \text{vk}_{\text{PEHT}}, \text{hk}_{\text{PEHT}}^{(1)}, \text{vk}_{\text{PEHT}}^{(1)}, \text{hk}_{\text{PEHT}}^{(2)}, \text{vk}_{\text{PEHT}}^{(2)}, \text{crs}_{\text{seBARG}})$.
5. Output $(\text{crs}, \text{td} = \text{td}_{\text{PEHT}})$.

$\mathcal{P}(\text{crs}, y, C, M, z, 1^T, w_1, \dots, w_k)$ does the following:

1. Let $(b_1, \dots, b_k) \in \{0, 1\}^k$ such that $b_i = 1$ iff $M(z, i, w_i)$ accepts within T steps.
2. Compute the values $(b_1, \dots, b_N)$ of all the wires in the circuit C on input $(b_1, \dots, b_k)$.
3. Parse $\text{crs} = (\text{hk}_{\text{PEHT}}, \text{vk}_{\text{PEHT}}, \text{hk}_{\text{PEHT}}^{(1)}, \text{vk}_{\text{PEHT}}^{(1)}, \text{hk}_{\text{PEHT}}^{(2)}, \text{vk}_{\text{PEHT}}^{(2)}, \text{crs}_{\text{seBARG}})$.
4. Compute $\text{v}_{\text{PEHT}} = \text{Hash}_{\text{PEHT}}(\text{hk}_{\text{PEHT}}, y, (b_1, \dots, b_k), (w_1, \dots, w_k))$.
5. Compute $\text{v}^{(\beta)} = \text{Hash}_{\text{PEHT}}(\text{hk}_{\text{PEHT}}^{(\beta)}, (C, y), (b_1, \dots, b_N))$ for $\beta \in \{1, 2\}$.
6. Define an instance $X = (M', z', N, T)$ of BatchIndexTMSAT. The input z' is defined as $z' = (C, x_1, \dots, x_k, \text{vk}_{\text{PEHT}}, \text{v}_{\text{PEHT}}, (\text{vk}_{\text{PEHT}}^{(\beta)}, \text{v}^{(\beta)})_{\beta \in \{1, 2\}})$. The batch size is set to N . The Turing machine $M'(z', j, w'_j)$ is defined to operate as follows:
 - (a) Parse $z' = (C, M, z, T, \text{vk}_{\text{PEHT}}, \text{v}_{\text{PEHT}}, (\text{vk}_{\text{PEHT}}^{(\beta)}, \text{v}^{(\beta)})_{\beta \in \{1, 2\}})$.
 - (b) If $1 \leq j \leq k$:
 - i. Parse $w'_j = (w_j, b_j, \rho_{\text{PEHT}}, \rho^{(1)}, \rho^{(2)})$.
 - ii. Check that $\text{Verify}_{\text{PEHT}}(\text{vk}_{\text{PEHT}}, y, \text{v}_{\text{PEHT}}, j, b_j, w_j, \rho_{\text{PEHT}}) = 1$.
 - iii. Check that $\text{Verify}_{\text{PEHT}}(\text{vk}_{\text{PEHT}}^{(\beta)}, (C, y), \text{v}^{(\beta)}, j, b_j, \rho^{(\beta)}) = 1$ for $\beta \in \{1, 2\}$.
 - iv. Check that $b_j = 1$ iff $M(z, j, w_j)$ accepts within T steps.
 - (c) If $j > k$:
 - i. Compute the j th gate of C , $g_j = (j, i_1, i_2, c \in \{\text{AND}, \text{OR}\})$.
 - ii. Parse $w'_j = (b_{i_1}, b_{i_2}, b_{i_3}, (\rho_{i_1}^{(\beta)}, \rho_{i_2}^{(\beta)}, \rho_{i_3}^{(\beta)})_{\beta \in \{1, 2\}})$.
 - iii. Check that $\text{Verify}_{\text{PEHT}}(\text{vk}_{\text{PEHT}}^{(\beta)}, (C, y), \text{v}^{(\beta)}, j, b_j, \rho_j^{(\beta)}) = 1$ for $\beta \in \{1, 2\}$.
 - iv. Check that $\text{Verify}_{\text{PEHT}}(\text{vk}_{\text{PEHT}}^{(\beta)}, (C, y), \text{v}^{(\beta)}, j, b_j, \rho_{i_\alpha}^{(\beta)}) = 1$ for $\alpha, \beta \in \{1, 2\}$.
 - v. Check that $b_j = c(b_{i_1}, b_{i_2})$. (That is, check that the gate is satisfied.)
 - vi. If $j = N$ is the output wire then check that $b_j = 1$.

The description length of M is a constant. Finally, the time bound T' is set so that the pseudocode above terminates.

7. For every $j \in [N]$, construct a witness (j, w'_j) for X , using the $\text{Open}_{\text{PEHT}}, \text{Open}_{\text{PEHT}}$ algorithms to produce openings for $(b_1, \dots, b_N)$ and $(b_1, \dots, b_k)$ with tags $(w_1, \dots, w_k)$ as appropriate.
8. Compute $\pi_{\text{seBARG}} = \mathcal{P}_{\text{seBARG}}(\text{crs}_{\text{seBARG}}, M', z', 1^{T'}, w'_1, \dots, w'_N)$.
9. Output $\pi = (\text{v}_{\text{PEHT}}, \text{v}^{(1)}, \text{v}^{(2)}, \pi_{\text{seBARG}})$. 65

$\mathcal{V}(\text{crs}, y, x, \pi)$ does the following:

1. Parse $\text{crs}_{\mathcal{V}} = (\text{vk}_{\text{PEHT}}, \text{vk}_{\text{PEHT}}^{(1)}, \text{vk}_{\text{PEHT}}^{(2)}, \text{crs}_{\text{seBARG}})$.
2. Parse $x = (C, M, z, T)$.
3. Parse $\pi = (\text{v}_{\text{PEHT}}, \text{v}^{(1)}, \text{v}^{(2)}, \pi_{\text{seBARG}})$.
4. Define $X = (M', z', N, T')$ as above.
5. Output $\mathcal{V}_{\text{seBARG}}(\text{crs}_{\text{seBARG}}, X, \pi_{\text{seBARG}})$.

$\text{Extract}(\text{td}, y, \pi)$ does the following:

1. Parse $\pi = (\text{v}_{\text{PEHT}}, \text{v}^{(1)}, \text{v}^{(2)}, \pi_{\text{seBARG}})$.
2. Compute $j = \text{ExtractIndex}_{\text{PEHT}}(\text{td}, y, \text{v}_{\text{PEHT}})$, and $w_j = \text{ExtractTag}(\text{td}, y, \text{v}_{\text{PEHT}})$.
3. Output (j, w_j) .

Fig. 6

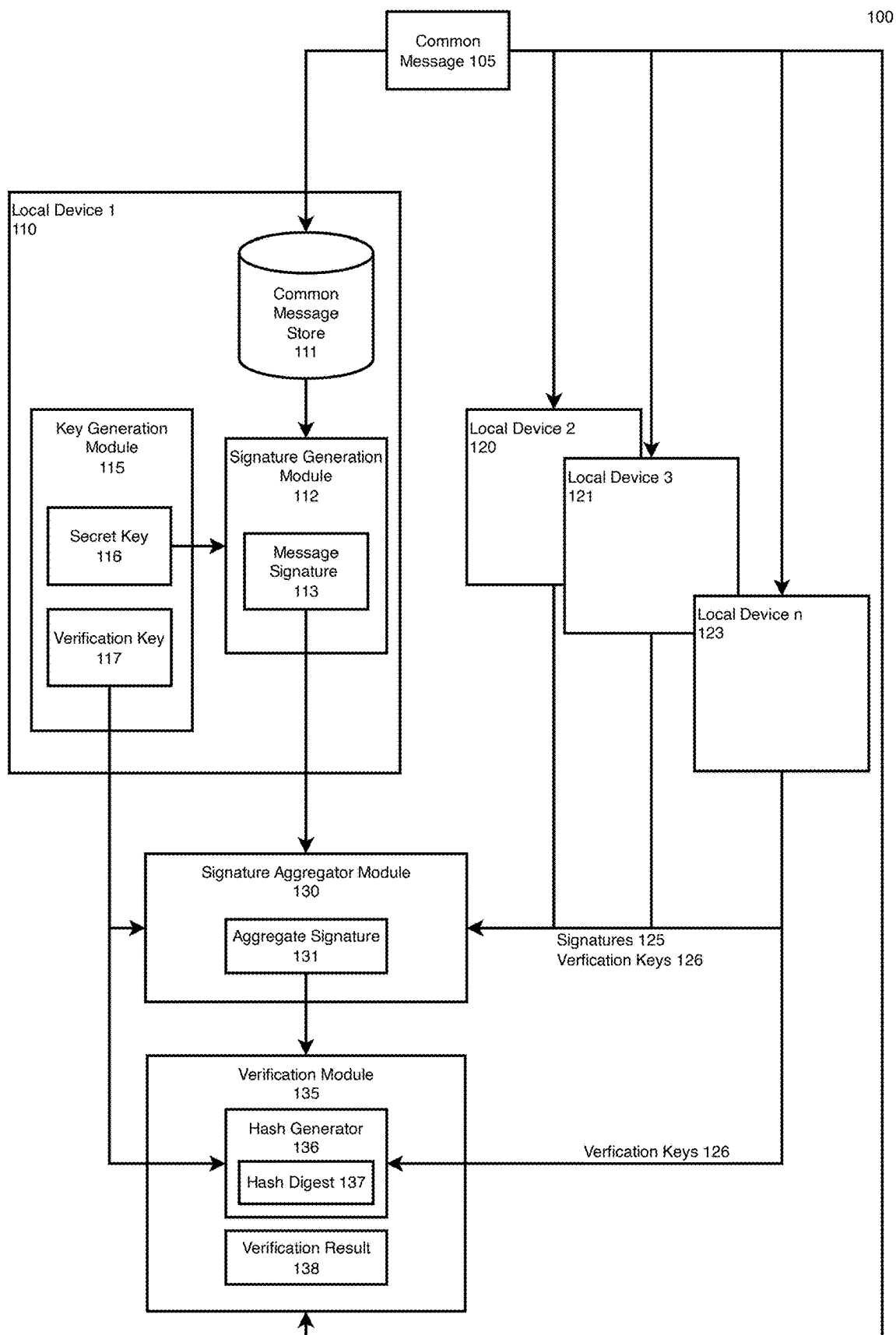


Fig. 7

200

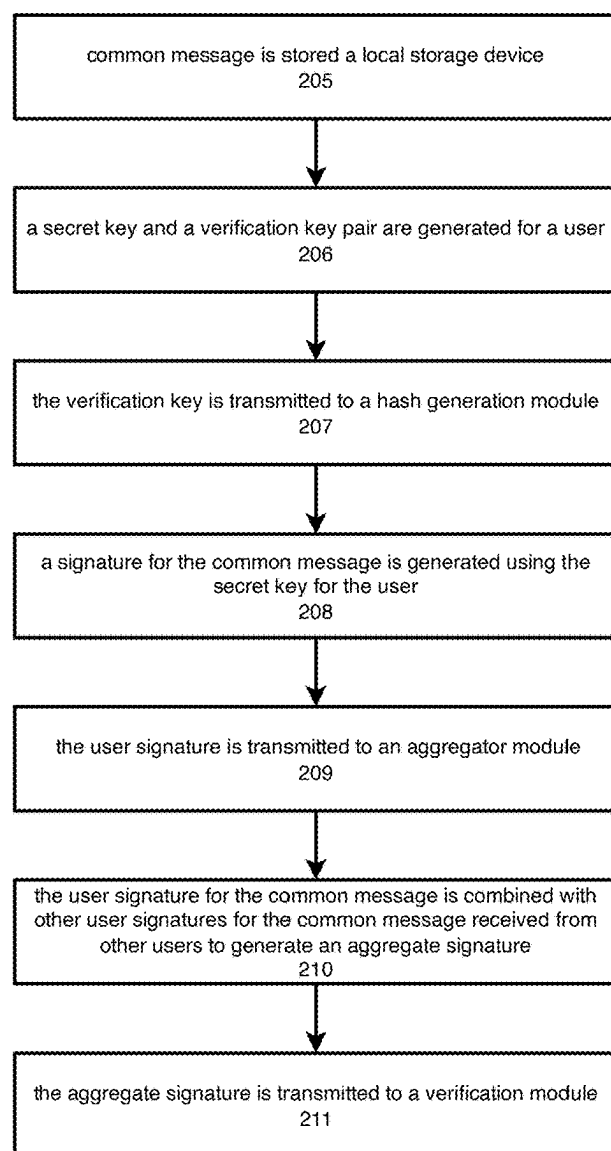


Fig. 8

300

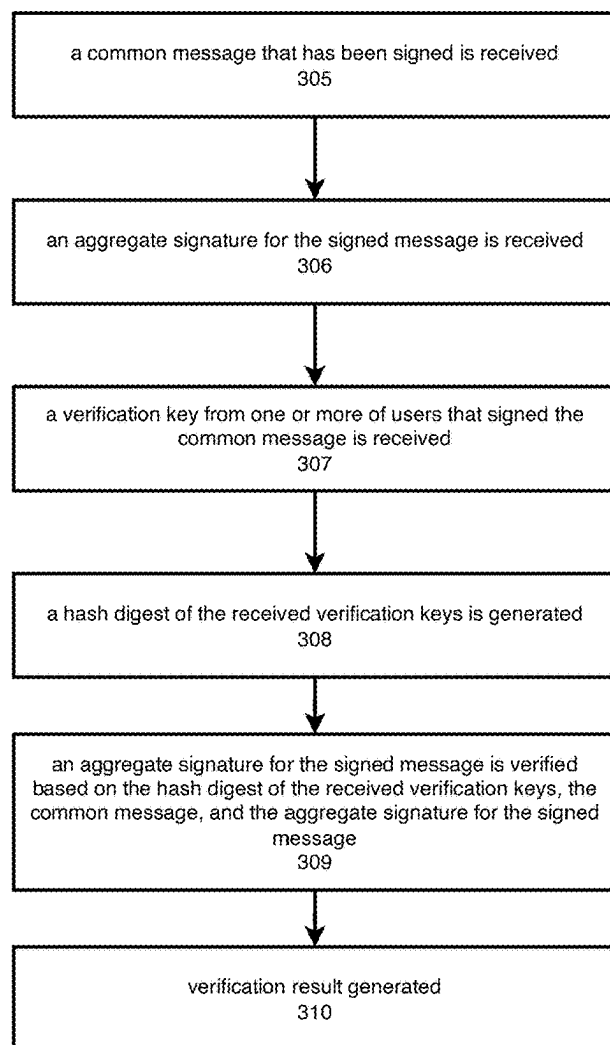


Fig. 9

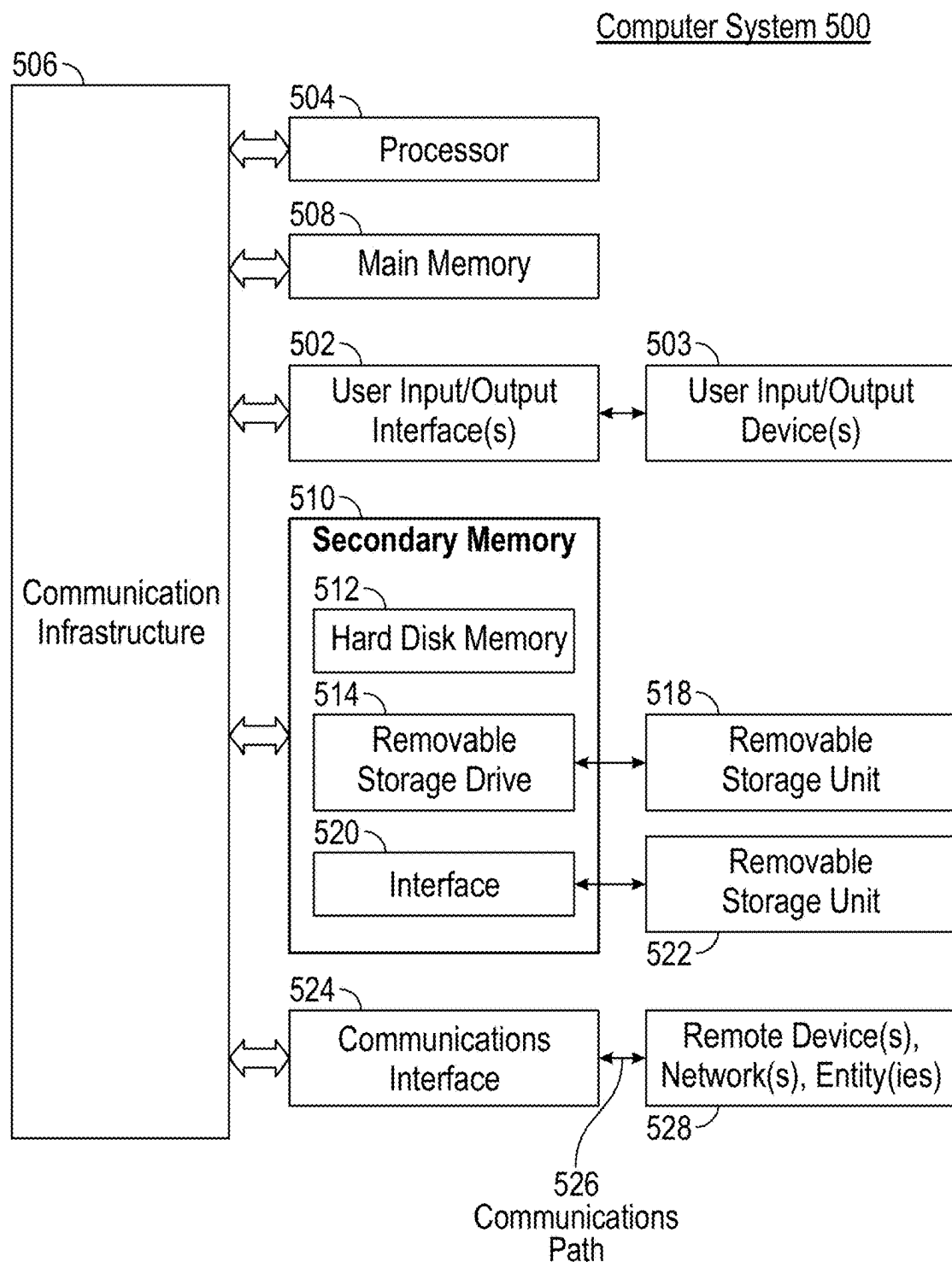


Fig. 10

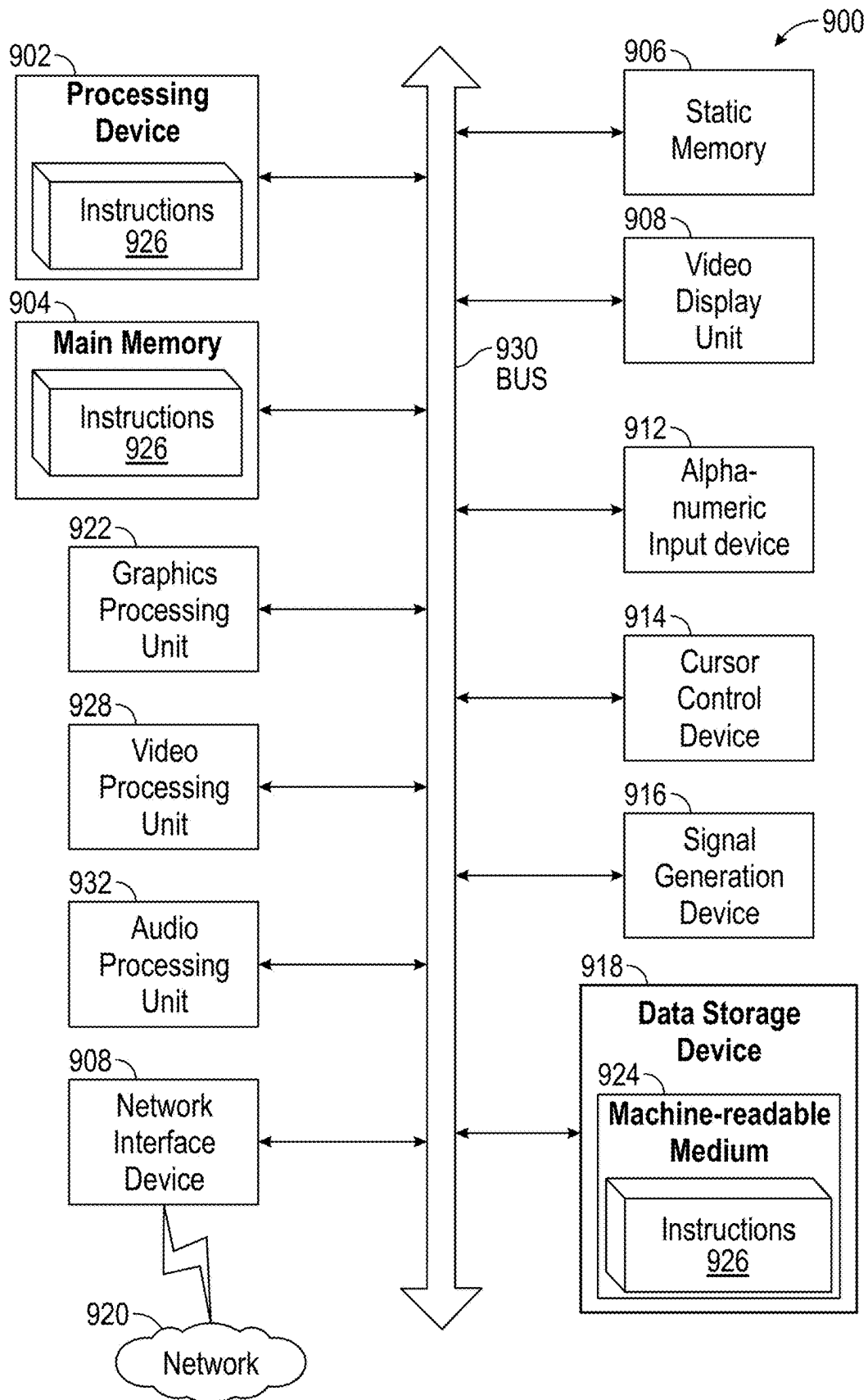


Fig. 11

FLEXIBLE ACCESS CONTROL USING EXPRESSIVE MONOTONE-POLICY AGGREGATE SIGNATURES

CROSS-REFERENCE TO RELATED APPLICATION

[0001] This application claims the benefit of U.S. Provisional Application Ser. No. 63/563,916 filed Mar. 11, 2024, the content of which is incorporated by reference herein in its entirety.

FIELD OF INVENTION

[0002] The present disclosure generally relates to the field of cryptographic signature schemes, and more specifically, to systems and methods for generating and verifying aggregate signatures for expressive monotone policies.

BACKGROUND

[0003] In the domain of digital security, the concept of expressive monotone policies over signatures has gained prominence as a means to enhance the flexibility and specificity of access control mechanisms. These policies enable fine-grained control over the authentication process by defining conditions under which a collective set of signatures is considered valid. Expressive monotone policies are particularly useful in scenarios where multiple signatories are involved, and the validity of a document or transaction depends on a complex combination of their approvals.

[0004] Expressive monotone policies can include a variety of logical constructs such as “AND,” “OR,” and threshold conditions, allowing for intricate policy definitions. For instance, a policy might require that a message be signed by at least three out of five designated authorities, or by one department head in conjunction with two staff members. The expressiveness of these policies, however, introduces several challenges in their implementation and enforcement.

[0005] A primary issue with expressive monotone policies is the computational complexity associated with verifying signatures against intricate policy conditions. As the number of signatories and the complexity of the policy increase, the verification process can become computationally intensive, potentially impacting system performance and scalability. This is particularly problematic in distributed systems, where resources may be limited and efficiency is of the essence.

[0006] Another challenge is the potential for ambiguity in policy interpretation, which can lead to inconsistencies in signature verification. Ensuring that all parties involved in the signature process have a consistent understanding and implementation of the policy is paramount to maintaining the integrity of the system.

[0007] Additionally, the dynamic nature of organizational structures and roles can complicate the management of expressive monotone policies. Changes in personnel or organizational hierarchy may necessitate frequent updates to the policies, which can be cumbersome and error-prone. This fluidity requires robust policy management tools that can adapt to changes without compromising security or introducing vulnerabilities.

[0008] Furthermore, the enforcement of expressive monotone policies often requires sophisticated cryptographic techniques that can support complex logical conditions while preserving the privacy and anonymity of the signato-

ries. This is especially challenging in the context of blockchain networks and other decentralized systems, where transparency and immutability are core principles.

[0009] Despite these challenges, the implementation of expressive monotone policies over signatures remains a sought-after feature in modern cryptographic systems. The ability to specify and enforce complex signing requirements is integral to the security and functionality of various digital platforms, from corporate document management systems to blockchain-based smart contracts. As such, there is a continuous pursuit of advanced cryptographic solutions that can efficiently and securely handle expressive monotone policies.

SUMMARY OF INVENTION

[0010] This summary is provided to introduce a selection of concepts in a simplified form that are further described below in the detailed description. This summary is not intended to identify key features or essential features of the claimed subject matter, nor is it intended to be used as an aid in determining the scope of the claimed subject matter.

[0011] According to an aspect of the present disclosure, a method for generating an aggregate signature of a message is provided. The method includes storing a common message at a local storage device.

[0012] At a local key generation module, a secret cryptographic signing key and a public cryptographic verification key pair for a user are generated using a collision-resistant hash function, and the public cryptographic verification key is transmitted to a hash generation module. At a local signature generation module, a user signature for the common message is generated using the secret cryptographic signing key for the user. The user signature is transmitted to an aggregator module. At the aggregator module, the user signature for the common message is combined with other user signatures for the common message received from other users to generate an aggregate signature using non-interactive batch arguments. The aggregate signature is transmitted to a verification module. The aggregate signature is constructed from the non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies. A size of the aggregate signature is sublinear in a number of signers and is based on a security parameter λ . The non-interactive batch arguments are constructed from verifiable private information retrieval. The aggregate signature is capable of being verified in sublinear time $O(\log k)$ when all parties sign the common message, where k is the number of signers.

[0013] According to other aspects of the present disclosure, the method may include one or more of the following features. The non-interactive batch arguments that support adaptive subset extraction for monotone policies may be constructed from Verifiable Private Information Retrieval.

[0014] An untrusted aggregator party may combine the signatures of multiple parties into an aggregate signature $\hat{\sigma}$ to certify expressive monotone policies over the signatures. At least t out of k parties may have signed a message. The method may further include weighted thresholds where each party has a different weight and a signing policy is defined with respect to a sum of the party weights. The method may rely only on standard cryptographic assumptions. The constructions may be in the common reference string (CRS) model where the size of the CRS grows only polylogarithmically in the number of signers. The size of the aggregate

signature may be sublinear in k and t and only depend on a security parameter. If all parties sign the same message, the verification time may be sublinear in t . If a vector of messages has a succinct representation, the verification time may be sublinear in t . The aggregation time may grow with a number of signatures t and not a number of parties k .

[0015] According to another aspect of the present disclosure, a method for verifying an aggregate signature of a message is provided. The method includes storing a common message at a local storage device. At a local key generation module, a secret key and a verification key pair for a user are generated, and the verification key is transmitted to a hash generation module. At a local signature generation module, a user signature for the common message is generated using the secret key for the user. The user signature is transmitted to an aggregator module. At the aggregator module, the user signature for the common message is combined with other user signatures for the common message received from other users to generate an aggregate signature.

[0016] The aggregate signature is transmitted to a verification module. At the verification module, the common message that has been signed by multiple users is received. The aggregate signature for the signed common message is received. Verification keys from one or more of the multiple users that signed the common message are received. A hash digest of the received verification keys is generated. The aggregate signature for the signed common message is verified based on the hash digest of the received verification keys, the common message, and the aggregate signature for the signed common message. A result of the aggregate signature verification is stored. The aggregate signatures are constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies.

[0017] According to other aspects of the present disclosure, the verification may be performed without knowing the identity of the signers.

[0018] According to another aspect of the present disclosure, an apparatus for generating an aggregate signature of a message is provided. The apparatus includes a local storage device configured to store a common message. A local key generation module is configured to generate a secret key and a verification key pair for a user, and transmit the verification key to a hash generation module. A local signature generation module is configured to generate a user signature for the common message using the secret key for the user. A transmission module is configured to transmit the user signature to an aggregator module. The aggregator module is configured to combine the user signature for the common message with other user signatures for the common message received from other users to generate an aggregate signature. The transmission module is further configured to transmit the aggregate signature to a verification module. The aggregate signature is constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies. A size of the aggregate signature is sublinear in a number of signers and is based on a security parameter λ . The non-interactive batch arguments are constructed from verifiable private information retrieval. The aggregate signature is capable of being verified in sublinear time $O(\log k)$ when all parties sign the common message, where k is the number of signers.

[0019] According to other aspects of the present disclosure, the apparatus may include one or more of the following features. The non-interactive batch arguments that support adaptive subset extraction for monotone policies may be constructed from Verifiable Private Information Retrieval. An untrusted aggregator party may combine the signatures of multiple parties into an aggregate signature to certify expressive monotone policies over the signatures. At least a threshold number of parties may have signed a message. The apparatus may further comprise a weighted threshold module where each party has a different weight and a signing policy is defined with respect to a sum of the party weights. The method may rely on standard cryptographic assumptions. The constructions may be in the common reference string (CRS) model where the size of the CRS grows polylogarithmically with the number of signers. The size of the aggregate signature may be sublinear in the number of signers and the threshold number of signers and depend on a security parameter. If all parties sign the same message, the verification time may be sublinear in the threshold number of signers. If a vector of messages has a succinct representation, the verification time may be sublinear in the threshold number of signers. The aggregation time may grow with the number of signatures and not the number of parties.

[0020] According to another aspect of the present disclosure, an apparatus for verifying an aggregate signature of a message is provided. The apparatus includes a local storage device configured to receive a common message that has been signed by multiple users. A first reception module is configured to receive an aggregate signature for the signed common message. A second reception module is configured to receive verification cryptographic keys from one or more of the multiple users that signed the common message. A hash generation module is configured to generate a hash digest of the received verification cryptographic keys. A verification module is configured to verify the aggregate signature for the signed common message based on the hash digest of the received verification cryptographic keys, the common message, and the aggregate signature for the signed common message. A storage module is configured to store a result of the aggregate signature verification. The aggregate signatures are constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies.

[0021] According to other aspects of the present disclosure, the verification may be performed without knowing the identity of the signers.

[0022] The notion of aggregate signatures allows for combining signatures from different parties into a short certificate that attests that all parties signed a message. In this work, we lift this notion to capture different, more expressive signing policies. For example, we can certify that a message was signed by a (weighted) threshold of signers.

[0023] We present the first constructions of aggregate signatures for monotone policies based on standard polynomial-time cryptographic assumptions. The size of the signatures and the verifier time in our schemes is independent of the number of signers. All prior work requires either interaction between the parties or non-standard assumptions (that imply SNARKs for NP).

[0024] Our signature schemes are based on non-interactive batch arguments (BARGs) for monotone policies [Brakerski-Brodsky-Kalai-Lombardi-Paneth, Crypto'23]. In contrast to previous constructions, our BARGs satisfy a new

notion of adaptive security which is instrumental to our application. Our new BARGs for monotone policies can be constructed from standard BARGs and other standard assumptions.

[0025] The foregoing general description of the illustrative embodiments and the following detailed description thereof are merely exemplary aspects of the teachings of this disclosure and are not restrictive.

[0026] The foregoing general description of the illustrative embodiments and the following detailed description thereof are merely exemplary aspects of the teachings of this disclosure and are not restrictive.

BRIEF DESCRIPTION OF FIGURES

[0027] Non-limiting and non-exhaustive examples are described with reference to the following figures.

[0028] FIG. 1 illustrates example Aggregate Signatures for Bounded Space Monotone Policies f from BARGs with Adaptive Subset Extraction for f .

[0029] FIG. 2 illustrates example Weakly Unforgeable Aggregate Signatures for Monotone Circuits f .

[0030] FIG. 3 illustrates an example construction of 2-Composable vPIR Scheme for Policies.

[0031] FIG. 4 illustrates an example construction of BARGs with Adaptive Subset Extraction for bounded-space policies.

[0032] FIG. 5 illustrates an example construction of Sub-linear Prover BARGs for weighted threshold policies.

[0033] FIG. 6 illustrates an example construction of BARGs with Functional Subset Extraction.

[0034] FIG. 7 presents a block diagram of an aggregate signature generation and verification apparatus, including the process of generating and verifying digital signatures using various cryptographic schemes.

[0035] FIG. 8 depicts a flowchart of a method for generating an aggregate signature, illustrating the step-by-step process involved in the generation of digital signatures.

[0036] FIG. 9 illustrates a flowchart for a method of verifying an aggregate signature, detailing the steps involved in the verification of an aggregate signature.

[0037] FIG. 10 presents a block diagram of a computer system, demonstrating the hardware components involved in the implementation of the cryptographic signature scheme.

[0038] FIG. 11 presents a block diagram of a system architecture for processing and managing data, highlighting the system architecture used for implementing the cryptographic signature scheme.

DETAILED DESCRIPTION

[0039] The following description sets forth exemplary aspects of the present disclosure. It should be recognized, however, that such description is not intended as a limitation on the scope of the present disclosure. Rather, the description also encompasses combinations and modifications to those exemplary aspects described herein.

[0040] The present disclosure relates to the field of cryptography, and more particularly, to methods and systems for generating and verifying aggregate signatures of a message. In some aspects, the disclosure provides methods and systems for generating an aggregate signature of a message, where multiple parties sign a common message, and these individual signatures are combined into a single aggregate

signature. This aggregate signature can then be verified to ensure the authenticity and integrity of the message.

[0041] In some embodiments, the methods and systems disclosed herein can be implemented in various types of computer systems and software tools, including local, cloud-based, or distributed systems. The choice of implementation can depend on various factors, such as the specific requirements of the system, the performance requirements, and the security considerations. The disclosed methods and systems can provide a secure and efficient way to authenticate and verify digital messages, thereby enhancing the security and reliability of data transmission in various applications.

[0042] In some cases, the signature system 100 may implement a method for generating and verifying aggregate signatures for expressive monotone policies. The method may involve several key components and processes, as illustrated in FIGS. 1-6.

[0043] FIG. 1 depicts an example of aggregate signatures for bounded space monotone policies f from batch arguments (BARGs) with adaptive subset extraction for f . The process may begin with a Setup procedure that takes a security parameter 1^λ as input. This procedure may generate a hash key hk using a function $\text{Gen}_{HT}(1^\lambda)$. Additionally, it may generate a common reference string and trapdoor pair $(\text{crs}_{BARG}, \text{td}_{BARG})$ using $\text{Setup}_{BARG}(1^\lambda)$. The output of this setup phase may be a common reference string $\text{crs}=(hk, \text{crs}_{BARG})$.

[0044] The KeyAgg procedure, as shown in FIG. 1, may take the common reference string crs and a set of verification keys $\{vk_i\}_{i \in [k]}$ as input. This procedure may parse the common reference string to extract the hash key hk . It may then compute an aggregate verification key $\hat{vk}=\text{Hash}_{HT}(hk, (vk_1, \dots, vk_k))$ using the hash function.

[0045] The SigAgg procedure illustrated in FIG. 1 may be responsible for generating the aggregate signature. This procedure may take as input the common reference string crs , a monotone policy f , a set of verification key and signature pairs $\{vk_i, \sigma_i\}_{i \in [k]}$, and a message m . The procedure may parse the common reference string, compute the aggregate verification key, and define a parameter $z=(hk, \hat{vk}, m)$. It may also define a Turing machine $M(z, j, w_j)$ with specific operational parameters. The procedure may construct witnesses w_j using an Open_{HT} algorithm and finally output an aggregate signature $\sigma = \mathcal{P}_{BARG}(\text{crs}_{BARG}, f, M, z, 1^\lambda, w)$.

[0046] FIG. 2 presents a similar process for generating weakly unforgeable aggregate signatures for monotone circuits f . The main difference in this figure is the inclusion of a null function g in the Setup procedure, which may be used in the generation of the BARG parameters.

[0047] FIG. 3 illustrates the construction of a 2-composable verifiable private information retrieval (vPIR) scheme for policies. The Query function in this scheme may take inputs $(1^\lambda, t)$ and produce outputs including a decryption key dk , a verification key vk_t , and a query q . The function may generate a hash key hk_{HT} using $\text{Gen}_{HT}(1^\lambda)$ and create a pair of parameters $(\text{crs}_{seBARG}, \text{td}_{seBARG})$ using $\text{Gen}_{seBARG}(1^\lambda, I)$, where I is defined as $\{1, t\}$.

[0048] FIG. 4 depicts the construction of BARGs with adaptive subset extraction for bounded-space policies. The Setup procedure in this construction may sample indices j , uniformly at random from $[2^j]$ for every $j \in [\lambda]$. It may then generate parameters (dk_j, vk_j, q_j) using a Query operation.

The output of this setup may include a common reference string $\text{crs}=(q, \text{vk}_j)_{j \in [\lambda]}$ and a trapdoor $\text{td}=(i, \text{dk}_j)_{j \in [\lambda]}$.

[0049] FIG. 5 shows a construction of sublinear prover BARGs for weighted threshold policies. This construction may be similar to the one in FIG. 4, but with additional steps in the prover procedure to handle weighted threshold functions.

[0050] FIG. 6 illustrates the construction of BARGs with functional subset extraction. This construction may include procedures for generation (Gen), proving (P), verification (V), and extraction (Extract). The Gen procedure may generate parameters for predicate-extractable hash functions and batch arguments. The P procedure may compute bit values, generate hash values, and construct witnesses for verification. The V procedure may verify the cryptographic parameters, while the Extract procedure may process the proof to extract certain values.

[0051] In some cases, these constructions may provide a framework for generating and verifying aggregate signatures that support expressive monotone policies. The use of BARGs with adaptive subset extraction may allow for efficient verification of aggregate signatures while maintaining security against adaptive adversaries.

[0052] FIG. 7 illustrates an example system embodiment of an aggregate signature generation and verification apparatus 100.

[0053] At outset, a common message 105 is created by any means, and the common message can represent any arbitrary data for any arbitrary purpose. It is assumed that multiple local devices, including local device 1 110, local device 2 120, local device 3 121, and local device π 123 all have access to the common message 105.

[0054] At local device 1 110, a common message store 111 may be used to store common message 105.

[0055] Local device 1 110 can have access to a key generation module 115, which can be configured to generate a secret key 116 and verification key 117 pair. The local device 1 110 can have access to a signature generation module 112. Signature generation module 112 can be configured to sign the common message 105 stored in the common message store 111 using the key generation module 115 to generate a message signature 113.

[0056] Other local devices 120, 121, and 123 also generate signatures 125 which are transmitted to signature aggregator module 130. Signature aggregator module 130 can be configured to generate an aggregate signature 131 from the multiple signatures 113 and 125 that it has received. Joint and aggregate and used interchangeably herein.

[0057] At a verification module 135, verification key 117 from local device 1 110 is received, as is verification keys 126 from other local devices 120, 121, and 123. At a hash generator 136, a hash digest 137 is generated based on the received verification keys 117 and 126. The verification module 135 then generates a positive or negative verification result 138 based on the hash digest 137, common message 105, and the aggregate signature 131.

[0058] Some elements may be local, other elements may be remote, or distributed. The apparatus 100 may be implemented in various types of computer systems and software tools, including local, cloud-based, or distributed systems. The choice of implementation can depend on various factors, such as the specific requirements of the system, the performance requirements, and the security considerations.

[0059] In some aspects, the apparatus 100 includes a local device 1 110, which houses a common message store 111 for storing a common message 105. The local device 1 110 also includes a signature generation module 112 and a cryptographic module, which may be a local cryptographic module or a remote cryptographic module. The cryptographic module includes a local cryptographic module 115 that generates a secret cryptographic key 116 and a verification cryptographic key 117 for a user. The verification cryptographic key 117 is transmitted to a signature generation module, which may be a local signature generation module or a remote signature generation module.

[0060] The signature generation module 112 of the local device 1 110 generates a user signature 113 for the common message 105 using the secret cryptographic key 116 for the user. The user signature 113 is then transmitted to an aggregator module 130. The aggregator module 130 may be a local aggregator module or a remote aggregator module. The aggregator module 130 combines the user signature 113 for the common message 105 with other user signatures 125 for the common message 105 received from other users to generate an aggregate signature 131. The aggregate signature 131 is then transmitted to a verification module 135.

[0061] The verification module 135 may be a local verification module or a remote verification module. The verification module 135 utilizes a hash generator 136 to produce a hash digest 137 of verification keys 117 and 126. The verification module 135 produces a verification result 138, ensuring the integrity of the signed common message 105.

[0062] In some cases, the aggregate signatures are constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies. These non-interactive batch arguments may be constructed from Verifiable Private Information Retrieval. In some aspects, an untrusted aggregator party combines the signatures of multiple parties into an aggregate signature to certify expressive monotone policies over the signatures. The constructions may be in the common reference string (CRS) model where the size of the CRS grows polylogarithmically with the number of signers. The size of the aggregate signature may be sublinear in the number of signers and the threshold number of signers and may depend on a security parameter. This approach allows for the aggregation of signatures from different parties into a compact form, while still maintaining the ability to verify the authenticity of the message. This can be particularly beneficial in scenarios where the number of signers is large, as it can reduce the computational and storage requirements associated with signature verification.

[0063] In some embodiments, the verification of the aggregate signature for the signed message is performed without knowing the identity of the signers. Various cryptographic signature schemes, as described herein, may be used in the aggregate signature generation and verification apparatus 100. The implementation of the aggregate signature generation and verification apparatus 100 may be done using different programming languages and libraries, such as Java, C, and Python. The public keys may be stored in a centralized repository, such as a Public Key Infrastructure (PKI), or in a decentralized system, such as a blockchain. A cryptographic hash function, as described herein, may be used in the aggregate signature generation and verification apparatus 100.

[0064] In some embodiments, a signature generation module 112 is used to generate a digital signature by encrypting the hash value of the message using the sender's private cryptographic key. The signed message is then transmitted to the recipient. A verification module 135 is used to verify the signature and a storage module is used to store the result of the aggregate signature verification.

[0065] FIG. 8 illustrates an example embodiment of the aggregate signature generation method 200. A common message is stored in a local storage device in step 205. At a local key generation module, a secret key and a verification key pair are generated for a user in step 206. In step 207, the verification key is transmitted to a hash generation module. In step 208, at a local signature generation module, a signature for the common message is generated using the secret key for the user.

[0066] In step 209, the user signature is transmitted to an aggregator module. In step 210, the user signature for the common message is combined with other user signatures for the common message received from other users to generate an aggregate signature. In step 211, the aggregate signature is transmitted to a verification module.

[0067] The method 200 may be implemented in various types of computer systems and software tools, including local, cloud-based, or distributed systems. The choice of implementation can depend on various factors, such as the specific requirements of the system, the performance requirements, and the security considerations.

[0068] In some aspects, the method 200 begins with a common message being stored in a local storage device during a message storage step 205. The common message may be any type of digital data, such as a text document, an image, a video, or a piece of software code. The common message may be stored in various types of storage devices, such as a hard disk, a solid-state drive, a flash memory, or a cloud storage service.

[0069] Following the message storage step 205, a secret cryptographic key and a verification cryptographic key pair for a user are generated during step 206. The cryptographic keys may be generated using various types of cryptographic algorithms, as described herein. The secret cryptographic key is used for generating digital signatures, while the verification cryptographic key is used for verifying digital signatures.

[0070] After the cryptographic keys are generated, the verification cryptographic key is transmitted to a hash generation module during a 207. The hash generation module may be a software module that is capable of generating cryptographic hash values from input data. The hash generation module may be implemented in various types of computer systems and software tools, including local, cloud-based, or distributed systems.

[0071] Subsequently, a signature for the common message is generated using the secret cryptographic hash function for the user during a signature generation step 208. The signature may be generated using various types of signature generation algorithms, as described herein. The signature is a digital code that is appended to the common message to verify the authenticity and integrity of the message.

[0072] Once the signature is generated, it is transmitted to an aggregator module during a signature transmission step 209. The aggregator module may be a software module that is capable of aggregating multiple signatures into a single aggregate signature. The aggregator module may be imple-

mented in various types of computer systems and software tools, including local, cloud-based, or distributed systems.

[0073] In some cases, the aggregator module combines the user signature for the common message with other user signatures for the common message received from other users to generate a joint signature during an aggregate signature generation step 210. The number of user signatures that are combined may vary depending on the specific requirements of the system. In some cases, at least a threshold number of parties signed a message. In other cases, each party has a different weight and a signing policy is defined with respect to a sum of the party weights. The aggregation time may grow with the number of signatures and not the number of parties.

[0074] Finally, the joint signature is transmitted to a verification module during a verification transmission step 211. The verification module may be a software module that is capable of verifying the authenticity and integrity of the joint signature. The verification module may be implemented in various types of computer systems and software tools, including local, cloud-based, or distributed systems.

[0075] FIG. 9 illustrates an example embodiment of the aggregate signature verification method 300. A common message that has been signed is received at a local storage device in step 305. An aggregate signature for the signed message is received in step 306. A verification key from one or more of users that signed the common message is received in step 307. A hash digest of the received verification keys is generated in step 308. In step 309, an aggregate signature for the signed message is verified based on the hash digest of the received verification keys, the common message, and the aggregate signature for the signed message. In step 310, a verification result is generated and stored.

[0076] A flowchart 300 for a method of verifying an aggregate signature is depicted. The method may be implemented in various types of computer systems and software tools, including local, cloud-based, or distributed systems. The choice of implementation can depend on various factors, such as the specific requirements of the system, the performance requirements, and the security considerations.

[0077] In some aspects, the method begins with a signed message reception 305, which initiates the process by receiving a common message that has been signed by multiple users. The common message may be any type of digital data, such as a text document, an image, a video, or a piece of software code. The common message may be received from various types of sources, such as a local storage device, a remote server, or a network connection.

[0078] Following the signed message reception 305, an aggregate signature reception 306 receives an aggregate signature for the signed message. The aggregate signature may be a digital code that is appended to the common message to verify the authenticity and integrity of the message. The aggregate signature may be received from various types of sources, such as a local storage device, a remote server, or a network connection.

[0079] Subsequently, a verification cryptographic key reception 307 takes place, where a verification cryptographic key from one or more users who signed the common message is received. The verification cryptographic key may be a digital code that is used to verify the authenticity and integrity of the aggregate signature. The verification cryp-

tographic key may be received from various types of sources, such as a local storage device, a remote server, or a network connection.

[0080] After the verification cryptographic key is received, a cryptographic hash function generation 308 generates a hash digest of the received verification cryptographic keys. The hash digest may be generated using various types of hash generation algorithms, as described herein.

[0081] The signature verification process 309 verifies the aggregate signature for the signed message based on the hash digest of the received verification cryptographic keys, the common message, and the aggregate signature for the signed message. The verification process may be performed using various types of verification algorithms, as described herein. In some cases, the verification of the aggregate signature is performed without knowing the identity of the signers.

[0082] Finally, a verification result output 310 generates the verification result, concluding the process. The verification result may be a digital code that indicates whether the aggregate signature is valid or not. The verification result may be stored in a local storage device, transmitted to a remote server, or displayed on a user interface.

[0083] In some embodiments, if all parties sign the same message, the verification time is sublinear in the threshold number of signers. This can improve the efficiency of the verification process, especially when the number of signers is large. In other cases, if a vector of messages has a succinct representation, the verification time is sublinear in the threshold number of signers. This can further improve the efficiency of the verification process, especially when the messages are large or complex.

[0084] A number of implementations are described herein. Nevertheless, it will be understood that various modifications may be made without departing from the spirit and scope of the disclosure. Accordingly, other implementations are within the scope of the following claims.

1 Introduction

[0085] Suppose that a group of parties wish to jointly sign a message m such that anyone can verify that a majority of the parties signed m . Such a scenario is commonplace in applications such as byzantine agreement, crypto wallets and many other settings involving decentralization of trust.

[0086] Consider the following notion of digital signatures that captures several properties desirable for such applications: each of the k parties locally computes and publishes a verification key for its preferred signature scheme and uses the corresponding secret key to sign messages. Later, an untrusted party called the “aggregator” can combine the signatures of multiple parties into a joint signature $\hat{\sigma}$ to certify that at least t out of k parties signed a message. The combined signature $\hat{\sigma}$ can be verified given a digest of all k verification keys and the vector of messages signed (but without knowing the identity of the t signers). The key requirement is succinctness:

[0087] The size of the aggregate signature should be sublinear in k and even t , and ideally only depend on the security parameter.

[0088] If all parties sign the same message (or, more generally, if the vector of messages has some succinct representation), the verification time should also be sublinear in t .

[0089] Ideally, the aggregation time should grow with the number of signatures t as opposed to the number of parties k .

The security requirement is that no polynomial-time adversary that corrupts less than t parties in an adaptive manner should be able to create valid signatures.

[0090] This notion combines the best features of two central notions of multiparty signatures: threshold signatures and aggregate signatures. (In the literature, there are two distinct notions of signature schemes that support signature aggregation: multisignatures, where all parties sign the same message, and aggregate signatures, where parties may sign distinct messages. For simplicity of exposition, we use the terminology of aggregate signatures to refer to both settings.) Threshold signatures, widely used in the blockchain ecosystem, support threshold signing with succinct verification but require interactive protocols for key setup and (in most cases) signing. Aggregate signatures dispense with the necessity of interaction but can only attest that all k parties signed (i.e., they do not support threshold signing). Therefore, the above notion can be viewed as a threshold variant of aggregate signatures or as an ad-hoc variant of threshold signatures. To see the appeal of this notion, consider decentralized autonomous organizations (DAO) where members vote on proposals to make joint decisions on governance of assets. Threshold signatures are a natural cryptographic tool for implementing such voting; however, running an interactive key setup between all members is unrealistic. Our notion eliminates this barrier.

[0091] Monotone-Policy Aggregate Signatures. We can generalize the above notion to capture more expressive signing policies. For example, consider weighted thresholds, where each party has a different weight and the signing policy is defined with respect to the sum of the party weights. Such a policy is useful in proof-of-stake blockchains. Another example is threshold of thresholds capturing users with some hierarchical structure. In general, the signing policy might be described by a monotone function that takes as input a list of signers and decides whether they are authorized. We refer to this primitive as monotone-policy aggregate signatures.

[0092] It is not difficult to see that this primitive can be realized by combining standard signature schemes with (adaptively sound) succinct non-interactive arguments of knowledge (SNARKs) for NP. The aggregate signature on a message m simply consists of a SNARK that proves the existence of valid signatures on m from an authorized set of signers. The succinctness property of the SNARK translates to the succinctness of aggregate signature. Sublinear verification time can be achieved by additionally relying on collision-resistant hash functions to compute verification key digests.

[0093] SNARKs for NP are currently only known based on heuristics or non-standard assumptions. Moreover, SNARK constructions with an explicit knowledge extractor are not known and are subject to strong barriers.

[0094] Signatures from Batch Arguments. Non-interactive batch arguments (BARGs) allow an efficient prover to compute a publicly verifiable proof of the validity of multiple NP statements, with size sublinear in the total witness length. If at least one statement is false, no polynomial-time adversary should be able to compute an accepting proof.

[0095] A recent sequence of works provided constructions of BARGs for NP in the common reference string (CRS)

model from various standard assumptions including learning with errors (LWE), decisional linear assumption (DLIN) over pairing groups, and sub-exponential decisional Diffie Hellman (DDH). Notably, these works achieve a somewhere extraction property that guarantees (given a CRS “trap-door”) efficient extraction of the witness of one statement in the batch from any accepting proof. This guarantee holds in the adaptive setting where the adversary can choose the NP statements as a function of the CRS. Using this property, recent works obtained the first constructions of aggregate signatures from standard assumptions.

[0096] It is easy to see that there is a direct correspondence between the k -out-of- k (i.e., conjunction) policies supported by BARGs and aggregate signatures. In light of this connection, and towards answering the above question, we ask whether there exist batch arguments that support more expressive policies of the following form: given a batch of statements $(x_1, \dots, x_k)$ and a monotone policy $f: \{0, 1\}^k \rightarrow \{0, 1\}$, a valid proof should attest whether $f(b_1, \dots, b_k) = 1$, where b_i indicates the veracity of x_i .

[0097] A very recent work investigates this problem for the setting where the policy f is described by a polynomial-size monotone circuit. They achieve an extraction property that extends the prior notion of somewhere extraction to general monotone policies. However, as we discuss in Section 2, this guarantee turns out to be inadequate for achieving monotone-policy aggregate signatures.

1.1 Our Results

[0098] We present the first constructions of aggregate signatures from standard assumptions that support expressive monotone policies and achieve poly-logarithmic signature size and verification time. Our constructions are in the common reference string (CRS) model where the size of the CRS grows only poly-logarithmically in the number of signers.

[0099] Before we state our results, we elaborate on our notion of aggregate signatures for monotone policies.

[0100] Monotone-Policy Aggregate Signatures. In an aggregate signature scheme for a monotone policy f , each party P_i publishes its own verification key vk_i , and there is a deterministic public algorithm that aggregates the verification keys of all k parties into a single aggregated verification key $\widehat{vk}$. Given a collection of signatures $\{\sigma_i\}_{i \in I}$ on a message m , an aggregator can produce an aggregated signature $\hat{\sigma}$ on m . The signature $\hat{\sigma}$ verifies with respect to $\widehat{vk}$ if $f(b_1, \dots, b_k) = 1$, where $b_j = 1$ for every j such that signature σ_j verifies with respect to vk_j .

[0101] In terms of security, we require that no efficient adversary can win the following game with non-negligible probability. The adversary may ask the challenger for the verification key of any party, and it can ask for a signature on any message under any of these verification keys. These queries can be completely adaptive. Finally, the adversary produces a sequence of verification keys $vk_1, \dots, vk_k$ that may include keys that were not generated by the challenger. Intuitively, we think of the keys generated by the challenger as belonging to honest parties while the other keys come from parties corrupted by the adversary. Together with the verification keys, the adversary also produces a target message m and a signature $\hat{\sigma}$. The adversary wins if $\hat{\sigma}$ verifies with respect to m and the key $\widehat{vk}$ aggregated from $vk_1, \dots, vk_k$, and if the forgery is non-trivial. Intuitively, the forgery

is non-trivial if the set of honest parties that signed m together with all corrupted parties does not satisfy the policy. That is, $f(b_1, \dots, b_k) = 0$ where $b_j = 1$ for every j such that either the adversary asked for signature on m with respect to vk_j , or vk_k was not generated by the challenger.

[0102] Aggregate Signatures for Bounded-Space Monotone Policies. Our first construction supports read-once, bounded-space monotone policies. Such policies are given by a monotone function $f: \{0, 1\}^k \rightarrow \{0, 1\}$ that is computable by an algorithm that reads each bit of the input once, and maintains a state of size S that is updated after reading each bit. This class of monotone policies includes, for example: the t -out-of- k threshold function with state of size $S = \log k$, or the weighted threshold function for weights in $[B]$ with state of size $S = \log kB$.

[0103] Theorem 1.1 (Informal). Assuming the existence of somewhere extractable BARGs with 2^S -security, there exists an aggregate signature scheme for all read-once $O(S)$ space polynomial-time monotone policies. The size of the CRS, the size of the aggregated signature, and the verification time are $\text{poly}(\log k, S, \lambda)$ where λ denotes the security parameter.

[0104] In particular, for $k = \text{poly}(\lambda)$ and $S = \log(k)$ the theorem only requires BARGs with polynomial security.

[0105] Under the same assumptions as in Theorem 1.1, we can also construct aggregate signatures for monotone policies f that are computable by monotone circuits of fan-in two and $\log(\lambda)$ -depth (or more generally, d -depth under 2^d -secure BARGs). This result is incomparable to Theorem 1.1. We refer the reader to Section 9 for further details.

[0106] Fast Aggregator. For threshold policies, our result in Theorem 1.1 can be extended to achieve a more efficient aggregator whose running time only depends on the number of signatures t as opposed to the total number of parties k .

[0107] Theorem 1.2 (Informal). Assuming the existence of somewhere extractable BARGs, there exist aggregate signature schemes for threshold policies such that the size of the CRS, the size of an aggregated signature and the verification time are $\text{poly}(\log k, \lambda)$, and the aggregation time is $\text{poly}(t, \lambda)$ for threshold t .

[0108] Weakly Unforgeable Aggregate Signatures for Polynomial-Size Monotone Policies. Our last construction of aggregate signatures supports any policy computable by polynomial-size monotone circuits. It achieves the same (poly-logarithmic) CRS size, signature size and verification time as in Theorem 1.1 but satisfies a weaker notion of security. Intuitively, this notion only guarantees that the adversary cannot forge signatures on messages that were not signed by any honest party. This definition relaxes the fully adaptive definition discussed above as follows: in the security game, signing queries on the challenge message m under any verification key generated by the challenger are not allowed. (The adversary is still free to output its own verification keys.) A similar relaxation was studied in the context of threshold signatures where the stronger notion of full-security has proven to be more useful, but also more challenging to achieve.

[0109] Theorem 1.3 (Informal). Assuming fully homomorphic encryption and somewhere extractable BARGs, there exist weakly unforgeable aggregate signature schemes for all polynomial-size monotone circuits. The size of the CRS, the size of an aggregated signature and the verification time is $\text{poly}(\log k, \lambda)$.

[0110] Universal Aggregation and Updatability. We now highlight some additional properties of our constructions of

aggregate signatures. First, our scheme supports universal aggregation: we allow each user to use a different signature without modifying the way that individual verification keys or signatures are generated. (For Theorem 1.3 the verification keys of the base signature are required to satisfy some natural property. See Section 5.4 for further details.) As a result, a signature issued by a party i can be reused for computing aggregate signatures with respect to different aggregate verification keys for different set of parties (that include i). Second, our schemes allow new users to dynamically join the system without the need to rerun system setup. The aggregate verification key defined for any subset of users can be efficiently updated to add new users. (The aggregated verification key in our scheme is simply a root of a hash tree computed over the individual verification keys.)

[0111] On Multi-hop Aggregation. Recently, prior work used rate-1 BARGs to construct multi-hop n -out-of- n aggregate signatures, where signatures can be aggregate incrementally. While, our schemes are only presented for the single-hop setting, we follow the same BARG based approach as that prior work. Therefore, it is plausible that similar ideas could be applied to our schemes to support multi-hop aggregation; we leave further exploration of this topic to future work.

[0112] Monotone-Policy Aggregate Signatures from Batch Arguments. Our aggregate signatures constructions are based on new notions of monotone-policy BARGs. To realize Theorem 1.1 we rely on monotone-policy BARGs with a new notion of security that we called adaptive subset extraction. Our definition models an adaptive adversary that given the CRS outputs a batch of k statements, a proof π and a necessary subset J for the monotone policy f . Here, we say that a subset $J \subseteq [k]$ is necessary for f if for every input $b_1, \dots, b_k$ such that $f(b_1, \dots, b_k) = 1$, there exists a $j \in J$ such that $b_j = 1$. Our definition requires an efficient extractor algorithm that extracts from π a witness w such that conditioned on π being an accepting proof and J being a necessary subset, the extractor outputs a valid witness w_j for an index $j \in J$ with probability at least $1/k$. Since the extractor outputs a witness without knowing the set J , hitting J with probability $1/k$ is optimal. (For example in the case of k -out-of- k threshold, each singleton set $\{j\}$ for $j \in [k]$ is necessary.)

[0113] This notion is incomparable to the notion of subset extraction for monotone-policy BARGs from prior work. In their definition, the necessary subset J is chosen non-adaptively and programmed into the CRS, but the extraction algorithm is required to succeed with probability negligibly close to 1. Our construction of monotone-policy aggregate signatures crucially relies on the adaptive nature of our new security definition.

[0114] We construct BARGs with adaptive subset extraction for read-once bounded-space monotone policies.

[0115] Theorem 1.4 (Informal). Assuming the existence of somewhere extractable BARGs with 2^S -security, there exist BARGs with adaptive subset extraction for all read-once $O(S)$ space polynomial-time monotone policies. The proofs are of size $|w| \cdot \text{poly}(\log k, S, \lambda)$ and the CRS is of size $\text{poly}(\log k, \lambda)$, where λ denotes the security parameter and $|w|$ is the length of a single witness.

[0116] We also give a variant of the BARGs in Theorem 1.4 where the running time of the prover grows with the number of witnesses it is given instead of the total number of statements k . This gives Theorem 1.2.

[0117] Finally, we realize Theorem 1.3 based on BARGs for monotone policies with a new notion of security called functional subset extraction which generalizes the notion of subset extraction from prior work. In BARGs with subset extraction, we program a necessary subset J into the CRS. In contrast, in functional subset extraction, we program a function g into the CRS. We also add a tag y that is given as an additional input to the BARG prover and verifier. The requirement is that given any proof π that is accepted with respect to a tag y , if $J = g(y)$ is a necessary subset then we can extract from π a valid witness w_j for an index $j \in J$ with probability negligibly close to 1.

[0118] Based on the construction of prior work, we obtain BARGs with functional subset extraction for all policies computable by polynomial-size monotone circuits. The proof is of size $|w| \cdot \text{poly}(\log k, \lambda)$ where $|w|$ is the size of a witness, and the CRS is of size $B \cdot \text{poly}(\log k, \lambda)$, where B is about on the description size of a function that can be programmed into the CRS.

1.2 Related Work

[0119] Threshold Signatures. There is an extensive body of work dedicated to the study of threshold signatures. The de facto design of such schemes involves a distributed setup phase to compute a verification key and a threshold secret sharing of the signing key. Subsequently, the parties can jointly sign messages using their key shares, typically using an interactive signing protocol.

[0120] While any signature scheme can be “thresholdized” using secure multiparty computation, prior work has primarily focused on designing efficient thresholdizers for popular signature schemes used in practice such as Schnorr, ECDSA and BLS. All such solutions, however, require interactive protocols for key setup and (except for BLS) signature computation. Furthermore, by their very design, such solutions do not support universal aggregation of signatures from different signature schemes. Very recently, prior work overcame the former limitation by presenting an efficient instantiation of the generic SNARK-based approach discussed earlier. The security of their schemes is proven in idealized models.

[0121] Aggregate Signatures. Aggregate signatures support aggregation of signatures from different parties on possibly different messages. The key requirement is succinctness of the aggregated signature. Most known schemes also support succinct verification given a digest of the verification key of all the signers. A variation of aggregate signatures where all signers sign the same message is referred to as multisignatures. For simplicity of exposition, we use the common terminology of aggregate signatures for both of these settings throughout this work.

[0122] Until recently, aggregate signatures were only known in the Random Oracle model from pairing-based assumptions (or only achieved weaker properties). Recently, using non-interactive BARGs, new constructions in the common reference string (CRS) model were obtained based on a variety of standard assumptions including LWE, DLIN assumption over pairing groups, and sub-exponential DDH.

[0123] Batch Arguments. Prior work gave the first construction of BARGs for NP from a non-standard but falsifiable assumption over bilinear maps. Recently, a sequence of works constructed BARGs for NP from a variety of standard assumptions with proof sizes ranging from slightly sublinear to poly-logarithmic in the number of statements.

Two lines of work devised efficiency boosting compilers for BARGs: prior work shows how to achieve poly-logarithmic proof sizes generically from sublinear-size proofs, and other prior work constructs rate-1 BARGs from BARGs with low rate. Very recently, other prior work constructed BARGs for monotone circuits from polynomial hardness of LWE.

[0124] Sigma Protocols. Sigma protocols are three-round public-coin zero-knowledge proof systems with special soundness. They have many applications including efficient constructions of digital signatures via the Fiat-Shamir heuristic. Sigma protocols are known to be closed under different types of composition such as monotone span programs and more. In the composition of sigma protocols, the communication complexity typically grows with the number of instances and the primary emphasis is on preserving the zero knowledge property. In contrast, our focus is on succinctness and our signatures do not have any hiding property.

2 Technical Overview

[0125] In this section, we provide an overview of our definitions and constructions. Our starting point is a simple template construction of aggregate signatures from BARGs. By instantiating this template with existing BARG constructions, we get aggregate signatures for the k -out-of- k (i.e., conjunction) policy. In what follows, we instantiate the template construction with more general notions of BARGs resulting in aggregate signature for more expressive policies. We start by describing the template construction focusing on the simple case of conjunctions.

[0126] Template Aggregate Signature from BARGs. The CRS of the aggregate signature scheme consists of a CRS for the BARG and a description of a collision-resistant hash function H . Fix a base signature scheme S that is used by each party to generate keys and sign/verify messages. To aggregate a sequence of verification keys $vk_1, \dots, vk_k$ we compute a hash tree over them using H and set the aggregate verification key $\hat{vk}$ to its root. An aggregate signature $\hat{\sigma}$ on a message m under $\hat{vk}$ is a BARG proof that m has a signature under each of the verification keys. In more detail, let $\mathcal{L}$ be the NP language that contains tuples $(\hat{vk}, m, i) \in \mathcal{L}$ if and only if there exists a verification key vk_i , a path authenticating vk_i as the i -th leaf of the hash tree rooted at $\hat{vk}$, and a valid signature σ_i on m under vk_i using the base scheme S . Given verification keys $vk_1, \dots, vk_k$, a message m and signatures $\sigma_1, \dots, \sigma_k$, we set the aggregate signature $\hat{\sigma}$ to be a BARG proof for the k statements $(\hat{vk}, m, i) \in \mathcal{L}$ for $i \in [k]$. To verify the aggregate signature we simply verify that the BARG proof is accepting.

[0127] The succinctness of this construction follows from that of the hash tree and the BARG. The size of the aggregate verification key (i.e. the root of the hash tree) is $\text{poly}(\lambda)$ where λ is the security parameter. By the succinctness of the BARG, the size of the aggregate signature (i.e. the BARG proof) is polynomial in λ and the size of the witness for a single statement $(\hat{vk}, m, i) \in \mathcal{L}$. The witness consists of a key and signature of the base scheme S and an authentication path in the hash tree and it is, therefore, of size $\text{poly}(\lambda, \log k)$. To get efficient verification we rely on BARGs with efficient verification known as BARGs for the index language. In such BARGs, verifying the k statements

that differ from each other only in the index $i \in [k]$ takes time that is polynomial in the size of just one statement and the proof. Therefore, verifying the k statements $(\hat{vk}, m, i) \in \mathcal{L}$ for $i \in [k]$, takes time $\text{poly}(\lambda, \log k)$.

[0128] To show the security of the construction, consider an adversary A that outputs verification keys $vk_1, \dots, vk_k$, a message m , and an aggregate signature $\hat{\sigma}$ such that $\hat{\sigma}$ is a valid signature on m under the aggregate key $\hat{vk}$ and there exists at least one index $j^* \in [k]$ such that the challenger generated the key vk_{j^*} and did not sign m under vk_{j^*} . Our goal is to give a reduction that can extract from the aggregate signature $\hat{\sigma}$ (i.e. the BARG proof) a forged signature on m under vk_j (or, alternatively, a collision in H). This seems to call for BARGs that are not only sound, but also extractable (i.e. BARGs of knowledge). Moreover, the BARGs should be adaptively secure since the statements depend on $\hat{vk}$, m which A may choose as a function of the CRS.

[0129] Existing BARG constructions, however, are not known to satisfy full-fledged knowledge soundness in the adaptive setting. Indeed, such BARGs would imply SNARKs for all of NP. Instead, existing BARGs provide a weaker guarantee known as somewhere extractability. In a somewhere extractable BARG we can generate, for every index $j \in [k]$, a CRS that is “programmed” at j and is computationally indistinguishable from an honestly generated CRS. Under the programmed CRS, we can extract a witness for the j -th statement from any accepting BARG proof using a trapdoor. Using somewhere extractable BARGs, the security reduction for the aggregate signature is as follows. The reduction generates a CRS for the BARG that is programmed at a random index $j \in [k]$. If A produces a valid aggregate signature $\hat{\sigma}$ (i.e. an accepting BARG proof) the reduction extracts a witness for the j -th statement $(\hat{vk}, m, j) \in \mathcal{L}$. Such a witness contains a verification key $\hat{vk}_j$, a path authenticating $\hat{vk}_j$ under $\hat{vk}$, and a valid signature σ_j on m under $\hat{vk}_j$. If $\hat{vk}_j \neq vk_j$, the reduction finds a collision in H . Otherwise, if $j = j^*$, the reduction finds a forged signature on m under vk_{j^*} . It remains to show that indeed $j = j^*$ with noticeable probability. This follows from the fact that the programmed CRS hides the index j . (Note that the reduction finds j^* efficiently and without the CRS trapdoor).

[0130] Towards More Expressive Policies. Going beyond conjunctions, we turn our attention to aggregate signatures for more expressive monotone policies. Before describing our schemes, we start with a simple construction of an aggregate signature scheme for general monotone policies, albeit, with weak succinctness. The high-level idea is to use the aggregate signature scheme for conjunctions described above and restrict it to the subset of the parties that signed the message. In more detail, given verification keys $vk_1, \dots, vk_k$ we again set the aggregate verification key $\hat{vk}$ to be the root of the hash tree over the k keys. Let $\{\sigma_i\}_{i \in I}$ be a collection of signatures on a message m by a subset of the parties $I \subseteq [k]$ that satisfies the policy f . We aggregate $\{\sigma_i\}_{i \in I}$ into a signature $\hat{\sigma}$ under $\hat{vk}$ as follows. First, we use the conjunction scheme to aggregate $\{vk_i\}_{i \in I}$ into an key $\hat{vk}_I$ and aggregate $\{\sigma_i\}_{i \in I}$ into a signature $\hat{\sigma}_I$ under $\hat{vk}_I$. The aggregate signature $\hat{\sigma}$ contains a description of the set I , the keys $\{vk_i\}_{i \in I}$ together with their authentication paths under $\hat{vk}$, and the signature $\hat{\sigma}_I$. To verify $\hat{\sigma}$ under

$\widehat{vk}$ we first check that the set I satisfies the policy and that all the authentication paths are valid. Then we use the conjunction scheme to recompute the aggregate key $\widehat{vk}_I$ and verify the signature $\widehat{\sigma}_I$. The main drawback of this approach is that size of the aggregate signature $\widehat{\sigma}$ can grow with the size of the authorized subset I . For example, for the t -out-of- k threshold policy, the signature will grow with the threshold t . In contrast, our results give succinct aggregate signatures of size $\text{poly}(\log k, \lambda)$ even for policies where the authorized subsets may be much larger.

[0131] Aggregate Signatures from Monotone-Policy BARGs. Our aggregate signature constructions follow a different approach: starting from the template construction of aggregate signature, we replace the BARGs with the stronger notion of BARGs for monotone policies. Intuitively, a BARG for a monotone policy f , can prove that a batch of statements $x_1, \dots, x_k$ satisfies f . That is, $f(b_1, \dots, b_k)=1$, where b_i indicates the veracity of x_i . Recall that in our template aggregate signature construction, a valid statement x_i corresponds to a signature under the i -th party's verification key. BARGs for all policies given by polynomial-size monotone circuits were constructed by a recent work of Brakerski, Brodsky, Kalai, Lombardi and Paneth, however, the security notion guaranteed by their BARGs seems insufficient to prove the security of the aggregate signatures. Instead, our results are based on new monotone-policy BARG constructions that satisfy different security notions. Before expanding on these contributions, we start by reviewing the security notion of prior work and explain why it is insufficient.

[0132] The prior work puts forward a generalization of the somewhere extractability property of other prior work to the settings of monotone-policy BARGs: Instead of programming the CRS on a single index $j \in [k]$, we can program the CRS on any necessary subset of indices $J \subseteq [k]$ and the programmed CRS hides J . A subset $J' \subseteq [k]$ is necessary if any other subset J'' that satisfies the policy intersects J . Or, equivalently, J is necessary if its complement $[k] \setminus J$ does not satisfy the policy. (We assume that the policy is not a constant function.) Using the programmed CRS, we can extract from any accepting BARG proof, a witness for the j -th statement for some $j \in J$. This holds even if the adversary can choose the BARG statements $x_1, \dots, x_k \in \mathcal{L}$ adaptively, as a function of the CRS. For example, for the conjunction policy, every singleton set $\{j\}$ for $j \in [k]$ is necessary, and thus, we recover the original notion of somewhere extractability for BARGs.

[0133] We can instantiate the template aggregate signature construction with somewhere extractable monotone-policy BARGs, but we do not know how to argue the security of the resulting scheme. (Another issue is that in the construction of prior work the CRS size grows linearly with k , while we are aiming for CRS of size $\text{poly}(\lambda, \log k)$.) Going back to the aggregate signatures security game, consider an adversary A that is given a CRS, and, after interacting with the challenger, outputs verification keys $vk_1, \dots, vk_k$, a target message m and an aggregate signature $\widehat{\sigma}$ on m that verifies under the aggregate key $\widehat{vk}$. Let $J \subseteq [k]$ be the subset of indices of honest parties that did not sign m . That is, J contains every index j such that the challenger generated vk_j but did not sign m under it. Recall that A 's forgery is considered non-trivial if the set of honest parties that signed m together with all corrupted parties does not satisfy the

policy. In other words, if A wins the game then J must be a necessary subset. Therefore, if we had programmed the CRS on the set J , we could have extracted from $\widehat{\sigma}$ (i.e. the BARG proof) a witness for the j -th BARG statement $(\widehat{vk}_j, m, j) \in \mathcal{L}$ for some $j \in J$. Such a witness must contain a forged signature σ_j on m under vk_j (or, a collision in the hash H) and, since $j \in J$, this is a non-trivial forgery of the base signature scheme S . The problem with this argument is that the necessary subset J is defined by A 's queries and output and, therefore, it is not known at the time the CRS is generated. In fact, A can even choose J adaptively as a function of the CRS. (Note that if the policy has necessary subsets that are sufficiently large, then guessing the set J may result in security loss that is exponential in k . Tolerating such loss would mean losing succinctness.)

[0134] To get around this hurdle we introduce new notions of security of monotone-policy BARGs where the extraction guarantee holds for necessary subsets that can be chosen adaptively, after the CRS is fixed. The BARGs behind Theorems 1.1 and 1.3 are described in Sections 2.1 and 2.2 respectively.

2.1 Aggregate Signatures for Bounded-Space Monotone Policies

[0135] In this section we first describe our new notion of monotone-policy BARGs with adaptive subset extraction which is the main tool behind Theorem 1.1. Then we overview our construction of monotone-policy BARGs with adaptive subset extraction for the class of read-once, bounded-space monotone policies based on somewhere-extractable BARGs. A monotone policy $f: \{0, 1\}^k \rightarrow \{0, 1\}$ is read-once, space S if there exists a polynomial time Turing machine Γ and a pair of states $s_0, s_k \in \{0, 1\}^S$ such that $f(b_1, \dots, b_k)=1$ if and only if there exist states $s_1, \dots, s_{k-1} \in \{0, 1\}^S$ such that $s_i = \Gamma(s_{i-1}, b_i)$ for every $i \in [k]$.

[0136] Monotone-Policy BARGs with Adaptive Subset Extraction. In BARGs with adaptive subset extraction the CRS is generated together with an extraction trapdoor. However, in contrast to the notion of somewhere extractability, the CRS is not programmed on any particular subset. Instead, the adversary outputs a description of a subset $J^* \subseteq [k]$ together with the statements $x_1, \dots, x_k \in \mathcal{L}$ and the proof, after receiving the CRS. Ideally, we would like to require that whenever J^* is necessary and the proof is accepting, we can extract a witness for the j -th statement for some $j \in J^*$ using a trapdoor. However, this requirement seems too strong since it implies full-fledged knowledge soundness. (I.e. we can extract witnesses for i -th statement for any $i \in I$ for some subset I that satisfies the policy. (To extract, start from the necessary subset $J^*=[k]$, run the extractor, and obtain a witness for the j -th statement for some $j \in J^*$. Remove j from J^* and repeat until J^* is no longer necessary.)) Instead, we only require that if the adversary outputs a necessary subset J^* and an accepting proof with some probability α , then we can extract a witness for the j -th statement for some $j \in J^*$ with probability that is at most negligibly smaller than α/k . We note that as long as we only extract a single witness from each proof, losing a factor of k in the extraction probability is inherent. Taking, for example, the conjunction policy, if the adversary outputs a necessary subset $\{j\}$ for a random $j \in [k]$, then the extracted witness fits the j -th statement with probability at most $1/k$. (We can relax adaptive subset extraction by considering a

different bound $\beta \geq 1/k$ on the loss in the extraction probability. The notion remains meaningful for any inverse polynomial β and is still sufficient for constructing aggregate signatures.)

[0137] By instantiating the template aggregate signature construction with monotone-policy BARGs that satisfy adaptive subset extraction (instead of somewhere extraction), we can fix the flawed analysis above: Consider an adversary A that wins the aggregate signatures security game with some noticeable probability α . As before, we let J^* be the subset that contains every index j such that the challenger generated vk_j but did not sign m under it. If A wins the game then the aggregate signature $\hat{\sigma}$ is valid (i.e. the BARG proof is accepting) and the set J^* is necessary. Therefore, we are guaranteed to extract a witness for the j -th BARG statement $(\hat{vk}, m, j) \in \mathcal{L}$ for some $j \in J^*$ with noticeable probability α/k -negl. Such a witness must contain a non-trivial forged signature σ_j under vk_j (or, a collision in the hash H).

[0138] BARGs with Adaptive Subset Extraction For Low-depth Policies. Our first construction of BARGs with adaptive subset extraction is based on any somewhere extractable BARG, and it supports policies that are computable by monotone circuits of fan-in two and $\log(\lambda)$ -depth (or, more generally, d -depth under 2^d -secure BARGs). The prior work gives a simple construction of BARGs for such policies. We observe that their analysis naturally extends to show adaptive subset extraction. For completeness, we describe this construction and its analysis in Section 9. We note that the main construction from prior work that supports more expressive policies does not guarantee adaptive subset extraction out of the box.

[0139] On Adaptive Subset Extraction from Somewhere Extraction. A natural approach for constructing monotone-policy BARGs with adaptive subset extraction would be to start from any somewhere-extractable monotone-policy BARG and transform it into a BARG with adaptive subset extraction for the same policy. Indeed, for the conjunction policy such a transformation exists: simply program the CRS on a necessary subset $\{j\}$ for a random $j \in [k]$. To argue that the resulting BARG satisfies adaptive subset extraction, consider an adversary that outputs a necessary subset J^* and an accepting proof for some batch of statements with probability α . The somewhere extraction property guarantees that we can extract a witness for the j -th statement from any accepting proof. Moreover, since the CRS hides the index j , the probability of extracting a witness for the j -th statement for $j \in J^*$ is at least α/k -negl.

[0140] This transformation, however, does not seem to extend to other policies. Consider, for example, the $(k-1)$ -out-of- k threshold policy where every set with more than one index is necessary. We focus on transformations where the CRS is programmed with a subset J sampled from some distribution over necessary subsets. Let A be an adversary that outputs a random subset J^* of size exactly 2 together with an accepting proof. The somewhere extraction property guarantees that we can extract a witness for the j -th statement for some $j \in J$ with probability 1 -negl. Now, suppose that that whenever $J \cap J^*$ is not empty, the extracted index j is in $J \cap J^*$. Indeed, we cannot rule out such an adversary since the extracted index j may depend both on the programmed set J and on A 's proof (which may be correlated with J^*). (We can even construct a contrived BARG for which this attack is feasible based on the scheme of prior work.) Since

J is of size at least 2 and J^* is a set of size exactly 2 chosen at random after J is already fixed, the probability that $J \cap J^*$ is empty is at most

$$\frac{2}{k(k-1)}.$$

Therefore, $j \in J^*$ with probability that is noticeably smaller than $1/k$. (More generally, if a policy f has exactly T “minimal” necessary subsets (that do not contain smaller necessary subsets) then we can transform any somewhere-extractable BARG for f into a BARG for f with adaptive subset extraction where the loss in extraction probability is bounded by $\beta = 1/T$. However for $\beta > 1/T$ we do not know of such a transformation.)

[0141] Extracting at an Index Instead of a Subset. In the above attempt to transform somewhere extraction into adaptive subset extraction, the high level idea was to guess some necessary subset J and program it into the CRS. The problem is that we had no control over the statement in J for which we extract a witness. As a result, we had to guess a subset J that is entirely contained in the subset J^* chosen by the adversary. For some policies, however, this loss in the probability of extraction is too high.

[0142] We therefore follow a different approach: instead of guessing a necessary subset J , we guess the particular index $j \in [K]$ of the statement x_j we wish to extract a witness for, and program j into the CRS. Indeed, since j is hidden from the adversary, the probability that $j \in J^*$ is at least $1/k$ -negl. However, now we can no longer expect to extract a witness for the j -statement from any accepting proof. Unless the subset $\{j\}$ happens to be necessary, the adversary might output statements $x_1, \dots, x_k \in \mathcal{L}$ where the subset $I \subseteq [k]$ of true statements satisfies the policy, but $x_j \in \mathcal{L}$ and so extraction must fail. Moreover, we cannot rule out an adversary that outputs such statements with probability 1, even when j is sampled randomly. The issue is that the adversary may choose the statements adaptively, as a function of the CRS, without knowing which of the statements are true.

[0143] A possible solution to this problem is to have the adversary prove that it knows the subset I of true statements. Indeed, if the adversary outputs a subset J^* that is necessary, then J^* and I intersect. Therefore, if the adversary knows I (i.e. we can efficiently extract I from the adversary) then we can use the fact that j is hidden to argue that, conditioned on J^* being necessary, $j \in J^* \cap I$ with probability at least $1/k$ -negl. The problem with this solution is without resorting to SNARKs for NP, we do not know how to prove knowledge of I without losing succinctness. Nonetheless, this approach turns out to be useful: instead of requiring the prover to know the subset I of true statements, in our solution we make a weaker requirement. Very roughly, we require that it is possible to simulate a subset I that satisfies the policy and is, in some useful sense, indistinguishable from the actual subset I of true statements chose by the adversary. We show how to enforce this weaker requirement using weaker machinery that can be based on BARGs.

[0144] Verifiable PIR. The main tool we use to construct monotone-policy BARGs with adaptive subset extraction is verifiable private information retrieval (vPIR) recently introduced by Ben-David, Kalai, and Paneth. In a plain PIR protocol a server holds a database $D = (r_1, \dots, r_k)$ and a client

can query a row r_i while keeping the index i hidden from the server. Each party sends one message, and the size of the server's answer should be sublinear in k . In vPIR we additionally require the server to use a database that satisfies some predicate P . Otherwise, its answer should be rejected by the (public) verification algorithm. This security requirement is formalized via the real-ideal paradigm: for every efficient malicious server A we require that there exists an efficient simulator Sim such that for every index i the output of the following two experiments are indistinguishable: (The actual notion we work with is slightly weaker: for every polynomial q there exists a simulator Sim such that the outputs are $1/q$ -indistinguishable.)

[0145] Real: Query A on index i . If the answer verifies output the decrypted row r . Otherwise output $\perp$.

[0146] Ideal: Sim samples a database $D=(r_1, \dots, r_k)$. If $P(D)=1$ output r_i . Otherwise output $\perp$.

[0147] One may consider a stronger MPC-style notion where Sim is required to simulate also the view of A in the real experiment (i.e. the query). However, we do not know how to satisfy this stronger notion. The prior work constructs vPIR for read-once, log-space predicates based on somewhere-extractable BARGs. (More generally, they can handle read-once space S predicates based on 2^S -secure BARGs.) We say that a predicate P is read-once space S if there exists a polynomial time Turing machine Γ and a pair of states $s_0, s_k \in \{0, 1\}^S$ such that $P(r_1, \dots, r_k)=1$ if and only if there exist states $s_1, \dots, s_{k-1} \in \{0, 1\}^S$ such that $s_i = \Gamma(s_{i-1}, r_i)$ for every $i \in [k]$.

[0148] Verifiable PIR for Policies. When constructing monotone-policy BARGs for a policy f , we use vPIR for particular predicates P_f that evaluates the policy f where the i -th input bit to the policy is computed by some local polynomial-time predicate L on the i -th database row. Moreover, it would be useful to allow the local predicate to depend on some row-specific instance. That is, given instances $x_1, \dots, x_k$ and a database $D=(r_1, \dots, r_k)$, we have $P(D)=f(b_1, \dots, b_k)$ where $b_i=L(x_i, r_i)$ for every $i \in [k]$. Observe that if f is a read-once space S policy, then P_f is also a read-once space S predicate.

[0149] Looking ahead, the instances will correspond to the BARG statements $x_1, \dots, x_k \in \mathcal{L}$ and, therefore, when defining security, we need to allow the adversary choose these instances adaptively as a function of CRS. While the analysis of prior work only supports predicates that are chosen non-adaptively, we extend their construction and show that for predicates of form P_f it satisfies a useful notion of security in the adaptive setting. (To get BARGs for the index language, our vPIR construction additionally supports fast verification for instances $x_1, \dots, x_k$ that differ from each other only on the index $i \in [k]$.) For every efficient malicious server A we require that there exists an efficient simulator Sim such that for every index i the output of the following two experiments are indistinguishable:

[0150] Real: Query A on index i and obtain instances $(x_1, \dots, x_k)$ and an answer a . If the answer verifies output (x_i, r) where r is the decryption of a . Otherwise output $\perp$.

[0151] Ideal: Sim samples instances $(\tilde{x}_1, \dots, \tilde{x}_k)$ and a database $D=(r_1, \dots, r_k)$. If $P(D)=1$ output $(\tilde{x}_i, r_i)$. Otherwise output $\perp$.

One may consider a stronger notion where Sim is required to simulate all the instances $x_1, \dots, x_k$ chosen by A in the real experiment instead of just x_i . However, we do not know

how to satisfy this stronger notion. See Section 6 for further details on the notion and construction of vPIR for policies.

[0152] BARGs with Adaptive Subset Extraction from vPIR: First Attempt. We start with a simple construction of monotone-policy BARGs from vPIR. (We will eventually need to modify this construction to show adaptive subset extraction.)

[0153] To construct BARGs for a policy f we will use vPIR for a related predicate P_f over the database of witnesses: for statements $x_1, \dots, x_k$, and database $D=(w_1, \dots, w_k)$ we have that $P(D)=f(b_1, \dots, b_k)$, where $b_i=1$ if and only if w_i is a valid witness for the i -th statement. The CRS of the BARG is a vPIR query for a random index $j \in [k]$. The prover computes the vPIR answer using the database of witnesses $D=(w_1, \dots, w_k)$ (if the prover is not given a witness for x_i it sets $w_i=\perp$) and outputs it as the proof. The verifier accepts the proof if the vPIR answer verifies. To extract a witness from the proof we decrypt the vPIR answer.

[0154] To explain the difficulty in proving adaptive subset extraction, consider the following (flawed) argument. Let A be an adversary that given a CRS (i.e. a vPIR query), outputs a statement $x_1, \dots, x_k$, an accepting proof (i.e. a vPIR answer) and a necessary subset J^* with probability α . By vPIR security, there is a simulator Sim that samples instances $\tilde{x}_1, \dots, \tilde{x}_k$ and a database $\tilde{D}=(\tilde{w}_1, \dots, \tilde{w}_k)$ such that $P_f(\tilde{D})=1$ with probability $\alpha - \text{negl}$. Moreover, if j is the random index queried in the CRS, then the output $(\tilde{x}_j, \tilde{w}_j)$ of the ideal experiment is indistinguishable from the output (x_j, w) of the real experiment where w denotes the decrypted vPIR answer (this is conditioned on the outputs being different than $\perp$). If $P_f(\tilde{D})=1$ then any necessary subset J^* must contain some index $i \in J^*$ such that $\tilde{w}_i$ is a valid witness for $\tilde{x}_i \in \mathcal{L}$. Since $\tilde{D}$ is sampled independently of j we have that $i=j$ with probability $1/k$. That is, with probability at least $\alpha/k - \text{negl}$, $\tilde{w}_j$ is a valid witness for $\tilde{x}_j \in \mathcal{L}$ and $j \in J^*$. By vPIR security, it follows that also in the real experiment, the extracted value, w is a valid witness for $x_j \in \mathcal{L}$ and $j \in J^*$ with probability at least $\alpha/k - \text{negl}$. The problem is that, while this argument holds for any fixed necessary subset J^* , it may fail if A chooses J^* adaptively, as a function of the CRS (i.e. the vPIR query). Indeed, vPIR security does not guarantee simulation of both w and the view of A together.

[0155] A Simple Analysis for Threshold Policies. We start by describing an alternative analysis tailored to the case of threshold policies. The solution for general policies is more complex and we discuss it next. For the t -out-of- k threshold policy, if the simulated database $\tilde{D}=(\tilde{w}_1, \dots, \tilde{w}_k)$ satisfies the predicate, then, since the index j is random and independent of $\tilde{D}$, we have that $\tilde{w}_j$ is a valid witness for $\tilde{x}_j \in \mathcal{L}$ with probability at least $\alpha \cdot t/k$. By vPIR security, it follows that also in the real experiment, w is a valid witness for $x_j \in \mathcal{L}$ with probability at least $\alpha \cdot t/k - \text{negl}$. (Note that we can use vPIR security here since this event does not depend on the set J^* or the view of A .) For the t -out-of- k threshold policy, every necessary subset is of size at least $k-t+1$. Therefore, in the real experiment, since A outputs a necessary subset J^* and since the index j is random and hidden from A , we have that $j \in J^*$ with probability at least $\alpha \cdot (k-t+1)/k - \text{negl}$. Therefore, by the Union bound, w is a valid witness for $x_j \in \mathcal{L}$ and $j \in J^*$ with probability at least $\alpha/k - \text{negl}$.

[0156] Going Beyond Threshold Policies via Composable vPIR. The analysis above crucially relies on the symmetric structure of threshold policies and it does not seem to extend

beyond that. As suggested by failed attempt above, to support general read-once bounded space policies, it is sufficient to simulate the witness w extracted from the proof together with the subset J^* chosen by A . While we do not know if such simulation is possible, our new simulation strategy will take into account the subset J^* . Very roughly, the main idea behind our simulation is to encode the set J^* as another database, and simulate both databases together using the machinery of vPIR. To this end, we introduce a stronger notion of vPIR that remains secure under composition. Intuitively, in a t -composable vPIR protocol for predicates $P^1, \dots, P^t$, the same query can be answered using t different databases, where the i -th database is required to satisfy the predicate P^i . To capture this, we modify the real and ideal experiments as follows:

[0157] Real: Query A on index i and obtain instances $(x_1, \dots, x_k)$ and t answers $a^1, \dots, a^t$. If all the answers verify output $(x_j, r^1, \dots, r^t)$ where r^j is the decryption of a^j . Otherwise output $\perp$.

[0158] Ideal: Sim samples instances $(\tilde{x}_1, \dots, \tilde{x}_k)$ and t databases $D^1, \dots, D^t$ where $D^i = (r_i^1, \dots, r_i^t)$. If $P^i(D^i) = 1$ for every $j \in [t]$ output $(\tilde{x}_j, r_i^1, \dots, r_i^t)$. Otherwise output $\perp$.

We show that the vPIR construction of prior work is t -composable for any $t = O(1)$. Very roughly, what enables such composition is that evaluating read-once S space predicates over $O(1)$ different databases can be done by a single read-once $O(S)$ space predicate operating over all the databases simultaneously. See Section 6 for further details on our composition theorem.

[0159] BARGs with Adaptive Subset Extraction from Composable vPIR. Our final construction of BARGs from vPIR remains unchanged, except that in the analysis we rely on the fact that the vPIR is 2-composable. To argue adaptive subset extraction, consider an adversary A against the BARG that given a CRS (i.e. a vPIR query) outputs statement $x_1, \dots, x_k$, a proof (i.e. a vPIR answer a) and a subset J^* . We turn A into a 2-composable vPIR adversary A' that outputs the instances $x_1, \dots, x_k$, A' 's vPIR answer a and an additional answer a' that encodes J^* as follows: Let $D' = (b_1', \dots, b_k')$ be the database such that $b_i' = 1$ if and only if $i \in J^*$. Let P_j' be the predicate such that $P_j'(D') = 1$ if and only if $f(D') = 0$ (i.e. if and only if J^* is necessary). Using vPIR for the predicate P_j' , compute the answer a' to the query in the CRS from the database D' .

[0160] Say A outputs an accepting proof and a necessary subset J^* with probability α . When this occurs, A' outputs two accepting vPIR answers. Therefore, by 2-composable vPIR security, there is a simulator Sim that samples instances $\tilde{x}_1, \dots, \tilde{x}_k$ and a pair of databases $\tilde{D} = (\tilde{w}_1, \dots, \tilde{w}_k)$ and $\tilde{D}' = (\tilde{b}_1', \dots, \tilde{b}_k')$ such that $P_j(\tilde{D}) = 1$ and $P_j'(\tilde{D}') = 1$ with probability $\alpha - \text{negl}$. Moreover, if j is the random index queried in the CRS, then the output $(\tilde{x}_j, \tilde{w}_j, \tilde{b}_j')$ of the ideal experiment is indistinguishable from the output (x_j, w_j, b_j') of the real experiment where w and b' denote the decryption of the answers α and α' respectively (this is conditioned on the outputs being different than $\perp$). If $P_j(\tilde{D}) = 1$ and $P_j'(\tilde{D}') = 1$ then there must exist some index $i \in [k]$ such that $\tilde{w}_i$ is a valid witness for $\tilde{x}_i \in \mathcal{L}$ and $\tilde{b}_i' = 0$. Since $\tilde{D}, \tilde{D}'$ are sampled independently of j we have that $i = j$ with probability $1/k$. That is, with probability at least $\alpha/k - \text{negl}$, $\tilde{w}_j$ is a valid witness for $\tilde{x}_j \in \mathcal{L}$ and $\tilde{b}_j' = 0$. By vPIR security, it follows that also in the real experiment, the extracted value, w is a valid witness for $x_j \in \mathcal{L}$ and $b' = 0$ with probability at least $\alpha/k - \text{negl}$. Finally,

by the definition of α' (which was computed honestly) we have that $b' = b_j'$ and, therefore, w is a valid witness for $x_j \in \mathcal{L}$ and $j \in J^*$ with probability at least $\alpha/k - \text{negl}$.

[0161] Fast Prover/Aggregator. For general read-once bounded-space policies, the number of true statements required to satisfy the policy may be large. Therefore, the running time of the honest prover must, in general, grow with k . When constructing aggregate signatures from BARGs this affects the time required to aggregate signatures. However, for specific policies we can hope to do better. For example, for the t -out-of- k threshold policy, we construct a BARG with adaptive subset extraction where the prover time grows with t instead of k . Plugging this BARG into the template aggregate signature construction gives Theorem 1.2. Going beyond thresholds, the ideas behind our construction can be extended to give schemes with fast prover/aggregator for other policies as well. We discuss such extensions in Section 7.2.

[0162] We modify our construction of BARG from vPIR to use a more efficient encoding of the database of witnesses. In the scheme above the prover constructs a database with k rows where the witness for the i -th statement is stored in the i -th row, and the remaining rows contain $\perp$. Instead, the prover constructs a database D with t rows such that each row contains a witness w and an index $j \in [k]$ such that w is a valid witness for the j -th statement and, the sequence of t indices stored in the database is strictly increasing. Indeed, such a database exists if and only if at least t out of the k statements are true. Checking that the database satisfies these two properties can be done by a read-once predicate P using $\log k + 1$ bits of space. (The predicate can store the index of the last row read using $\log k$ bits and use another bit to remember if any of the previously read rows violated the constraints.)

[0163] The analysis of the new construction is similar to that above, with some key modifications. We again turn any adversary A against the BARG into a 2-composable vPIR adversary A' . When A outputs a vPIR answer a and a necessary subset J^* , A' outputs an additional answer a' as above, however, α' is computed in a different way: The answer a' is computed from a database D' using vPIR for a predicate P' where D', P' are as follows: Let $\tilde{J}$ be a set of size $t-1$ such that $J^* \cup \tilde{J} = [k]$ (such a set exists since J^* is necessary). Let $j_1, \dots, j_{t-1}$ be the indices in $\tilde{J}$ in increasing order. For $i \in [t]$, the i -th row of D' contains the pair of indices (j_{i-1}, j_i) where at the edges we set $j_0 = 0$ and $j_t = k+1$. We can think of the row (j_{i-1}, j_i) as describing an open interval. The predicate P' checks that the intervals in D' are indeed ordered and cover all indices in $[k]$ except for the $t-1$ end points in $\tilde{J}$. (Note that P' is indeed a read-once $\log k + 1$ space predicate.) The key property that enables the analysis to go through is that in the ideal experiment, if the simulator samples a pair of databases $\tilde{D}, \tilde{D}'$ such that $P(\tilde{D}) = 1$ and $P'(\tilde{D}') = 1$ then there must exist some index $i \in [t]$ such that if the i -th rows of $\tilde{D}$ and $\tilde{D}'$ are $(\tilde{w}_i, \tilde{j}_i)$ and $(\tilde{j}_{i-1}, \tilde{j}_i)$ respectively, then $\tilde{w}_i$ is a valid witness for the $\tilde{j}_i$ -th statement and $\tilde{j}_{i-1} < \tilde{j}_i$. Intuitively, the latter guarantees that in the real world, the extracted index j is outside the set $\tilde{J}$ (and therefore, inside J^*) with sufficiently high probability. See Section 7 for further details.

2.2 Weakly Unforgeable Aggregate Signatures for Polynomial-Size Monotone Policies

[0164] In this section we overview the proof of Theorem 1.3 for constructing aggregate signatures for all monotone

policies given by polynomial-size circuits with weakly unforgeable security. Recall that in the aggregate signatures security game, the adversary outputs verification keys $vk_1, \dots, vk_k$, a target message m and a signature $\hat{\sigma}$. The adversary wins if $\hat{\sigma}$ is a valid signature on m under the aggregate key and if the forgery is non-trivial. In the case of weakly unforgeable security, the forgery is considered non-trivial if the set of corrupted parties, whose keys were not generated by the challenger, does not satisfy the policy, and if no honest party signed m . (This is in contrast to the fully adaptive notion where honest parties are allowed to sign m , but the set of honest parties that signed m together with all corrupted parties should not satisfy the policy.)

[0165] Our construction is, once again, based on the template aggregate signature construction instantiated with a new notion of monotone-policy BARGs. To motivate this new notion we briefly recall our previous attempts: in a nutshell, our proof strategy was to define a necessary subset of indices J^* based on the adversary's queries and then try to extract a witness for the j -th BARG statement for some $j \in J^*$. Since J^* was only fixed at the time the adversary outputs its forgery, we could not program J^* into the CRS. Instead, we relied on the stronger notion of adaptive subset extraction where the subset J^* can be chosen by the adversary, as a function of the CRS. Our construction of BARGs with adaptive subset extraction, however, is limited to read-once bounded space policies.

[0166] The main idea behind our construction in Theorem 1.3 is to fix the subset J^* as a function of the BARG statements. This will allow us to target J^* not only during extraction, but already in the construction and verification of the BARG proof. In more detail, in the weakly unforgeable aggregate signatures security game, the subset J^* contains the indices of all honest parties whose verification keys were generated by the challenger. (This is in contrast to the fully adaptive notion where J^* does not contain indices of honest parties that signed m .) Therefore, to recover the subset J^* from the BARG statements (i.e. the verification keys $vk_1, \dots, vk_k$) it is sufficient to distinguish keys generated by the challenger from keys generated by the adversary. To this end, we rely on an underlying signature scheme that supports trapdoor keys. In such a signature scheme, it is possible to generate "marked" verification keys that can be recognized using a trapdoor. Without the trapdoor, however, the adversary cannot distinguish the marked keys from honestly generated keys, or generate new marked keys. (Starting from any signature scheme, we can construct a scheme with trapdoor keys by adding a random string to each verification key. To mark a key, we replace the random string with a PRF-based MAC.)

[0167] To implement this idea, we instantiate the template aggregate signature construction with a new notion of monotone-policy BARGs with functional subset extraction that generalise the somewhere extraction property. A construction satisfying this notion is implicit in prior work. Recall that in BARGs with somewhere extraction, if we program the CRS on a necessary subset J , then, using a trapdoor, we can extract a witness for the j -th statement for some $j \in J$ from any accepting proof. In functional subset extraction, the subset J depends both on the programmed CRS and on an additional input that is fixed together with the BARG statements. In more detail, the CRS is programmed with a function g and it is indistinguishable from an honestly generated CRS. (The length of the CRS grows with an upper

bound on the description of g .) The BARG prover and verifier take an additional input y . The guarantee is that if $J=g(y)$ is a necessary subset, then we can extract a witness for the j -th statement for some $j \in J$ from any accepting proof using a trapdoor. We additionally require that the verification time does not grow with the running time of g . Moreover, in case the input y is itself long, the verifier only requires a short digest of y and its running time does not grow with $|y|$.

[0168] We plug in BARGs with functional subset extraction into the template aggregate signature construction. To aggregate a signature under a sequence of verification keys $vk_1, \dots, vk_k$, we use this sequence as the input y to the BARG prover. We also include a short digest (a hash root) of y in the aggregate verification key, and the BARG verifier uses this digest to verify an aggregate signature. In the analysis, we consider an alternative challenger that gives the adversary marked verification keys, and programs the CRS with a function g that is hard-coded with the trapdoor for the base signature. Given an input $y=(vk_1, \dots, vk_k)$, g uses the trapdoor to recognize the marked keys and outputs the subset J containing their indices. We note that this analysis does not extend to the fully adaptive setting. In the fully adaptive security game, the adversary can ask the challenger to sign the target message m under some of the marked verification keys. In this case, the indices of these keys should not be included in the subset J , or we may extract a trivial forgery. In our weakly unforgeable construction, however, the subset J is fixed only as a function of the verification keys chosen by the adversary, regardless of its signing queries.

[0169] Our construction of monotone-policy BARGs with functional subset extraction is based on the somewhere extractable BARGs for monotone policies given by polynomial-size circuits from prior work. In their construction, the CRS contains the programmed subset J encrypted with a fully homomorphic encryption scheme. Therefore, given an input y and a CRS that contains an encryption of a function g , the prover homomorphically evaluates a CRS that is programmed on $g(y)$ and computes its BARG proof with respect to the evaluated CRS. The verifier given the input y can recompute the evaluated CRS itself and verify the proof. To allow verification given only the hash of y and to reduce the verification time to be independent of the running time of g , we use a RAM SNARK for deterministic computation to delegate the computation of the evaluated CRS to the prover.

3 Preliminaries

[0170] Notations. We use PPT to denote probabilistic polynomial-time, and denote the set of all positive integers up to n as $[n] := \{1, \dots, n\}$. For any $x \in \{0, 1\}^n$ and any subset $J \subseteq [n]$ we denote by $x_J = (x_j)_{j \in J}$. For any finite set S , $x \leftarrow S$ denotes a uniformly random element x from the set S . Similarly, for any distribution $\mathcal{D}$, $x \leftarrow \mathcal{D}$ denotes an element x drawn from the distribution $\mathcal{D}$. We will also use the notation $\mathbb{1}_{[k] \cup}$ often to denote a vector in $\{0, 1\}^k$ such that the i -th position is 0 if and only if $i \in J$.

[0171] The universal language. Let $\mathcal{L}_U$ be the language of all triplets (Γ, x, y, T) such that Γ is a description of a Turing machine that on input x outputs y in T steps. We write $(\Gamma, x, T) \in \mathcal{L}_U$ as a shorthand for $(\Gamma, x, 1, T) \in \mathcal{L}_U$, i.e., Γ accepts x in T steps.

3.1 Digital Signatures

[0172] In this section we define (standard) digital signatures.

[0173] Syntax. A digital signature scheme consists of the following polynomial-time algorithms:

[0174] $\text{KeyGen}(1^\lambda) \rightarrow (\text{sk}, \text{vk})$. This is a probabilistic algorithm that takes as input the security parameter 1^λ . It outputs a signing key sk and a verification key vk .

[0175] $\text{Sign}(\text{sk}, m) \rightarrow \sigma$. This is a probabilistic algorithm that takes as input the signing key sk and a message $m \in \{0, 1\}^\lambda$. It outputs a signature σ .

[0176] $\text{Verify}(\text{vk}, m, \sigma) \rightarrow 0/1$. This is a deterministic algorithm that takes as input the verification key vk , a message $m \in \{0, 1\}^\lambda$ and a signature σ . It outputs a bit (1 to accept, 0 to reject).

[0177] Definition 3.1. A digital signature scheme (KeyGen , Sign , Verify) is required to satisfy the following properties:

[0178] Correctness. For any $\lambda \in \mathbb{N}$ and $m \in \{0, 1\}^\lambda$

$$\Pr \left[\text{Verify}(\text{vk}, m, \sigma) = 1 : \begin{array}{l} (\text{sk}, \text{vk}) \leftarrow \text{KeyGen}(1^\lambda) \\ \sigma \leftarrow \text{Sign}(\text{sk}, m) \end{array} \right] = 1.$$

[0179] Unforgeability. For any admissible poly-size adversary $\mathcal{A}$, there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\text{Verify}(\text{vk}, m^*, \sigma^*) = 1 : \begin{array}{l} (\text{sk}, \text{vk}) \leftarrow \text{KeyGen}(1^\lambda) \\ (m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}(1^\lambda, \text{vk}) \end{array} \right] \leq \text{negl}(\lambda),$$

[0180] where we say that $\mathcal{A}$ is admissible if it did not query the $\text{Sign}(\text{sk}, \cdot)$ oracle with m^* .

[0181] Remark 3.1 (Message space). We assume that the message space is $\{0, 1\}^\lambda$. This is without loss of generality, since we can sign arbitrary messages by first applying a hash function (that satisfies targeted collision resistance), then signing the hashed message.

3.2 Hash Family with Local Opening

[0182] In this section we recall the definition of a hash family with local opening. (In what follows we use the notation HT to denote a hash family with local opening, where HT symbolizes a Hash Tree construction. We emphasize that we are not restricted to such a construction, and use this notation only to give the reader an example to have in mind.)

[0183] Syntax. A hash family (HT) with succinct local opening consists of the following algorithms:

[0184] $\text{Gen}(1^\lambda) \rightarrow \text{hk}$. This is a PPT algorithm that takes as input the security parameter 1^λ in unary and outputs a hash key hk .

[0185] $\text{Hash}(\text{hk}, x) \rightarrow \text{rt}$. This is a deterministic poly-time algorithm that takes as input a hash key hk and an input $x \in \{0, 1\}^N$ for $N \geq 2^\lambda$, and outputs a hash value rt .

[0186] $\text{Open}(\text{hk}, x, j) \rightarrow \rho$. This is a deterministic poly-time algorithm that takes as input a hash key hk , an input $x \in \{0, 1\}^N$ for $N \geq 2^\lambda$, and an index $j \in [N]$, and outputs an opening ρ .

[0187] $\text{Verify}(\text{hk}, r', j, b, \rho) \rightarrow 0/1$. This is a deterministic poly-time algorithm that takes as input a hash key hk ,

a hash value rt , an index $j \in [N]$, a bit $b \in \{0, 1\}$ and an opening ρ . It outputs a bit (1 to accept, 0 to reject).

[0188] Definition 3.2. (Properties of HT) A HT family (Gen , Hash , Open , Verify) is required to satisfy the following properties.

[0189] Opening completeness. For any $\lambda \in \mathbb{N}$, any $N \geq 2^\lambda$, any $x \in \{0, 1\}^N$, and any index $j \in [N]$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{vk}, \text{rt}, j, x_j, \rho) = 1 : \\ \text{hk} \leftarrow \text{Gen}(1^\lambda), \\ \text{rt} = \text{Hash}(\text{hk}, x), \\ \rho = \text{Open}(\text{hk}, x, j) \end{array} \right] = 1 - \text{negl}(\lambda).$$

[0190] Succinctness. In the completeness experiment above, we have that $|\text{hk}| + |\text{rt}| + |\rho| = \text{poly}(\lambda)$.

[0191] Collision resistance w.r.t. opening. For any poly-size adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{hk}, \text{rt}, j, 0, \rho_0) = 1 \\ \wedge \text{Verify}(\text{hk}, \text{rt}, j, 1, \rho_1) = 1 \end{array} : \begin{array}{l} \text{hk} \leftarrow \text{Gen}(1^\lambda), \\ (\text{rt}, j, \rho_0, \rho_1) \leftarrow \mathcal{A}(\text{hk}) \end{array} \right] = \text{negl}(\lambda).$$

[0192] Remark 3.2. We say that a hash family with local opening is Λ -secure, for $\Lambda = \Lambda(\lambda)$, if the collision resistance w.r.t. opening property holds against any $\text{poly}(\Lambda)$ -size adversary (as opposed to $\text{poly}(\lambda)$ -size) and the probability that the adversary finds a collision is $\text{negl}(\Lambda)$ (as opposed to $\text{negl}(\lambda)$). We refer to this property as Λ -collision-resistance w.r.t. opening.

[0193] Theorem 3.3. Assuming the existence of a collision resistant hash family there exists a hash family with local opening (according to Definition 3.2).

3.3 Somewhere Extractable Batch Arguments (seBARGs)

[0194] A batch argument system BARG for an NP language $\mathcal{L}$ enables proving that k NP statements are true with communication cost that is polylogarithmic in k . There are many BARG variants which are known to be existentially equivalent under mild computational assumptions. In this work, for simplicity in our constructions, we make use of an argument system for “batch index Turing machine SAT” (BatchTMSAT), defined in prior work.

[0195] Definition 3.4. The language BatchIndexTMSAT consists of instances of the form $x = (M, z, k, T)$, where:

[0196] M is the description of a Turing machine.

[0197] z is an input string (to M)

[0198] k is a batch size, and

[0199] T is a running time.

An instance $x = (M, z, k, T)$ is in BatchIndexTMSAT if for all $1 \leq i \leq k$, there exists a string w_i such that $M(z, i, w_i)$ accepts within T steps.

[0200] We sometimes use the notation $\mathcal{R}(x, i, w_i)$ to denote the relation with instance (x, i) and corresponding witness w_i .

[0201] Syntax. A (publicly verifiable and non-interactive) somewhere extractable batch argument system seBARG for BatchIndexTMSAT consists of the following polynomial-time algorithms:

[0202] $\text{Gen}(1^\lambda, i^*) \rightarrow (\text{crs}, \text{td})$. This is a probabilistic algorithm that takes as input a security parameter 1^λ , and an index $i^* \in [2^\lambda]$. It outputs a common reference string crs along with a trapdoor td .

- [0203] $\mathcal{P}(\text{crs}, M, z, 1^T, w_1, \dots, w_k) \rightarrow \pi$. This is a deterministic algorithm that takes as input a crs, Turing machine M , input z , runtime 1^T , and k witnesses $w_1, \dots, w_k$. It outputs a proof π .
- [0204] $\mathcal{V}(\text{crs}, x, \pi) \rightarrow 0/1$. This is a deterministic algorithm that takes as input a crs, instance $x=(M, z, k, T)$, and a proof π . It outputs a bit (1 to accept, 0 to reject).
- [0205] $\text{Extract}(td, \pi) \rightarrow w$. This is a deterministic algorithm that takes as input a trapdoor td and a proof π . It outputs a witness w .
- [0206] Definition 3.5 (seBARG). A somewhere-extractable batch argument scheme $\text{seBARG}=(\text{Gen}, \mathcal{P}, \mathcal{V}, \text{Extract})$ for BatchIndexTMSAT is required to satisfy the following properties:
- [0207] Completeness. For any $\lambda \in \mathbb{N}$, any $k, n, m, T \leq 2^\lambda$, any instance $x=(M, z, k, T) \in \text{BatchIndexTMSAT}$ with $|M|+|z|=n$, any corresponding witnesses $w_1, \dots, w_k \in \{0, 1\}^m$ and any index $i^* \in [k]$,

$$\Pr \left[\mathcal{V}(\text{crs}, x, \pi) = 1 : \begin{array}{l} (\text{crs}, td) \leftarrow \text{Gen}(1^\lambda, i^*), \\ \pi \leftarrow \mathcal{P}(\text{crs}, M, z, 1^T, w_1, \dots, w_k) \end{array} \right] = 1.$$

- [0208] Efficiency. In the completeness experiment above, $|\text{crs}|+|\pi| \leq m \cdot \text{poly}(\lambda)$. The running time of the verifier is at most $\text{poly}(|\text{crs}|+|\pi|)+\text{poly}(\lambda) \cdot |x|$.
- [0209] Index hiding. For any poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and every pair of indices $i_0, i_1 \in [2^\lambda]$,

$$\Pr \left[\mathcal{A}(\text{crs}) = b : \begin{array}{l} b \leftarrow \{0, 1\}, \\ (\text{crs}, td) \leftarrow \text{Gen}(1^\lambda, i_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

- [0210] Somewhere argument of knowledge. For any poly-size adversary $\mathcal{A}$ and any polynomials $k(\lambda), T(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for any index $i^* \in [k]$ and for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \mathcal{V}(\text{crs}, x, \pi) = 1 \\ \wedge (x, i^*, w^*) \notin \mathcal{R} : \begin{array}{l} (\text{crs}, td) \leftarrow \text{Gen}(1^\lambda, i^*) \\ (M, z, \pi) = \mathcal{A}(\text{crs}) \\ w^* \leftarrow \text{Extract}(td, \pi) \\ x = (M, z, k, T) \end{array} \end{array} \right] \leq \text{negl}(\lambda).$$

[0211] Remark 3.3. We say that a seBARG scheme is Λ -secure, for $\Lambda=\Lambda(\lambda)$, if the index hiding property and the somewhere argument of knowledge property hold w.r.t. a $\text{poly}(\Lambda)$ -size adversary (as opposed to a $\text{poly}(\lambda)$ -size), and the advantage probability is $\text{negl}(\Lambda)$ (as opposed to $\text{negl}(\lambda)$). We refer to these properties as Λ -index-hiding and Λ -somewhere-argument-of-knowledge, respectively.

[0212] Remark 3.4. Given an seBARG , one can naturally extend the definition of the key generation algorithm Gen to take as input an index set $I \subset [k]$, as opposed to a single index. $\text{Gen}(1^\lambda, I)$ will simply run $\text{Gen}(1^\lambda, i)$ for every $i \in I$. The prover algorithm $\mathcal{P}$, given a crs that encodes the $|I|$ indices, will simply generate $|I|$ proofs (one for each crs), and the verifier will check these $|I|$ proofs independently.

[0213] Theorem 3.6. There exists a Λ -secure seBARG for BatchIndexTMSAT assuming Λ -hardness of LWE or DLIN.

4 Aggregate Signatures for Monotone Policies

[0214] In this section we define aggregate signature schemes for monotone policies. We additionally define a weaker notion, and define a special case of aggregate signatures where the prover time depends only on the number of instances for which it has witnesses.

[0215] Syntax. Let $F=\{F_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of monotone policies, such that each $f \in F_\lambda$ is a monotone function $f: \{0, 1\}^k \rightarrow \{0, 1\}$. Let $S=(\text{KeyGen}, \text{Sign}, \text{Verify})$ be a digital signature scheme. An F -aggregation scheme for S consists of the following polynomial-time algorithms:

[0216] $\text{Setup}(1^\lambda) \rightarrow \text{crs}$. This is a probabilistic setup algorithm that takes as input the security parameter 1^λ . It outputs a common reference string crs .

[0217] $\text{KeyAgg}(\text{crs}, \{\text{vk}_i\}_{i \in [k]}) \rightarrow \hat{\text{vk}}$. This is a deterministic key aggregation algorithm that takes as input the common reference string crs , and a collection of verification keys $\{\text{vk}_i\}_{i \in [k]}$. It outputs an aggregate verification key $\hat{\text{vk}}$.

[0218] $\text{SigAgg}(\text{crs}, f, \{\text{vk}_i, \sigma_i\}_{i \in [k]}, m) \rightarrow \hat{\sigma}$. This is a deterministic signature aggregation algorithm that takes as input the common reference string crs , a policy f , a collection of verification keys and signatures $\{\text{vk}_i, \sigma_i\}_{i \in [k]}$, and a message m . It outputs an aggregate signature $\hat{\sigma}$.

[0219] $\text{AggVerify}(\text{crs}, f, \hat{\text{vk}}, m, \hat{\sigma}) \rightarrow 0/1$. This is a deterministic aggregate verification algorithm that takes as input the common reference string crs , a policy f , an aggregate verification key $\hat{\text{vk}}$, a message m , and an aggregate signature $\hat{\sigma}$. It outputs a bit (1 to accept, 0 to reject).

[0220] Definition 4.1 (Aggregation Scheme). An F -aggregation scheme $(\text{Setup}, \text{KeyAgg}, \text{SigAgg}, \text{AggVerify})$ for $S=(\text{KeyGen}, \text{Sign}, \text{Verify})$ is required to satisfy the following properties:

[0221] Correctness. For any $\lambda \in \mathbb{N}$, any policy $f \in F_\lambda$, any message $m \in \{0, 1\}^\lambda$, and any collection of verification keys and signatures $\{\text{vk}_i, \sigma_i\}_{i \in [k]}$ such that for $b_i = \text{Verify}(\text{vk}_i, m, \sigma_i)$ it holds that $f(b_1, \dots, b_k) = 1$,

$$\Pr \left[\text{AggVerify}(\text{crs}, f, \hat{\text{vk}}, m, \hat{\sigma}) = 1 : \right.$$

$$\left. \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ \hat{\text{vk}} \leftarrow \text{KeyAgg}(\text{crs}, \{\text{vk}_i\}_{i \in [k]}) \\ \hat{\sigma} \leftarrow \text{SigAgg}(\text{crs}, f, \{\text{vk}_i, \sigma_i\}_{i \in [k]}, m) \end{array} \right] = 1.$$

[0222] Efficiency. There exists a fixed polynomial poly such that in the correctness experiment above, $|\hat{\text{vk}}|+|\hat{\sigma}| = \text{poly}(\lambda)$.

[0223] Unforgeability. For any $\lambda \in \mathbb{N}$ and $f \in F_\lambda$, define the unforgeability game between an adversary $\mathcal{A}$ and a challenger as follows:

[0224] The challenger samples $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, and gives crs to $\mathcal{A}$.

[0225] The adversary can now make queries to the challenger. Each query is of one of two types:

[0226] Verification key queries: $\mathcal{A}$ can request a verification key from the challenger. The challenger generates $(sk, vk) \leftarrow \text{KeyGen}(1^\lambda)$, gives vk to $\mathcal{A}$ and saves the pair (sk, vk) .

[0227] Signing queries: Given a verification key vk and a message $m \in \{0, 1\}^\lambda$, if the challenger previously saved the pair (sk, vk) it gives $\text{Sign}(sk, m)$ to $\mathcal{A}$. Otherwise, it gives $\perp$.

[0228] At the end of the game the adversary outputs a collection of verification keys $\{vk_i\}_{i \in [k]}$, a message $m \in \{0, 1\}^\lambda$, and an aggregate signature $\hat{\sigma}$.

[0229] Let $\hat{vk} = \text{KeyAgg}(crs, \{vk_i\}_{i \in [k]})$. For $i \in [k]$, let $b_i = 1$ if $\mathcal{A}$ did not make a verification key query that was answered with vk_i (that is, if vk_i is a maliciously generated key), or if $\mathcal{A}$ made a signing query for vk_i , m . Otherwise, let $b_i = 0$.

[0230] The adversary $\mathcal{A}$ wins the game if $f(b_1, \dots, b_k) = 0$ and $\text{AggVerify}(crs, f, \hat{vk}, m, \hat{\sigma}) = 1$.

[0231] For any poly-size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and $f \in F_\lambda$, $\mathcal{A}$ wins the unforgeability game with probability at most $\text{negl}(\lambda)$.

[0232] We also consider a relaxed definition satisfying a weaker notion of unforgeability.

[0233] Definition 4.2 (Weakly Unforgeable Aggregation Scheme). A weakly unforgeable F-aggregation scheme (Setup , KeyAgg , SigAgg , AggVerify) for S is required to satisfy correctness and efficiency (as in Definition 4.1), and a weaker notion of unforgeability.

[0234] The weak unforgeability game is identical to the unforgeability game in Definition 4.1, except that to win the game, in addition to the condition specified in Definition 4.1, the adversary $\mathcal{A}$ may not make any signing query of the form (vk_i, m) for any $i \in [k]$.

4.1 Fast Aggregation

[0235] In the notion of aggregation scheme as defined above, to compute an aggregated signature under an aggregated key $\hat{vk}$ the signature aggregator must know all the verification keys aggregated in $\hat{vk}$. In what follows we formalize the notion of aggregation scheme with “sublinear” aggregation where the signature aggregator is only given a subset of the verification keys aggregated in $\hat{vk}$ and signatures under the keys in this subset such that the subset satisfies the policy. In addition, for every verification key in the subset, the signature aggregator is also given a “proof” that the key is indeed one of the keys aggregated in $\hat{vk}$. For a signature aggregator that is given t signatures and verification keys, we want the aggregator run time to depend only on t .

[0236] Syntax. An F-aggregation scheme for S with fast aggregation contains polynomial-time key aggregation and signature aggregation algorithms with the following syntax (the syntax of the setup and aggregated verification algorithm is unchanged):

[0237] $\text{KeyAgg}(crs, \{vk_i\}_{i \in [k]}) \rightarrow (\hat{vk}, \{\rho_i\}_{i \in [k]})$. This is a deterministic key aggregation algorithm that takes as input the common reference string crs , and a collection of verification keys $\{vk_i\}_{i \in [k]}$.

[0238] It outputs an aggregate verification key $\hat{vk}$, and openings $\rho_1, \dots, \rho_k$.

[0239] $\text{SigAgg}(crs, f, \hat{vk}, \{vk_i, \rho_i, \sigma_i\}_{i \in [t]}, m) \rightarrow \hat{\sigma}$. This is a deterministic signature aggregation algorithm that takes as input the common reference string crs , a policy f , the aggregate verification key $\hat{vk}$, a collection of verification keys, openings and signatures $\{vk_i, \rho_i, \sigma_i\}_{i \in [t]}$, and a message m . It outputs an aggregate signature $\hat{\sigma}$.

[0240] We modify Definition 4.1 to support the syntax above. In particular, the modified correctness property is as follows:

[0241] Correctness. For any $\lambda \in \mathbb{N}$, any $k \leq 2^\lambda$, any policy $f \in F_\lambda$, any message $m \in \{0, 1\}^\lambda$, any set $S \subseteq [k]$ such that $f \upharpoonright_S = 1$, and any collection of verification keys and signatures $\{vk_i, \sigma_i\}$ such that $\text{Verify}(vk_i, m, \sigma_i) = 1$ for every $i \in S$,

$$\Pr \left[\text{AggVerify}(crs, f, \hat{vk}, m, \hat{\sigma}) = 1 : \right.$$

$$\left. \begin{array}{l} crs \leftarrow \text{Setup}(1^\lambda) \\ (\hat{vk}, \{\rho_i\}_{i \in [k]}) \leftarrow \text{KeyAgg}(crs, \{vk_i\}_{i \in [k]}) \\ \hat{\sigma} \leftarrow \text{SigAgg}(crs, f, \hat{vk}, \{vk_i, \rho_i, \sigma_i\}_{i \in S}, m) \end{array} \right] = 1.$$

Batch Arguments for Monotone Policies

[0242] In this section we provide new definitions for batch arguments, specifically BARGs with adaptive subset extraction, and BARGs with functional subset extraction, both for monotone policies. The definitions extend the definition of BARGs for monotone policies considered in prior work. We then show how to construct aggregate signatures from BARGs with adaptive subset extraction, and weakly unforgeable aggregate signatures from BARGs with functional subset extraction.

[0243] We defer the construction of these BARGs to later technical sections. Our BARGs are going to be defined for the following batch language considered in prior work.

[0244] Definition 5.1. The language Monotone Policy-TM-SAT consists of instances of the form $x = (f, M, z, T)$, where:

[0245] $f: \{0, 1\}^k \rightarrow \{0, 1\}$ is the description of a monotone function.

[0246] M is the description of a Turing machine.

[0247] z is an input string (to M), and

[0248] T is a running time.

[0249] An instance $x = (f, M, z, T)$ is in Monotone Policy-TM-SAT if it satisfies any one of the following witness relations. The two relations define the same language, but differ in the witness representation. Notably, $\mathcal{R}_{\text{sub}}$ allows for witnesses that are sublinear in the batch size k , which will allow us to obtain more efficient provers for our batch arguments.

[0250] A $\mathcal{R}_{full}$ -witness for x is $w=(w_1, \dots, w_k)$ such that $f(b_1, \dots, b_k)=1$, where $b_i \in \{0, 1\}$ are defined by $b_i=1$ if and only if $M(z, i, w_i)$ accepts within T steps.

[0251] A $\mathcal{R}_{sub}$ -witness for x is $w=(i_1, w_1, \dots, i_t, w_t)$ such that:

[0252] 1. For every $j \in [t]$, it holds that $i_j \in [k]$ and $M(z, i_j, w_j)$ accepts within T steps.

[0253] 2. $f(b_1, \dots, b_k)=1$, where $b_i=1$ if and only if $i=i_j$ for some $j \in [t]$.

[0254] Further, let $F=\{F_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of policies, where each $f \in F_\lambda$ is a monotone function $f: \{0, 1\}^k \rightarrow \{0, 1\}$. We say that subset $J \subseteq [k]$ is necessary for a monotone function f if $f|_{[k] \setminus J} = 0$.

5.1 Batch Arguments with Adaptive Subset Extraction

[0255] The syntax of batch arguments with adaptive subset extraction is similar to the syntax of somewhere-extractable BARGs for monotone policy in prior work, except that the trapdoor generated by the setup algorithm does not depend on a particular subset of indices (or on a single index as in standard somewhere extractable batch arguments). Instead the adaptive subset extraction guarantee will be defined with respect to necessary subsets chosen by the prover as a function of the crs.

[0256] Syntax. Let $\mathcal{R}$ be a witness relation for Monotone PolicyTMSAT (either $\mathcal{R}_{full}$ or $\mathcal{R}_{sub}$). Let $\alpha=\alpha(\lambda)$ be a polynomial. A F-batch argument with α -adaptive subset extraction for relation $\mathcal{R}$ consists of the following polynomial-time algorithms:

[0257] $\text{Setup}(1^\lambda) \rightarrow (\text{crs}, \text{td})$. This is a probabilistic setup algorithm that takes as input the security parameter 1^λ . It outputs a common reference string crs and a trapdoor td.

[0258] $\mathcal{P}(\text{crs}, f, M, z, 1^T, w) \rightarrow \pi$. This is a deterministic prover algorithm that takes as input the common reference string crs, policy f , Turing machine M , input z , runtime 1^T , and witness w (for the relation $\mathcal{R}$). It outputs a proof π .

[0259] $\mathcal{V}(\text{crs}, x, \pi) \rightarrow 0/1$. This is a deterministic verification algorithm that takes as input the common reference string crs, instance $x=(f, M, z, T)$, and a proof π . It outputs a bit (1 to accept, 0 to reject).

[0260] $\text{Extract}(\text{td}, \pi) \rightarrow (i, w_i)$. This is a deterministic extraction algorithm that takes as input the trapdoor td, and a proof π . It outputs an index $i \in [k]$, and a witness w_i .

[0261] Definition 5.2 (Batch Argument for Policies with Adaptive Subset Extraction). A F-batch argument with α -adaptive subset extraction ($\text{Setup}, \mathcal{P}, \mathcal{V}, \text{Extract}$) for relation $\mathcal{R}$ is required to satisfy the following properties:

[0262] Completeness. For any $\lambda \in \mathbb{N}$, any $n, m, T \leq 2^\lambda$, any instance $x=(f, M, z, T) \in \text{Monotone PolicyTMSAT}$ with $|M|+|z|=n$ and corresponding witness w such that $(x, w) \in \mathcal{R}$ and $|w_i|=m$,

$$\Pr \left[\mathcal{V}(\text{crs}, x, \pi) = 1 : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \text{Setup}(1^\lambda) \\ \pi \leftarrow \mathcal{P}(\text{crs}, f, M, z, 1^T, w) \end{array} \right] = 1.$$

[0263] Succinctness. In the completeness experiment above, $|\pi| \leq m \cdot \text{poly}(\lambda)$. The running time of the verifier is at most $\text{poly}(|\text{crs}| + |\pi|) + \text{poly}(\lambda) \cdot |x|$.

[0264] α -Adaptive Subset Extraction. For any polynomial $T(\lambda)$, any poly-size cheating prover $\mathcal{P}^*$, and any sequence $\{f_\lambda \in F_\lambda\}_{\lambda \in \mathbb{N}}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr_{\text{EXP}} [i \in J \wedge M(z, i, w_i) = 1] \geq \frac{1}{\alpha(\lambda)} \cdot \Pr_{\text{EXP}} \left[\mathcal{V}(\text{crs}, x, \pi) = 1 \wedge f|_{[k] \setminus J} = 0 \right] - \text{negl}(\lambda),$$

[0265] where EXP is the experiment defined as follows:

[0266] Generate $(\text{crs}, \text{td}) \leftarrow \text{Setup}(1^\lambda)$.

[0267] Run the cheating prover and obtain $(M, z, \pi, J) \rightarrow \mathcal{P}^*(\text{crs})$.

[0268] Let $x=(f_\lambda, M, z, T)$.

[0269] Extract $(i, w_i) \leftarrow \text{Extract}(\text{td}, \pi)$.

5.2 Batch Arguments with Functional Subset Extraction

[0270] We now consider a related notion of functional subset extraction, which generalizes the notion of somewhere extractable BARGs for monotone policy defined in prior work. Here, we program the crs on some function g whose outputs are subsets of $[k]$. The extraction guarantee we will require that if prover and verifier both use a function input y , we can extract from the proof a witness from the set $J=g(y)$, as long as J is a necessary subset. Unlike the definition of adaptive subset extraction considered in Section 5.1, where the crs does not require additional input, we note that here we do have to program a crs but the adversary is allowed to adaptively pick the input y to the function g .

[0271] Syntax. Let $\mathcal{R}$ be a witness relation for Monotone PolicyTMSAT (defined in Definition 5.1). The syntax remains largely the same as in the case of BARGs adaptive subset extraction, with the primary difference pertaining to the additional function g specified during setup, and the additional input y used by both parties.

[0272] A F-batch argument with functional subset extraction consists of the following polynomial-time algorithms:

[0273] $\text{Setup}(1^\lambda, g) \rightarrow (\text{crs}, \text{td})$. This is a probabilistic setup algorithm that takes as input the security parameter 1^λ and a function $g \in G_\lambda$. It outputs a common reference string crs and a trapdoor td.

[0274] $\mathcal{P}(\text{crs}, y, f, M, z, 1^T, w) \rightarrow \pi$. This is a deterministic prover algorithm that takes as input the common reference string crs, function input y , policy f , Turing machine M , input z , runtime 1^T , and witness w (for the relation $\mathcal{R}$). It outputs a proof π .

[0275] $\mathcal{V}(\text{crs}, y, x, \pi) \rightarrow 0/1$. This is a deterministic verification algorithm that takes as input the common reference string crs, function input y , instance $x=(f, M, z, T)$, and a proof π . It outputs a bit (1 to accept, 0 to reject).

[0276] $\text{Extract}(\text{td}, y, \pi) \rightarrow (i, w_i)$. This is a deterministic extraction algorithm that takes as input the trapdoor td, and a proof π . It outputs an index $i \in [k]$, and a witness w_i .

[0277] Definition 5.3 (Batch Argument for Policies with Functional Subset Extraction). A F-batch argument with G-functional subset extraction ($\text{Setup}, \mathcal{P}, \mathcal{V}, \text{Extract}$) for relation $\mathcal{R}$ is required to satisfy the following properties:

[0278] Completeness. For any $\lambda \in \mathbb{N}$, any $k, n, m, \ell, T \leq 2^\lambda$, any instance $x=(f, M, z, T) \in \text{Monotone PolicyTMSAT}$ with $|M|+|z|=\pi$ and corresponding witness w such that $(x, w) \in \mathcal{R}$ and $|w_i|=m$, any function $g \in G_\lambda$, and any function input $y \in \{0, 1\}^\ell$,

$$\Pr \left[\mathcal{V}(\text{crs}, \mathcal{V}^f, y, x, \pi) = 1 : \begin{array}{l} (\text{crs}, \text{td}) \leftarrow \text{Setup}(1^\lambda, g) \\ \pi \leftarrow \mathcal{P}(\text{crs}, y, f, M, z, 1^T, w) \end{array} \right] = 1.$$

[0279] Succinctness. In the completeness experiment above, $|\pi| \leq m \cdot \text{poly}(\lambda)$ and $|\text{crs}| \leq (m + |g|) \cdot \text{poly}(\lambda)$ where $|g|$ is the description length of the function g .

[0280] Key Indistinguishability. For any poly-size adversary $\mathcal{A}$, any polynomial $T(\lambda)$ and any functions $g_1, g_2 \in G_\lambda$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$

$$\Pr \left[\mathcal{A}(\text{crs}) = b : \begin{array}{l} b \leftarrow \{0, 1\}, \\ (\text{crs}, \text{td}) \leftarrow \text{Setup}(1^\lambda, g_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

[0281] G-Functional Subset Extraction. For any polynomial $T(\lambda)$ and any poly-size cheating prover $\mathcal{P}^*$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and $g \in G_\lambda$

$$\Pr_{\text{EXP}} \left[\begin{array}{l} \mathcal{V}(\text{crs}_V, y, x, \pi) = 1 \wedge \\ f(\mathbb{1}_{[k] \cup J}) = 0 \wedge \\ (i \notin J \vee M(z, i, w_i) = 0) \end{array} \right] \leq \text{negl}(\lambda).$$

[0282] where EXP is the experiment defined as follows:

[0283] Generate $(\text{crs}, \text{td}) \leftarrow \text{Setup}(1^\lambda, g)$.

[0284] Run the cheating prover and obtain $(M, z, \pi, y) \rightarrow \mathcal{P}^*(\text{crs})$.

[0285] Let $x = (f_\lambda, M, z, T)$ and $J = g(y)$.

[0286] Extract $(i, w_i) \leftarrow \text{Extract}(\text{td}, y, \pi)$.

[0287] Remark 5.1 (Hashed inputs). We additionally support a syntax where the verifier receives a hash of the function input y (for some hash family with local opening), rather than the full input. This allows for an efficient verifier in case the function input is long. Looking ahead, this will be the case in our construction in Section 5.4.

[0288] With this syntax, security is modified so that it holds if the verifier receives an honestly computed hash of y . We note that this notion can be obtained generically, using RAM SNARGs (where the BARG proof contains a RAM SNARG proof that the verifier accepts). A similar transformation was implemented in prior work, to obtain a predicate-extractable hash family with efficient verification.

5.3 From Adaptive Subset Extraction to Aggregate Signatures

[0289] We present our first transformation, where we construct aggregate signatures from BARGs with adaptive subset extraction, and any signature scheme. Specifically, let $F = \{F_\lambda\}$ be a family of monotone policies. We construct an F-aggregation scheme for S (Definition 4.1) from the following building blocks:

[0290] A digital signature scheme $S = (\text{KeyGen}, \text{Sign}, \text{Verify})$.

[0291] An F-batch argument with α -adaptive subset extraction (Definition 5.2)

[0292] $(\text{Setup}_{\text{BARG}}, \mathcal{P}_{\text{BARG}}, \mathcal{V}_{\text{BARG}}, \text{Extract}_{\text{BARG}})$ for relation $\mathcal{R}$, where α is some polynomial. For simplicity we assume $\mathcal{R} = \mathcal{R}_{\text{full}}$. We note that a

similar construction with $\mathcal{R} = \mathcal{R}_{\text{sub}}$ gives a F-aggregation scheme with sublinear aggregation (Section 4.1).

[0293] A hash family with local opening (Definition 3.2)

[0294] $(\text{Gen}_{\text{HT}}, \text{Hash}_{\text{HT}}, \text{Open}_{\text{HT}}, \text{Verify}_{\text{HT}})$.

[0295] We describe the aggregate signature algorithms herein.

[0296] Theorem 5.4. Assuming the existence of a digital signature scheme, hash family and F-batch argument with α -adaptive subset extraction, the construction given herein is an F-aggregation scheme for S .

5.4 From Functional Subset Extraction to Weakly Unforgeable Aggregate Signatures

[0297] We present our second transformation, where we construct weakly unforgeable aggregate signatures from BARGs with functional subset extraction. Our construction will require signature schemes with an additional property that we call signature scheme with trapdoor keys. We start by describing this notion, before we move to the construction of the aggregate signature scheme.

5.4.1 Signature with Trapdoor Keys

[0298] We extend the definition of digital signatures $S = (\text{KeyGen}, \text{Sign}, \text{Verify})$ with an additional property of trapdoor verification keys. This is a requirement on the key generation algorithm of a signature scheme that when provided with a randomly sampled trapdoor $\text{td} \leftarrow \{0, 1\}^\lambda$, we can generate “trapdoor keys”. Such trapdoor keys are indistinguishable from honest keys, but their verification keys vk can be recognized using the corresponding trapdoor. Additionally, without the trapdoor, it is hard to forge new “false positive” verification keys vk that are recognized as trapdoor keys, even if given a polynomial number of trapdoor key samples.

[0299] Definition 5.5 (Signature with Trapdoor Keys). A digital signature scheme $S = (\text{KeyGen}, \text{Sign}, \text{Verify})$ has trapdoor keys if it additionally supports the following syntax:

[0300] $\text{KeyGen}(1^\lambda, \text{td}) \rightarrow (\text{sk}, \text{vk})$. This is a PPT algorithm that takes as input the security parameter 1^λ and a trapdoor $\text{td} \in \{0, 1\}^\lambda$. It outputs a signing key sk and a verification key vk .

[0301] $\text{Extract}(\text{td}, \text{vk}) \rightarrow \{0, 1\}$. This is a deterministic polynomial-time algorithm that takes as input a trapdoor $\text{td} \in \{0, 1\}^\lambda$ and a verification key vk . It outputs a bit (0 if it is a normal key, 1 if it is a trapdoor key).

[0302] We additionally require the following properties:

[0303] Key Indistinguishability. For any poly-size adversary $\mathcal{A}$, and for any polynomial $n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\mathcal{A}(\{sk_i^b, vk_i^b\}_{i \in [n]}) = b : \begin{array}{l} \text{td} \leftarrow \{0, 1\}^\lambda \\ \forall i \in [n], (sk_i^0, vk_i^0) \leftarrow \text{KeyGen}(1^\lambda, \text{td}) \\ \forall i \in [n], (sk_i^1, vk_i^1) \leftarrow \text{KeyGen}(1^\lambda) \\ b \leftarrow \{0, 1\} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

[0304] We remark that this property implies that correctness and unforgeability holds for keys sampled by $\text{KeyGen}(1^\lambda, \text{td})$ for a random $\text{td} \leftarrow \{0, 1\}^\lambda$.

[0305] Extraction Correctness. For any $\lambda \in \mathbb{N}$

$$\Pr \left[\text{Extract}(td, vk) = 1 : \begin{array}{l} td \leftarrow \{0, 1\}^\lambda \\ (sk, vk) \leftarrow \text{KeyGen}(1^\lambda, td) \end{array} \right] = 1.$$

[0306] Additionally, for any poly-size adversary $\mathcal{A}$, and for any polynomial $n(\lambda)$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} \text{Extract}(td, vk) = 1 \wedge \\ vk \notin \{vk_i\}_{i \in [n]} \end{array} : \begin{array}{l} td \leftarrow \{0, 1\}^\lambda \\ \forall i \in [n], (sk_i, vk_i) \leftarrow \text{KeyGen}(1^\lambda, td) \\ vk \leftarrow \mathcal{A}(1^\lambda, \{sk_i^b, vk_i^b\}_{i \in [n]}) \end{array} \right] \leq \text{negl}(\lambda).$$

[0307] We note that given a signature scheme $S=(\text{KeyGen}, \text{Sign}, \text{Verify})$, we can construct a signature scheme S' with trapdoor keys, by appending a random padding $\text{pad} \leftarrow \{0, 1\}^\lambda$ to its verification keys. To generate a trapdoor key $(sk', vk') \leftarrow \text{KeyGen}'(1^\lambda, td)$, we instead append a pseudorandom padding $\text{pad} = \text{PRF}(td, vk)$. Extraction is implemented by comparing pad to $\text{PRF}(td, vk)$, and the required properties follow from PRF security. We state the following lemma without proof for the above stated construction, and note that the proof follows in a straightforward manner akin to the construction of message authentication codes (MAC) from PRF: (i) extraction correctness follows from the guarantee that the PRF behaves like a MAC on the verification key; and (ii) key indistinguishability follows from the pseudorandomness of the PRF.

[0308] Lemma 5.6. Assuming the existence of a PRF and a signature scheme, there exists a signature scheme with trapdoor keys.

5.4.2 Construction

[0309] We now describe our construction that uses BARGs with functional subset extraction and a signature scheme with trapdoor keys. Specifically, let $\mathcal{F}=\{\mathcal{F}_\lambda\}$ be a family of monotone policies. We construct an F-aggregation scheme with weak unforgeability for $\mathcal{S}$ (Definition 4.2) from the following building blocks:

[0310] A signature scheme with trapdoor keys $S=(\text{KeyGen}, \text{Sign}, \text{Verify})$ (Definition Definition 5.5).

[0311] A F-batch argument with G-functional subset extraction for relation $\mathcal{R}_{full}$ with hashed inputs (Definition 5.3 and Remark 5.1)

[0312] $\text{Setup}_{BARG}, \mathcal{P}_{BARG}, \mathcal{V}_{BARG}, \text{Extract}_{BARG}$,

[0313] where $G=\{G_\lambda\}_{\lambda \in \mathbb{N}}$ is a family of polynomial size circuits, with a polynomial bound set to support computing the Extract algorithm of the underlying signature scheme with trapdoor keys.

[0314] A hash family with local opening (Definition 3.2)

[0315] $(\text{Gen}_{HT}, \text{Hash}_{HT}, \text{Open}_{HT}, \text{Verify}_{HT})$.

[0316] We describe the aggregate signature algorithms. The construction is identical to Section 5.3, except that the BARG algorithms now receive a function and tag as needed, where we program the null function (later modified in the security proof), and our tag is the tuple of verification keys

$(vk_1, \dots, vk_k)$ (or the hashed aggregate verification keys $\widehat{vk}$ for the verifier, following the syntax in Remark 5.1).

[0317] Theorem 5.7. Assuming the existence of a digital signature scheme with trapdoor keys, hash family and F-batch argument with G-functional subset extraction, the construction given herein is an F-aggregation scheme for $\mathcal{S}$.

6 Composable Verifiable Private Information Retrieval for Policies

[0318] In this section, we introduce and construct the main building block in the construction of our BARGs with adaptive subset extraction for read-once monotone policies, composable verifiable private information retrieval (vPIR). Our notion of composable vPIR extends the notion of vPIR to handle a composable definition of security, and additionally monotone policies, both of which is necessary to construct our BARGs with adaptive subset extraction for read-once monotone policies in Section 7.

[0319] We start by defining of composable vPIR for policies in Section 6.1. In Section 6.2 we give a construction of vPIR that supports our extensions, and in Section 6.3 we extend the analysis of prior work to show that this construction satisfies our new composable security definition. We first briefly describe vPIR for policies, and how it differs from the notion considered in prior work.

[0320] vPIR for policies. The prior work considers vPIR for bounded space global database constraints. Such constraints are evaluated by reading the rows of the database one by one and maintaining a state of bounded size between rows. In the adaptive setting, where the adversary may choose the constraint as a function of the vPIR query, the security of their construction degrades exponentially with the description length of the constraint (given by a program that updates state). However, in the non-adaptive setting, their construction does not incur such loss.

[0321] We consider a particular type of bounded space global constraints that are suitable for describing policies. The description of such constraints can be separated into two parts: a program Γ and a vector of instances X , one for every row in the database. The program updates the state as a function of both the current row and the corresponding instance. Looking ahead, in our application the instances may be chosen adaptively. Therefore, relying on the construction from prior work would result in exponential security loss. However, in our instantiation the security loss is only exponential in the description length of the program Γ and not of the instances. Moreover, in the adaptive setting, the simulator in the definition from prior work is required to produce the entire constraint (Γ, X) (this constraint together with the simulated client's output row $D[t]$ is required to be indistinguishable from the constraint and output in the real world), while our notion is relaxed: we only simulate the constraint Γ and the relevant instance $X[t]$.

6.1 Definition

[0322] We define a simulation security notion for multiple parallel executions of vPIR. While one could consider general forms of composition, we focus on the setting where a single client is sending a single vPIR query to p (potentially colluding) servers, each holding a different database and constraint. In this setting, we require the simulator to produce the output of the client (including the database row and constraint) in all p interactions.

[0323] For simplicity, in Definition 6.1 we simplify this further and only consider the case of $p=2$ colluding servers, which suffices for our applications to BARGs with adaptive subset extraction and aggregate signatures. However we note that the definition can be naturally extended to define p -composable vPIR.

[0324] Read-once bounded-space policy constraints. For parameters T, S , we define a read-once bounded-space constraint represented as a Turing machine $\Gamma \in \{0, 1\}^N$ and instances $X = (x_i)_{i \in [k]} \in \{0, 1\}^{k \times n}$. We say that a database $D = (r_i)_{i \in [k]} \in \{0, 1\}^{k \times w}$ satisfies the policy defined by (Γ, X) if and only if for every $i \in [k-1]$ there exists $c_i \in \{0, 1\}^S$ such that $\Gamma(c_{i-1}, x_i, r_i)$ outputs c_i within T steps where c_0, c_k are some fixed starting and accepting configurations. We denote by $U_{T,S}(\Gamma, X, D)$ the bit that indicates whether or not D satisfies the policy (Γ, X) .

[0325] Syntax. A verifiable private information retrieval (vPIR) scheme for policies consists of the following polynomial-time algorithms:

[0326] $\text{Query}(1^\lambda, t) \rightarrow (dk, vk, q)$. This is a probabilistic algorithm that takes as input the security parameter 1^λ , and a row index $t \in [2^\lambda]$. It outputs a decryption key dk , a verification key vk and a query q .

[0327] $\text{Answer}(D, 1^T, \Gamma, X, q) \rightarrow a$. This is a deterministic algorithm that takes as input a database $D \in \{0, 1\}^{k \times w}$, a time-bound 1^T , a constraint Γ , instances $X \in \{0, 1\}^{k \times n}$, and a query q . It outputs an answer a .

[0328] $\text{Dec}(dk, a) \rightarrow r$. This is a deterministic algorithm that takes as input a decryption key dk , and an answer a . It outputs a row $\Gamma \in \{0, 1\}$.

[0329] $\text{Verify}(vk, \Gamma, X, a) \rightarrow 0/1$. This is a deterministic algorithm that takes as input a verification key vk , a constraint Γ , instances $X \in \{0, 1\}^{k \times n}$, and an answer a . It outputs a bit (1 to accept, 0 to reject).

[0330] Remark 6.1 (Index instances). In our constructions in Section 7, we only need to consider instances X of the form $X = (x, i)_{i \in [k]}$ for some $x \in \{0, 1\}^n$ that is the same in all rows. We note that in this case, the Answer, Verify algorithms may receive just x rather than $X = (x, i)_{i \in [k]}$ (and so the verification time is independent of k).

[0331] Definition 6.1. A Λ -secure 2-composable vPIR scheme for policies (Query, Answer, Dec, Verify) is required to satisfy the following properties:

[0332] Completeness. For any $\lambda \in \mathbb{N}$, any $T, N, S, k, n, w \leq 2^\lambda$, database $D \in \{0, 1\}^{k \times w}$, row index $t \in [k]$, constraint $\Gamma \in \{0, 1\}^N$ and instances $X \in \{0, 1\}^{k \times n}$ such that $U_{T,S}(\Gamma, X, D) = 1$,

$$\Pr \left[\begin{array}{l} \text{Dec}(dk, a) = D[t] \wedge \\ \text{Verify}(vk, \Gamma, X, a) = 1 \end{array} : \begin{array}{l} (dk, vk, q) \leftarrow \text{Query}(1^\lambda, t) \\ a \leftarrow \text{Answer}(D, 1^T, \Gamma, X, q) \end{array} \right] = 1.$$

[0333] Efficiency. In the completeness experiment above, $|vk| + |q| + |a| \leq w \cdot \text{poly}(\lambda)$.

[0334] Λ -Privacy. For any $\text{poly}(\Lambda)$ -size adversary $\mathcal{A}$ there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ and row indices $t_0, t_1 \in [2^\lambda]$,

$$\Pr \left[\mathcal{A}(vk, q) = b : \begin{array}{l} b \leftarrow \{0, 1\} \\ (dk, vk, q) \leftarrow \text{Query}(1^\lambda, t_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

[0335] 2-Composable Λ -simulation security. For any functions $T(\lambda), N(\lambda), n(\lambda), k(\lambda), w(\lambda) \leq \Lambda(\lambda)$, function $S(\lambda) = O(\log \Lambda)$, any $\text{poly}(\Lambda)$ -size adversary $\mathcal{A}$ and polynomial P there exists a $\text{poly}(\Lambda)$ -size simulator Sim such that for any $\text{poly}(\Lambda)$ -size distinguisher $\mathcal{D}$, $\lambda \in \mathbb{N}$, constraints $\Gamma, \Gamma' \in \{0, 1\}^N$, and row index $t \in [k]$,

$$\left| \Pr \left[\mathcal{D}(\text{Real}_{\mathcal{A}}(\lambda, \Gamma, \Gamma', t)) = 1 \right] - \Pr \left[\mathcal{D}(\text{Ideal}_{\text{Sim}}(\lambda, \Gamma, \Gamma', t)) = 1 \right] \right| < \frac{1}{P(\Lambda)},$$

[0336] where the experiments $\text{Real}_{\mathcal{A}}(\lambda, \Gamma, \Gamma', t)$ and $\text{Ideal}_{\text{Sim}}(\lambda, \Gamma, \Gamma', t)$ are defined as follows:

[0337] $\text{Real}_{\mathcal{A}}(\lambda, \Gamma, \Gamma', t)$:

[0338] Generate a query $(dk, vk, q) \leftarrow \text{Query}(1^\lambda, t)$.

[0339] Run the adversary and obtain $(X, X', a, a') \leftarrow \mathcal{A}(vk, q)$.

[0340] If $\text{Verify}(vk, \Gamma, X, a) = 1$ and $\text{Verify}(vk, \Gamma', X', a') = 1$ output $(X[t], X'[t], \text{Dec}(dk, a), \text{Dec}(dk, a'))$.

[0341] Otherwise output $\perp$.

[0342] $\text{Ideal}_{\text{Sim}}(\lambda, \Gamma, \Gamma', t)$:

[0343] Run the simulator and obtain $(X, X', D, D') \leftarrow \text{Sim}(\lambda, \Gamma, \Gamma')$.

[0344] If $U_{T,S}(\Gamma, X, D) = 1$ and $U_{T,S}(\Gamma', X', D') = 1$ output $(X[t], X'[t], D[t], D'[t])$.

[0345] Otherwise output $\perp$.

6.2 Construction

[0346] In this section we construct a 2-composable vPIR for policies. The construction is similar to the construction given in prior work, slightly modified to handle our extensions. It uses the following building blocks:

[0347] A Λ -secure hash family with local opening (Definition 3.2)

[0348] $(\text{Gen}_{HT}, \text{Hash}_{HT}, \text{Open}_{HT}, \text{Verify}_{HT})$.

[0349] A Λ -secure seBARG scheme (Definition 3.5)

[0350] $(\text{Gen}_{\text{seBARG}}, \mathcal{P}_{\text{seBARG}}, \mathcal{V}_{\text{seBARG}}, \text{Extract}_{\text{seBARG}})$.

[0351] We are now ready to describe our 2-composable vPIR scheme for policies.

6.3 Analysis

[0352] In this section we analyze the construction given in Section 6.2, and prove the following theorem:

[0353] Theorem 6.2. Assuming a Λ -secure hash family with local opening and a Λ -secure somewhere-extractable batch argument scheme, there exists a Λ -secure 2-composable vPIR scheme for policies.

[0354] Our analysis, detailed below, is similar to the analysis of simulation secure vPIR in prior work, with some parts taken verbatim from the full version. We extend it to handle policy constraints, and 2-composable simulation security.

[0355] Before proceeding with the formal proof, we describe the high-level ideas in our analysis. We first recall the simulator given in prior work. It simulates an accepting database using coupling: for each row index in the database $i \in [k]$, it repeatedly queries the server for this index and constructs a list L_i of tuples (c_{i-1}, r_i) , where c_{i-1} is the intermediate configuration of the bounded-space constraint Γ before reading the corresponding row r_i . Then, it couples

the lists $L_1, \dots, L_k$ to create a list L of full databases, so that the intermediate configurations “connect” to a consistent execution of Γ . Finally it samples a random database from this list.

[0356] We now describe the changes in our analysis. To support policies (where we also need to simulate instances $X \in \{0, 1\}^{k \times n}$), our simulator constructs the list L_i so that it contains tuples (c_{i-1}, x_i, r_i) (where x_i is the i th instance given by the prover in this query), then couples the lists in the same way. To extend the analysis to composition of p parallel executions, each element in the list L_i is a p -tuple of $(c_{i-1}^j, x_i^j, r_i^j)_{j \in [p]}$, with one configuration-instance-row tuple corresponding to each server. We then couple $L_1, \dots, L_k$ to obtain a list L such that each individual database connects.

[0357] We note that, in order for the coupling argument to work, we need to assume hardness assumptions that are exponential in the support size of the function according to which we’re coupling (which, in our case, is the combined size of all intermediate configurations cf_i in the p parallel executions, for any given i). In prior work this is what limits the space S of the constraint to $O(\log \Lambda)$, and in our case the limit is $p \cdot S = O(\log \Lambda)$, assuming Λ -hardness of the underlying building blocks.

[0358] Remark 6.2. Finally, we note that the above construction and analysis extend to any p -composable vPIR for $p = O(1)$ (or, more generally, to $p \cdot S = O(\log \Lambda)$ assuming Λ -security of the underlying building blocks, where S is the space bound of the constraint Γ).

7 BARGs with Adaptive Subset Extraction for Bounded-Space Policies

[0359] In this section, we use our constructed 2-composable vPIR for policies (Definition 6.1) to construct batch arguments with adaptive subset extraction.

[0360] We proceed by defining the families of policies we support and stating our main theorems.

[0361] Definition 7.1 (Read-Once Space- S Policies). Let $S(\lambda)$ be a function and let $F = \{F_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of functions. F is a read-once space- S family of policies if there exist polynomials $T(\lambda), N(\lambda), k(\lambda)$ such that each $f \in F_\lambda$ is a monotone function $f: \{0, 1\}^k \rightarrow \{0, 1\}$, and there exists a Turing machine $\Gamma \in \{0, 1\}^N$ such that $f(b_1, \dots, b_k) = 1$ if and only if for every $i \in [k-1]$ there exists $c_i \in \{0, 1\}^S$ such that $\Gamma(c_{i-1}, b_i)$ outputs c_i within T steps where c_0, c_k are some fixed starting and accepting configurations.

[0362] Theorem 7.2 (Section 7.1). Let $\Lambda = \Lambda(\lambda)$ be a function. Assuming a Λ -secure 2-composable vPIR scheme for policies, there exists a scheme $\text{BARG} = (\text{Setup}, \mathcal{P}, \mathcal{V}, \text{Extract})$ such that for any family F of read-once space S policies where $S = O(\log \Lambda)$, BARG is a F -batch argument with $O(k)$ -adaptive subset extraction for relation $\mathcal{R}_{\text{full}}$.

[0363] Using Theorem 5.4 and Theorem 7.2 above, we obtain the following theorem on aggregate signatures.

[0364] Theorem 7.3. Let $\Lambda = \Lambda(\lambda)$ be a function. Assuming a Λ -secure 2-composable vPIR scheme for policies and a Λ -secure hash family with local opening, there exists a scheme $\text{AggS} = (\text{Setup}, \text{KeyAgg}, \text{SigAgg}, \text{AggVerify})$ such that for any family F of read-once space S policies where $S = O(\log \Lambda)$ and for any digital signature scheme S , AggS is a F -aggregation scheme for S .

[0365] We define below a specific family of monotone function, that of threshold policies.

[0366] Definition 7.4 (Weighted Threshold Policies). Let $F = \{F_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of functions. F is a family of weighted threshold policies if there exist polynomials $k(\lambda)$,

$\{a_i(\lambda)\}_{i \in [k]}$, $t(\lambda)$ such that each $f \in F_\lambda$ is a function $f: \{0, 1\}^k \rightarrow \{0, 1\}$ such that $f(b_1, \dots, b_k) = 1$ if and only if $b_i \in \{0, 1\}$ and $\sum_{i=1}^k a_i b_i \geq t$.

[0367] Theorem 7.5 (Section 7.2). Assuming a (polynomially-secure) 2-composable vPIR scheme for policies, there exists a scheme $\text{BARG} = (\text{Setup}, \mathcal{P}, \mathcal{V}, \text{Extract})$ such that for any family F of weighted threshold policies with threshold t , BARG is a F -batch argument with $O(t)$ -adaptive subset extraction for relation $\mathcal{R}_{\text{sub}}$ (where the aggregation time is polynomial in t).

[0368] Thus, again using Theorem 5.4 and the Theorem 7.5 we have the following theorem.

[0369] Theorem 7.6. Assuming a 2-composable vPIR scheme for policies and a hash family with local opening, there exists a scheme $\text{AggS} = (\text{Setup}, \text{KeyAgg}, \text{SigAgg}, \text{AggVerify})$ such that for any family F of weighted threshold policies with threshold t and for any digital signature scheme S , AggS is a F -aggregation scheme for S with aggregation time polynomial in t .

[0370] Note that by setting $\alpha_i = 1$, in the above theorems, we revert to the (unweighted) threshold setting, where the aggregation time now depends on the number of signatures t .

7.1 Adaptive Subset Extraction for Bounded-Space Policies

7.1.1 Construction.

[0371] Our construction uses as a building block a Λ -secure 2-composable vPIR scheme for policies

[0372] (Query, Answer, Dec, Verify),

where we use the syntax for index instances (as in Remark 6.1).

7.2 Adaptive Subset Extraction with Sublinear Prover for Threshold Policies

[0373] In this section, we use 2-composable vPIR to construct batch arguments with adaptive subset extraction for the sublinear relation $\mathcal{R}_{\text{sub}}$.

[0374] On sublinear provers for general monotone policies. We begin by describing a general approach towards achieving a sublinear prover for any read-once bounded-space policy. The idea is to consider a compressed vPIR database, corresponding to the $\mathcal{R}_{\text{sub}}$ -witness: suppose the witness is $(i_1, w_1, \dots, i_t, w_t)$, where $i_1, \dots, i_t$ are sorted in increasing order. We consider the vPIR database that contains t rows, each containing an index-witness pair. Then, the BARG verifier would verify that this database satisfies the constraint corresponding to the validity of this witness: (i) the indices are sorted, (ii) all witnesses are valid, and (iii) the input $b_1, \dots, b_k$ where $b_i = 1 \Leftrightarrow \exists j. i = j$, satisfies the policy f .

[0375] First, we observe that for general bounded-space policies, it is not clear that the prover can evaluate this constraint on the database in sublinear time (i.e. in time that only depends on the compressed database size and not on k). Indeed, the difficulty lies in (iii) above: to update the function’s state between rows j and $j+1$, the prover needs to compute $i_{j+1} - i_j$ steps of the function, which could take up to linear time. We propose two ways to handle this issue:

[0376] First, we can restrict the class of policies f to ones with a “fast-forward” property, where one can update f ’s state reading any number of repeated zeroes in time that depends sublinearly in this number. For example, weighted threshold policies satisfy this property (in fact, zeroes do not change their state at all).

[0377] Alternatively, we can also support general monotone policies, at the expense of having a long crs, which the prover has random access to. This crs could contain a table of states of f after any number of zeroes, which the prover can later use instead of performing the computation during proof generation (we note that in order to convince the verifier, the long crs should also contain SNARG proofs that each table entry is correct).

[0378] We next discuss security of our candidate construction, and whether we can obtain it from our 2-composable vPIR notion.

[0379] We start by looking at the case of threshold policies. Similarly to our construction in Section 7.1, given a cheating prover that outputs a vPIR answer a and a necessary subset J , we consider a composed prover that gives an additional vPIR answer based on the database corresponding to this subset J , and the constraint that checks that it is necessary, i.e. that $|J| > k - t$. To use composition, we encode J as a database D' of size t (matching the size of the prover's database), consisting of rows $(\tilde{i}_{j-1}, \tilde{i}_j)$ where $\tilde{i}_j$ are such that the edges are $\tilde{i}_0 = 0$ and $\tilde{i}_t = k + 1$, and the points cover J , i.e. $J \subseteq \{\tilde{i}_1, \dots, \tilde{i}_{t-1}\}$.

[0380] Given this cheating prover, our composition theorem guarantees a simulator for the joint distribution of the databases $D = (i_j, w_j)_{j \in [t]}$ and $D' = (\tilde{i}_{j-1}, \tilde{i}_j)_{j \in [t]}$. Now, we use the combinatorial property that every pair of valid databases D, D' must contain a row j such that i_j is contained in the open interval $(\tilde{i}_{j-1}, \tilde{i}_j)$, which implies that $i_j \in J$. This gives us the adaptive subset extraction property: sampling a random row in the database, with probability $1/k$ we're guaranteed to find a row j such that w_j is valid and $i_j \in J$.

[0381] This analysis also extends to weighted threshold policies, by expanding the database to contain multiple copies of each witness, corresponding to the weight. Unfortunately, it seems that the same approach doesn't extend to general bounded-space policies. Instead, we can argue security using a direct analysis, utilizing additional properties of the vPIR construction in Section 6 that do not seem to be captured by our composition theorem.

[0382] In a nutshell, the issue is with defining the encoding of J as a database. For the case of thresholds, we exploited the fact that the complement of a necessary subset must be smaller in size than any satisfying subset to the policy, to encode J as a sequence of t open intervals in J . For general policies, where this fact does not hold, it's not clear how to encode J . We could try to compress J to t rows by encoding multiple such intervals in each row, but for any fixed choice of the number of intervals in each row, we could find a set of valid databases where our combinatorial property doesn't hold, i.e. there is no row j such that the index i_j is contained in some open interval in the same row.

[0383] To solve this problem, we can open up the proof of our composition theorem, and define a "dynamic" database structure for J that depends on the rows (i_j, w_j) in D to choose precisely how many elements of J to include in each row. In more detail, using the same simulation strategy as in Section 6 we can repeatedly query $\mathcal{P}^*$ on each index j , by running $\mathcal{P}^*(q) \rightarrow (a, J)$ for $(dk, vk, q) \leftarrow \text{Query}(1^\lambda, j)$ and decrypting the database row $(i_{j-1}, i_j, w_j) = r_j = \text{Dec}(dk, a)$ (we include the previous index i_{j-1} as part of each database row). Then, we construct a list L_j of answers of the form $(i_{j-1}, i_j, w_j, J \cap (i_{j-1}, i_j])$. Finally, using the coupling lemma, we can couple together the lists $L_1, \dots, L_t$ according to $(i_{j-1}, i_j, w_j)_{j \in [t]}$ being an accepting database (i.e., i_j that appears in the j th and

$j+1$ -th rows should be equal, and w_j should all be valid witnesses), and the set J composed of the intervals $J \cap (i_{j-1}, i_j]$ not satisfying the policy, i.e. J being a necessary subset (note these two conditions can be checked with a bounded size state between rows).

[0384] We then have that in any such pair of databases, since the set of witnesses satisfy the policy and J is a necessary subset, there must exist an index $j \in J$ such that w_j is an accepting witness for the i_j th statement, and $i_j \in J$. But now, since we defined J 's database structure such that its rows $J \cap (i_{j-1}, i_j]$ exactly match the indices of valid witnesses, we have that just row j in the simulated pair of databases is enough to decide if J contains the index i_j . Thus this property is maintained in the real world experiment, and we get $O(t)$ -adaptive subset extraction by extracting at a random row index j .

7.2.1 Construction.

[0385] Our construction uses as a building block a composable polynomially-secure vPIR scheme for policies

[0386] (Query, Answer, Dec, Verify), where we use the syntax for index instances (as in Remark 6.1).

8 BARGs with Functional Subset Extraction for Monotone Circuit Policies

[0387] In this section, we construct functional subset extraction for monotone functions that can be represented as polynomial size circuits. We state the following theorem, and provide a proof sketch since analysis follows identically to the analysis presented in prior work.

[0388] Theorem 8.1. Assuming a somewhere-extractable batch argument scheme (Definition 3.5) and fully homomorphic encryption, there exists a scheme $\text{BARG} = (\text{Setup}, \mathcal{P}, \mathcal{V}, \text{Extract})$ such that for any family F of monotone circuit policies and family G of functions computable by polynomial-size circuits, BARG is a F -batch argument with G -functional subset extraction for relation $\mathcal{R}_{\text{full}}$ with hashed inputs.

[0389] Combining Theorem 5.7 and Theorem 8.1 above, we have the following theorem about weakly unforgeable aggregate signatures.

[0390] Theorem 8.2. Assuming a somewhere-extractable batch argument scheme (Definition 3.5) and fully homomorphic encryption, there exists a scheme $\text{AggS} = (\text{Setup}, \text{KeyAgg}, \text{SigAgg}, \text{AggVerify})$ such that for any family F of functions computable by polynomial-size monotone circuits and for any digital signature scheme S with trapdoor keys, AggS is a F -aggregation scheme for S .

8.1 Construction and Proof Sketch

[0391] In this section we give a proof sketch for Theorem 8.1. We recall the construction of batch arguments for monotone circuit policies from prior work, and show that it can support functional subset extraction (defined in Section 5.2).

[0392] In Section 8.1.1, we recall the notion of a predicate-extractable hash, and extend the definition to support functional predicate extraction. This extension was in fact already used in prior work, to get a shorter common reference string for their SNARG.

[0393] Then, in Section 8.1.2 we recall the construction of a somewhere-extractable SNARG for monotone policy BatchNP from prior work. We show that instantiating this

construction with a functional predicate-extractable hash, we get a batch argument with functional subset extraction.

8.1.1 Predicate Extractable Hash (PEHash)

[0394] In this section we define predicate-extractable hash with tags for bit-fixing predicates. The definition is taken from prior work. This is a notion that generalizes the notion of a somewhere-extractable hash. Whereas a somewhere-extractable hash allows to generate a hash key hk , programmed on an index i and use a corresponding trapdoor td , to extract x_i from $rt = \text{Hash}(hk_i, x)$, a predicate-extractable hash for a family $\mathcal{F}$ allows to generate a hash key programmed on a global predicate $f \in \mathcal{F}$, and extract the value of $f(x)$. Prior work constructs such hash families for the bit-fixing family of predicates, where each $f \in \mathcal{F}$ is defined by a set $J \subseteq [k]$ and a string $s \in \{0, 1\}^J$, and we have $f(x) = 1 \text{ iff } \forall j \in J, x_j = s_j$.

[0395] We consider an extended definition that supports functional predicate extraction. This allows to program the PEHash on a function g during key generation (rather than on a predicate specified by J, s), then specify an input y to the function when hashing a string x . This produces a hash value, from which we can extract the value of the predicate $f = g(y)$ on x .

[0396] Syntax. Let $G = \{G_\lambda\}_{\lambda \in \mathbb{N}}$ be a collection of functions such that every $g \in G_\lambda$ maps inputs to bit-fixing predicates f . For $g \in G_\lambda$, we denote its description length as a circuit by $|g|$.

[0397] A G -functional predicate extractable hash family PEHash with tags with respect to the bit-fixing predicate family consists of the following polynomial-time algorithms:

[0398] $\text{Gen}(1^\lambda, g) \rightarrow (hk, vk, td)$. This is a probabilistic setup algorithm that takes as input a security parameter 1^λ in unary, and a function $g \in G_\lambda$. It outputs a hash key hk , verification key vk and trapdoor td .

[0399] $\text{Hash}(hk, y, x, \vec{t}) \rightarrow v$. This is a deterministic algorithm that takes as input a hash key hk , a function input y , a hash input $x \in \{0, 1\}^N$, and tags $\vec{t} = (t_1, \dots, t_N) \in (\{0, 1\}^T)^N$. It outputs a hash value $v \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$.

[0400] $\text{Open}(hk, y, x, \vec{t}, j) \rightarrow p$. This is a deterministic algorithm that takes as input a hash key hk , a function input y , a hash input $x \in \{0, 1\}^N$, tags $\vec{t} = (t_1, \dots, t_N) \in (\{0, 1\}^T)^N$ and an index $j \in [N]$. It outputs an opening $p \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$.

[0401] $\text{Verify}(vk, y, v, j, b, t, p) \rightarrow 0/1$. This is a deterministic algorithm that takes as input a verification key vk , a function input y , a hash value $v \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$, an index $j \in [N]$, a bit $b \in \{0, 1\}$, a tag $t \in \{0, 1\}^T$ and an opening $p \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$, and outputs 1 (accept) or 0 (reject).

[0402] $\text{Extract}(td, y, v) \rightarrow u$. This is a deterministic extraction algorithm that takes as input a trapdoor td , a function input y and a hash value $v \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$, and outputs a bit $u \in \{0, 1\}$.

[0403] $\text{ExtractIndex}(td, y, v) \rightarrow j$. This is a deterministic extraction algorithm that takes as input a trapdoor td , a function input y and a hash value $v \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$, and outputs an index $j \in [N]$.

[0404] $\text{ExtractTag}(td, y, v) \rightarrow t$. This is a deterministic extraction algorithm that takes as input a trapdoor td , a function input y and a hash value $v \in \{0, 1\}^{T \cdot \text{poly}(\lambda)}$, and outputs a tag $t \in \{0, 1\}^T$.

[0405] A PEHash is required to satisfy the following basic properties, and additional consistency properties.

[0406] Definition 8.3 (PEHash Basic Properties). A G -functional predicate extractable hash family PEHash satisfies the following properties:

[0407] Completeness. For any $\lambda \in \mathbb{N}$, any $N, T, \ell \leq 2^\lambda$, any function $g \in G_\lambda$, any function input $y \in \{0, 1\}^\ell$, any index $j \in [N]$, and any $x \in \{0, 1\}^N$ and tags $\vec{t} = (t_1, \dots, t_N) \in (\{0, 1\}^T)^N$ let $f = g(y)$, then

$$\Pr \left[\begin{array}{l} \text{Extract}(td, v) = f(x) \wedge \\ \text{Verify}(vk, y, v, j, x_j, t_j, \rho) = 1 \\ \rho = \text{Open}(hk, y, x, \vec{t}, j) \end{array} : \begin{array}{l} (hk, vk, td) \leftarrow \text{Gen}(1^\lambda, g), \\ v = \text{Hash}(hk, y, x, \vec{t}), \\ \rho = \text{Open}(hk, y, x, \vec{t}, j) \end{array} \right] = 1.$$

[0408] Succinctness. In the completeness experiment above, the size of the verification key vk and the hash value v is $\text{poly}(\lambda)$. The size of the hash key hk is at most $|g| \cdot \text{poly}(\lambda)$.

[0409] Computational binding. For any poly-size adversary $\mathcal{A}$ and polynomial $\ell(\lambda)$ there exists a negligible function $\text{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$, any function $g \in G_\lambda$ and any function input $y \in \{0, 1\}^\ell$,

$$\Pr \left[\begin{array}{l} \text{Verify}(vk, y, v, j, 0, t_0, \rho_0) = 1 \wedge \\ \text{Verify}(vk, y, v, j, 1, t_1, \rho_1) = 1 \end{array} : \begin{array}{l} (hk, vk, td) \leftarrow \text{Gen}(1^\lambda, g), \\ (v, j, t_0, t_1, \rho_0, \rho_1) \leftarrow \mathcal{A}(hk, vk) \end{array} \right] \leq \text{negl}(\lambda).$$

[0410] Function hiding. For any poly-size adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$ and $g_0, g_1 \in G_\lambda$ such that $|g_0| = |g_1|$,

$$\Pr \left[\mathcal{A}(hk, vk) = b : \begin{array}{l} b \leftarrow \{0, 1\} \\ (nk, vk, td) \leftarrow \text{Gen}(1^\lambda, g) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

[0411] Definition 8.4 (PEHash Consistency Properties). The hash family is furthermore required to satisfy the following consistency properties:

[0412] Index extraction correctness. For any $\lambda \in \mathbb{N}$, any $\ell \leq 2^\lambda$, any function $g \in G_\lambda$ and function input $y \in \{0, 1\}^\ell$ such that for $f = g(y) = (J, s)$ we have $J \neq \emptyset$, and any hash value v ,

$$\Pr [\text{ExtractIndex}(td, y, v) \in J : (hk, vk, td) \leftarrow \text{Gen}(1^\lambda, g)] = 1.$$

[0413] Consistency of extraction. For any poly-size adversary $\mathcal{A}$ holds that for any $g \in G_\lambda$, there exists a negligible function $\text{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$Pr \left[\begin{array}{l} \text{Extract}(td, y, v) = 1 \wedge j \in J \wedge \\ \text{Verify}(vk, y, v, j, 1, -s_j, t, \rho) = 1 \end{array} \right] :$$

$$\left[\begin{array}{l} (hk, vk, td) \leftarrow \text{Gen}(1^\lambda, g), \\ (y, v, j, t, \rho) \leftarrow \mathcal{A}(hk, vk), \\ (J, s) = g(y) \end{array} \right] \leq \text{negl}(\lambda),$$

and

$$Pr \left[\begin{array}{l} \text{Extract}(td, y, v) = 0 \wedge \\ \text{ExtractIndex}(td, y, v) = j \wedge \\ \text{Verify}(vk, y, v, j, s_j, t, \rho) = 1 \end{array} \right] :$$

$$\left[\begin{array}{l} (hk, vk, td) \leftarrow \text{Gen}(1^\lambda, g), \\ (y, v, j, t, \rho) \leftarrow \mathcal{A}(hk, vk), \\ (J, s) = g(y) \end{array} \right] \leq \text{negl}(\lambda).$$

[0414] Consistency of tag extraction. For any poly-size adversary $\mathcal{A}$ it holds that for any $g \in G_\lambda$, there exists a negligible function $\text{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$Pr \left[\begin{array}{l} \text{Extract}(td, y, v) = 0 \wedge \\ \text{ExtractIndex}(td, y, v) = j \wedge \\ \text{ExtractTag}(td, y, v) \neq t \wedge \\ \text{Verify}(vk, y, v, j, -s_j, t, \rho) = 1 \end{array} \right] : \left[\begin{array}{l} (hk, vk, td) \leftarrow \text{Gen}(1^\lambda, g) \\ (y, v, j, t, \rho) \leftarrow \mathcal{A}(hk, vk) \end{array} \right]$$

[0415] Theorem 8.5. Assuming fully homomorphic encryption, there exists a PEGHash family with tags with respect to the bit-fixing predicate family, which is G-functional with respect to any family G of polynomial-size circuits.

8.1.2 Functional Subset Extraction from Functional PEGHash

[0416] We now recall the construction of SNARGs for monotone policy BatchNP from prior work, and replace the PEGHash with a functional PEGHash to obtain a F-batch argument with G-functional subset extraction, for any family F of monotone circuit policies and G of polynomial size circuits. The construction uses the following building blocks:

[0417] A G'-functional PEGHash family with tags (Definition 8.4)

[0418] $(\text{Gen}_{\text{PEHT}}, \text{Hash}_{\text{PEHT}}, \text{Open}_{\text{PEHT}}, \text{Verify}_{\text{PEHT}}, \text{Extract}_{\text{PEHT}}, \text{ExtractIndex}_{\text{PEHT}}, \text{ExtractTag}_{\text{PEHT}})$ with respect to the bit-fixing predicate family, where G' is a family of polynomial size circuits, containing a function g' for every $g \in G_\lambda$, defined as follows:

[0419] 1. Receive an input $y \in \{0, 1\}^\ell$ and compute $g(y) = J \subseteq [k]$.

[0420] 2. Output $f = (J, s)$ where $s = 0^r$ is the all-zero string.

[0421] A G"-functional PEGHash family

[0422] $(\text{Gen}_{\text{PEH}}, \text{Hash}_{\text{PEH}}, \text{Open}_{\text{PEH}}, \text{Verify}_{\text{PEH}}, \text{Extract}_{\text{PEH}}, \text{ExtractIndex}_{\text{PEH}})$

[0423] with respect to the bit-fixing predicate family, where G" is a family of polynomial size circuits, containing a function g''_i for every $g \in G_\lambda$ and $i \in [d]$, defined as follows:

[0424] 1. Receive an input $y' = (C, y)$ where $C \in F_\lambda$ is a monotone circuit, and $y \in \{0, 1\}^\ell$ is an input to g .

[0425] 2. Compute $g(y) = J \subseteq [k]$.

[0426] 3. Let $(b^*_1, \dots, b^*_N)$ be the values of all the wires in C on input $\mathbb{1}[k]J$.

[0427] 4. Let $J' \subseteq [N]$ be the set of wires in the i th layer of C whose values b^* are 0.

[0428] 5. Output $f = (J', s)$ where $s = 0^{r'}$ is the all-zero string.

[0429] A seBARG scheme (Definition 3.5)

[0430] $(\text{Gen}_{\text{seBARG}}, \mathcal{P}_{\text{seBARG}}, \mathcal{V}_{\text{seBARG}}, \text{Extract}_{\text{seBARG}})$.

[0431] The analysis of the construction above follows identically to the analysis of the somewhere-extractable SNARG for monotone policy BatchNP in prior work. In particular, the G-functional subset extraction property follows similarly to the somewhere argument of knowledge property in prior work, and using the functional properties of the PEGHash. We thus obtain Theorem 8.1.

9 BARGs with Adaptive Subset Extraction for Low-Depth Policies

[0432] In this section we give a batch argument with adaptive subset extraction for families of monotone policies implemented by a low-depth monotone circuits.

[0433] We proceed by defining low-depth monotone circuit families, then state our main theorem and a corollary.

[0434] Definition 9.1 (Depth-d Monotone Circuit Policies). Let $d(\lambda)$ be a function and let $F = \{F_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of functions. F is a depth-d family of policies if there exist polynomials $N(\lambda)$, $k(\lambda)$ such that each $f \in F_\lambda$ is a monotone function $f: \{0, 1\}^k \rightarrow \{0, 1\}$ computable by a monotone circuit C of size N and depth d.

[0435] Theorem 9.2. Assuming a somewhere extractable batch argument scheme and a hash family with local opening, there exists a scheme $\text{BARG} = (\text{Setup}, \mathcal{P}, \mathcal{V}, \text{Extract})$ such that for any family F of depth-d policies where $d = O(\log \lambda)$, BARG is a F-batch argument with $O(k \cdot 2^d)$ -adaptive subset extraction for relation $\mathcal{R}_{\text{full}}$.

[0436] Corollary 9.3. Assuming a somewhere extractable batch argument scheme and a hash family with local opening, there exists a scheme $\text{AggS} = (\text{Setup}, \text{KeyAgg}, \text{SigAgg}, \text{AggVerify})$ such that for any family F of depth-d policies where $d = O(\log \lambda)$ and for any digital signature scheme S, AggS is a F-aggregation scheme for S.

10 Verifiable Private Information Retrieval (vPIR)

[0437] In this section we define vPIR. For simplicity and consistency with Section 6, we make the following modifications:

[0438] The Setup and Query algorithms are merged.

[0439] We do not consider verification with multiple queries.

[0440] We only consider publicly verifiable vPIR schemes.

[0441] We only consider read-once bounded-space constraints.

[0442] We consider simulation security for non-adaptive constraints. This differs from the definition in prior work, that considers adaptively chosen constraints, which requires limiting the constraint's description length to $N = O(\log \lambda)$. However, as noted in a remark in their work, simulation security for non-adaptive constraints holds even when their length is $N \geq \lambda$.

[0443] Read-once bounded-space constraints. For parameters T, S, we define a read-once bounded-space constraint represented as a Turing machine $\Gamma \in \{0, 1\}^N$. We say that a database $D = (r_i)_{i \in [k]} \in \{0, 1\}^{k \times \infty}$, satisfies Γ if and only if for every $i \in [k-1]$ there exists $c_i \in \{0, 1\}^S$ such that $\Gamma(c_{i-1}, r_i)$

outputs c_i within T steps where c_0, c_k are some fixed starting and accepting configurations. We denote by $U_{T,S}(\Gamma, D)$ the bit that indicates whether or not D satisfies the constraint r .

[0444] Syntax. A verifiable private information retrieval (vPIR) scheme consists of following polynomial-time algorithms:

- [0445] $\text{Query}(1^\lambda, t) \rightarrow (dk, vk, q)$. This is a probabilistic algorithm that takes as input the security parameter 1^λ and a row index $t \in [2^\lambda]$. It outputs a decryption key dk , a verification key vk and a query q .
- [0446] $\text{Answer}(D, 1^T, \Gamma, q) \rightarrow a$. This is a deterministic algorithm that takes as input a database $D \in \{0, 1\}^{k \times \omega}$, a time bound 1^T , a constraint Γ , and a query q . It outputs an answer a .
- [0447] $\text{Dec}(dk, a) \rightarrow \pi$. This is a deterministic algorithm that takes as input a decryption key dk , and an answer a . It outputs a row $\Gamma \in \{0, 1\}^\omega$.
- [0448] $\text{Verify}(vk, \Gamma, a) \rightarrow 0/1$. This is a deterministic algorithm that takes as input a verification key vk , a constraint Γ , and an answer a . It outputs a bit (1 to accept, 0 to reject).
- [0449] Definition 10.1. A Λ -secure vPIR scheme (Query, Answer, Dec, Verify) for $\mathcal{U}$ is required to satisfy the following properties:

- [0450] Completeness. For any $\lambda \in \mathbb{N}$, any $T, N, S, k, \omega \leq 2^\lambda$, database $D \in \{0, 1\}^{k \times \omega}$, row index $t \in [k]$ and constraint $\Gamma \in \{0, 1\}^N$ such that $U_{T,S}(\Gamma, D) = 1$,

$$\Pr \left[\begin{array}{l} \text{Dec}(dk, a) = D[t] \wedge (dk, vk, q) \leftarrow \text{Query}(1^\lambda, t) \\ \text{Verify}(vk, \Gamma, a) = 1 : a \leftarrow \text{Answer}(D, 1^T, \Gamma, q) \end{array} \right] = 1.$$

- [0451] Efficiency. In the completeness experiment above, $|vk| + |q| + |a| \leq \omega \cdot \text{poly}(\lambda, \log(kT))$.

- [0452] Λ -Privacy. For any $\text{poly}(\Lambda)$ -size adversary $\mathcal{A}$ and function $k(\lambda) \leq \Lambda(\lambda)$ there exists a negligible function negl such that for any $\lambda \in \mathbb{N}$ and row indices $t_0, t_1 \in [k(\lambda)]$,

$$\Pr \left[\mathcal{A}(vk, q) = b : \begin{array}{l} b \leftarrow \{0, 1\} \\ (dk, vk, q) \leftarrow \text{Query}(1^\lambda, t_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\Lambda).$$

- [0453] Λ -Simulation security. For any function $T(\lambda), N(\lambda), k(\lambda), \omega(\lambda) \leq \Lambda(\lambda)$, function $S(\lambda) = O(\log \Lambda)$, any $\text{poly}(\Lambda)$ -size adversary $\mathcal{A}$ and polynomial P there exists a $\text{poly}(\Lambda)$ -size simulator Sim such that for any $\text{poly}(\Lambda)$ -size distinguisher $\mathcal{D}$, $\lambda \in \mathbb{N}$, constraint Γ and row index $t \in [k]$,

$$\left| \Pr \left[\mathcal{D}(\text{Real}_{\mathcal{A}}(\lambda, \Gamma, t)) = 1 \right] - \Pr \left[\mathcal{D}(\text{Ideal}_{\text{Sim}}(\lambda, \Gamma, t)) = 1 \right] \right| < \frac{1}{P(\Lambda)},$$

where the experiments $\text{Real}_{\mathcal{A}}(\lambda, \Gamma, t)$ and $\text{Ideal}_{\text{Sim}}(\lambda, \Gamma, t)$ are defined as follows:

- [0454] $\text{Real}_{\mathcal{A}}(\lambda, \Gamma, t)$:
- [0455] Generate a query $(dk, vk, q) \leftarrow \text{Query}(1^\lambda, t)$.
- [0456] Run the adversary and obtain $a \leftarrow \mathcal{A}(vk, q)$.
- [0457] If $\text{Verify}(vk, \Gamma, a) = 1$ output $\text{Dec}(dk, a)$. Otherwise output $\perp$.

- [0458] $\text{Ideal}_{\text{Sim}}(\lambda, \Gamma, t)$:

- [0459] Run the simulator and obtain $D \leftarrow \text{Sim}(\lambda, \Gamma)$.

- [0460] If $U_{T,S}(\Gamma, D) = 1$ output $D[t]$. Otherwise output $\perp$.

[0461] Theorem 10.2. Assuming a Λ -secure somewhere extractable batch argument scheme and a Λ -secure hash family with local opening, there exists a Λ -secure vPIR scheme for read-once bounded-space constraints.

EXAMPLE EMBODIMENT APPLICATIONS

Blockchain and Cryptocurrency Applications

[0462] In some cases, the signature system **100** may be utilized in various blockchain and cryptocurrency applications, leveraging the power of monotone-policy aggregate signatures to enhance security, efficiency, and flexibility in decentralized systems. The method **200** for generating an aggregate signature and the method **300** for verifying an aggregate signature may be particularly valuable in these contexts.

[0463] One significant application may be in the realm of multi-signature wallets. Traditional multi-signature implementations often require all signers to coordinate during wallet creation and typically support only simple threshold policies. However, the signature system **100** may allow for more complex authorization rules without necessitating interactive setup phases. For example, a multi-signature wallet may implement a policy such as “any 2 of 3 regular users AND at least 1 recovery key holder” using the aggregate signature generation method **200**.

[0464] In this scenario, the common message **105** may represent a transaction to be authorized. Each local device (**110, 120, 121, 123**) may correspond to a different key holder. The key generation module **115** in each local device may generate a secret key **116** and verification key **117** pair. When a transaction needs to be authorized, the signature generation module **112** in each participating device may generate a message signature **113** using its secret key **116**.

[0465] The signature aggregator module **130** may then combine these individual signatures into an aggregate signature **131**, which can be verified by the verification module **135**. The policy encoded in the aggregate signature may ensure that the required combination of regular users and recovery key holders have signed off on the transaction.

[0466] The verification process, as outlined in method **300**, may involve receiving the signed common message at step **305**, the aggregate signature at step **306**, and the verification keys at step **307**. The hash generator **136** may generate a hash digest of the received verification keys at step **308**, which is then used in the verification process at step **309**.

[0467] Another powerful application of the signature system **100** may be in the context of Decentralized Autonomous Organizations (DAOs). DAOs could implement sophisticated voting mechanisms where voting weight depends on token holdings, reputation scores, or stakeholder categories. For instance, a DAO might require “approval from 60 percent of token weight OR unanimous consent from the core developer multisig AND the treasury multisig.”

[0468] In this case, the common message **105** might represent a proposal to be voted on. Each local device (**110, 120, 121, 123**) may represent a different stakeholder or group of stakeholders. The signature generation module **112** in each device may generate a signature based on the stakeholder's voting power or role.

[0469] The signature aggregator module 130 may combine these signatures, taking into account the different weights or categories of the signers. The resulting aggregate signature 131 may encapsulate the complex voting logic, allowing for efficient verification by the verification module 135.

[0470] The sublinear verification time of the signature system 100, as described in claim 9, may be particularly beneficial in this scenario. If all parties sign the same message (in this case, the proposal), the verification time may be sublinear in t , where t represents the threshold number of signers. This property may allow the DAO to efficiently process votes even with a large number of participants.

[0471] Cross-chain verification is another area where the signature system 100 may provide significant benefits. As blockchain ecosystems become more interconnected, there's a growing need for secure and efficient cross-chain communication. The monotone-policy aggregate signatures generated by the signature system 100 may enable cross-chain verification with policies that account for the varying trust levels of different chains.

[0472] For example, a cross-chain transaction might require "signatures from 2 of 3 validators on the source chain AND 3 of 5 validators on the destination chain." In this scenario, each local device (110, 120, 121, 123) may represent a validator from either the source or destination chain. The signature aggregator module 130 may combine the signatures from both chains, creating an aggregate signature 131 that encapsulates the cross-chain verification policy.

[0473] The verification module 135 may then efficiently verify this aggregate signature, ensuring that the required number of validators from each chain have approved the transaction. This approach may create more secure bridges between ecosystems, reducing the risk of cross-chain attacks.

[0474] In the context of proof-of-stake systems, the signature system 100 may enable more nuanced consensus rules than simple threshold approaches. Validator signatures could be weighted according to their stake amounts, leveraging the support for weighted thresholds described in claim 5 of the patent application.

[0475] For instance, in a proof-of-stake blockchain, each local device (110, 120, 121, 123) may represent a validator. The key generation module 115 in each device may generate a key pair where the verification key 117 is associated with the validator's stake amount. When participating in consensus, the signature generation module 112 may generate a signature, which is then sent to the signature aggregator module 130.

[0476] The signature aggregator module 130 may combine these signatures, taking into account the stake weight of each validator. The resulting aggregate signature 131 may represent a stake-weighted consensus decision. The verification module 135 may then efficiently verify this aggregate signature, ensuring that the required stake threshold has been met.

[0477] This approach may allow for more democratic and economically secure consensus mechanisms in proof-of-stake systems. Moreover, as stated in claim 11, the aggregation time may grow with the number of signatures t and not the number of parties k . This property may be particularly beneficial in large proof-of-stake networks, where the

number of active validators (t) in a given consensus round may be much smaller than the total number of registered validators (k).

Enterprise Systems Applications

[0478] The signature system 100 may also find significant applications in enterprise environments, where complex authorization policies and efficient verification are crucial. The ability to express intricate authorization rules without requiring interactive setup makes the system particularly valuable in corporate settings.

[0479] One key application may be in implementing hierarchical approval workflows. Corporate approval systems could leverage the signature system 100 to implement policies such as "any Vice President AND two Directors AND the department head" for financial authorizations. In this scenario, each level of the corporate hierarchy may be represented by a local device (110, 120, 121, 123) in the signature system 100.

[0480] For example, consider a financial transaction that requires approval from multiple levels of management. The common message 105 may represent the details of the financial transaction. Each approver's device may contain a key generation module 115 that generates a secret key 116 and verification key 117 pair specific to their role in the organization.

[0481] When the transaction needs approval, each relevant manager may use their local device to generate a signature using their signature generation module 112. These individual signatures may then be sent to the signature aggregator module 130, which combines them into an aggregate signature 131 that encodes the hierarchical approval policy. [0482] The verification module 135 may then verify this aggregate signature, ensuring that the required combination of approvals has been obtained. This process may remain efficient even as the organization scales, because, as stated in claim 9, if all parties sign the same message, the verification time may be sublinear in t , where t represents the threshold number of signers.

[0483] Another valuable application in enterprise systems may be in regulatory compliance automation. For regulated industries, the signature system 100 could be used to ensure that transactions satisfy specific regulatory requirements. For instance, a policy might require signatures that satisfy the rule "the compliance officer AND the department head AND either the CFO or CEO."

[0484] In this case, each of these roles may be associated with a local device (110, 120, 121, 123) in the signature system 100. The common message 105 may represent a transaction or decision that requires regulatory approval. Each required party may generate a signature using their signature generation module 112.

[0485] The signature aggregator module 130 may then combine these signatures into an aggregate signature 131 that encodes the regulatory compliance policy. The verification module 135 may verify this aggregate signature, ensuring that all regulatory requirements have been met. This approach may create an auditable enforcement of compliance requirements, as the aggregate signature 131 itself serves as cryptographic proof that the proper approvals were obtained.

[0486] The signature system 100 may also be particularly useful in multi-party contract execution scenarios. Legal agreements involving multiple organizations could establish

signature policies that reflect the contractual requirements. For example, a merger agreement might require “signatures from the CEO, CFO, and legal counsel from each participating company.”

[0487] In this scenario, each party to the agreement may be represented by a local device (110, 120, 121, 123) in the signature system 100. The common message 105 may represent the contract document. Each required signatory may generate a signature using their signature generation module 112.

[0488] The signature aggregator module 130 may combine these signatures into an aggregate signature 131 that encodes the complex signing requirements of the multi-party agreement. The verification module 135 may then verify this aggregate signature, ensuring that all required parties have signed off on the agreement. This approach may allow each organization to maintain independent control of their keys while still enabling efficient verification of the collective agreement.

[0489] Distributed treasury management is another area where the signature system 100 may provide significant benefits. Corporate treasury operations could implement controls requiring different levels of authorization for different transaction amounts. For instance, a policy might state “any two financial officers AND the treasurer for transactions under \$1M, or any three financial officers AND the CFO AND treasurer for transactions over \$1M.”

[0490] In this case, each financial officer, the CFO, and the treasurer may have their own local device (110, 120, 121, 123) in the signature system 100. The common message 105 may represent a treasury transaction. Depending on the transaction amount, the appropriate signatories may generate signatures using their signature generation modules 112.

[0491] The signature aggregator module 130 may combine these signatures into an aggregate signature 131 that encodes the treasury control policy. The verification module 135 may then verify this aggregate signature, ensuring that the appropriate level of authorization has been obtained based on the transaction amount. This approach may allow for granular financial controls that align with enterprise governance requirements, while still maintaining efficient verification.

[0492] The signature system 100 may also be valuable in enforcing segregation of duties policies. Security policies requiring separation of responsibilities could be encoded as monotone policies, such as “approval from one person in group A AND one person in group B, where no individual belongs to both groups.”

[0493] Each group may be represented by a set of local devices (110, 120, 121, 123) in the signature system 100. The common message 105 may represent an operation that requires segregated approval. Signatories from each group may generate signatures using their signature generation modules 112.

[0494] The signature aggregator module 130 may combine these signatures into an aggregate signature 131 that encodes the segregation of duties policy. The verification module 135 may then verify this aggregate signature, ensuring that the required separation of responsibilities has been maintained. This approach may provide cryptographic enforcement of critical security policies that would otherwise rely on application-level controls.

[0495] In multi-organization supply chains, the signature system 100 may be used to create tamper-evident documen-

tation that spans organizational boundaries. For example, a document authenticity policy might require “signatures from the manufacturer AND the quality inspector AND at least two logistics providers.”

[0496] Each party in the supply chain may be represented by a local device (110, 120, 121, 123) in the signature system 100. The common message 105 may represent a supply chain document or transaction. Each relevant party may generate a signature using their signature generation module 112.

[0497] The signature aggregator module 130 may combine these signatures into an aggregate signature 131 that encodes the supply chain verification policy. The verification module 135 may then verify this aggregate signature, ensuring that all required parties in the supply chain have signed off on the document or transaction. This approach may create a cryptographically verifiable trail of approvals across the supply chain, enhancing transparency and trust without requiring complex setup procedures between the various organizations involved.

[0498] The adaptability of the signature system 100 to complex organizational structures makes it particularly valuable for enterprises with sophisticated governance requirements or regulatory constraints. By offering cryptographic enforcement of authorization policies that would otherwise rely on application-level controls, the system may provide a higher level of security and auditability for critical business processes.

Cross-Chain Bridges and Interoperability

[0499] The signature system 100 may play a crucial role in enhancing the security and efficiency of cross-chain bridges, which are critical infrastructure for enabling interoperability between distinct blockchain networks. Cross-chain bridges facilitate the transfer of assets, data, and smart contract states across chains that would otherwise operate in silos, and the signature system 100 may address several key challenges in this domain.

[0500] One of the primary challenges in cross-chain bridge design is the need for robust security mechanisms that can withstand potential attacks while maintaining efficiency. The signature system 100, with its support for expressive monotone policies and efficient verification, may provide a powerful tool for addressing these challenges.

[0501] For example, consider a cross-chain bridge that requires signatures from validators on both the source and destination chains to authorize a transfer. The policy might state “signatures from 7 out of 10 validators on the source chain AND 6 out of 8 validators on the destination chain.” In this scenario, each validator may be represented by a local device (110, 120, 121, 123) in the signature system 100.

[0502] The common message 105 may represent the cross-chain transfer transaction. Validators from both chains may generate signatures using their signature generation modules 112. The signature aggregator module 130 may then combine these signatures into an aggregate signature 131 that encodes the cross-chain validation policy.

[0503] The verification module 135 may efficiently verify this aggregate signature, ensuring that the required number of validators from each chain have approved the transaction. This approach may create a more secure bridge between ecosystems by cryptographically enforcing the participation of validators from both chains.

[0504] Moreover, the sublinear verification time of the signature system **100**, as described in claim **9**, may be particularly beneficial in this context. If all validators sign the same transfer message, the verification time may be sublinear in t , where t represents the threshold number of signers. This property may allow for efficient verification of cross-chain transfers even with a large number of validators involved.

[0505] The signature system **100** may also address the scalability challenges faced by cross-chain bridges. As blockchain ecosystems grow and the number of cross-chain interactions increases, the efficiency of verification becomes crucial. The system's ability to provide succinct proofs and efficient verification may help prevent bottlenecks in cross-chain operations.

[0506] For instance, in a scenario where a cross-chain bridge needs to handle transfers between multiple chains simultaneously, the signature system **100** may allow for the creation of complex, multi-chain validation policies. A policy might require "signatures from 2 out of 3 validators on chain A AND 3 out of 5 validators on chain B AND 4 out of 6 validators on chain C."

[0507] Each validator across the multiple chains may be represented by a local device (**110**, **120**, **121**, **123**) in the signature system **100**. The signature aggregator module **130** may combine signatures from validators across all involved chains into a single aggregate signature **131**. This aggregate signature may encapsulate the entire multi-chain validation policy.

[0508] The verification module **135** may then efficiently verify this aggregate signature, ensuring that the required number of validators from each chain have approved the multi-chain operation. This approach may allow for complex, multi-chain interactions to be verified efficiently, potentially enabling new forms of cross-chain applications and protocols.

[0509] Furthermore, the adaptive security property of the signature system **100** may be particularly valuable in the context of cross-chain bridges. As stated in the patent application, the system provides resistance to adaptive subset extraction, ensuring that even if adversaries corrupt validators after seeing the common reference string (CRS), the threshold policy remains enforceable.

[0510] This property may be crucial for cross-chain bridges, where the set of validators may change over time and where adversaries might attempt to target specific validators after observing the system's parameters. The adaptive security of the signature system **100** may provide an additional layer of protection against such targeted attacks.

[0511] In the context of modular blockchains and rollup ecosystems, which increasingly depend on secure bridging for liquidity sharing and interoperability, the signature system **100** may offer significant advantages. For example, in a rollup system where multiple layer-2 solutions need to interact with a base layer, the signature system **100** could be used to create efficient and secure validation mechanisms for state transitions between layers.

[0512] A policy for such a system might require "signatures from 3 out of 5 rollup validators AND 2 out of 3 base layer validators." Each validator, whether from the rollup or the base layer, may be represented by a local device (**110**, **120**, **121**, **123**) in the signature system **100**. The signature aggregator module **130** may combine signatures from both

sets of validators into an aggregate signature **131** that encodes this multi-layer validation policy.

[0513] The verification module **135** may then efficiently verify this aggregate signature, ensuring that the required participation from both the rollup and base layer validators has been achieved. This approach may enable secure and efficient state transitions in complex, multi-layer blockchain architectures.

[0514] The signature system **100** may also address regulatory concerns surrounding cross-chain bridges. As bridges come under increased regulatory scrutiny due to their role in facilitating cross-chain transactions, the ability to create and efficiently verify complex authorization policies may become increasingly important.

[0515] For instance, a policy might require "signatures from 2 out of 3 bridge operators AND 1 compliance officer AND either the CFO or CEO of the bridge operating entity." Each of these roles may be represented by a local device (**110**, **120**, **121**, **123**) in the signature system **100**. The signature aggregator module **130** may combine signatures from these various parties into an aggregate signature **131** that encodes this compliance-oriented policy.

[0516] The verification module **135** may efficiently verify this aggregate signature, ensuring that all regulatory requirements have been met. This approach may create an auditable trail of approvals for cross-chain transactions, potentially helping bridge operators meet evolving compliance demands.

Detailed Description of Local, Remote, and Distributed Components

[0517] The present disclosure provides methods and systems for generating and verifying aggregate signatures of a message, which can be implemented in various types of computer systems and software tools. These systems can be local, cloud-based, or distributed, depending on the specific requirements of the system, performance requirements, and security considerations.

Local Implementation

[0518] In a local implementation, all components of the aggregate signature generation and verification apparatus **100** are housed within a single device or system. This includes the local storage device, the local cryptographic module, the local signature generation module, the transmission module, the aggregator module, and the verification module. The local storage device stores the common message, and the local cryptographic module generates a secret cryptographic key and a verification cryptographic key pair for a user. The local signature generation module generates a user signature for the common message using the secret cryptographic key for the user. The transmission module transmits the user signature to the aggregator module, which combines the user signature with other user signatures to generate an aggregate signature. The aggregate signature is then transmitted to the verification module for verification.

Remote Implementation

[0519] In a remote or cloud-based implementation, some or all components of the aggregate signature generation and verification apparatus **100** may be housed on a remote server or cloud platform. The common message and the user signatures may be transmitted to the remote server or cloud

platform over a network, and the aggregate signature generation and verification processes are performed remotely. The aggregate signature is then transmitted back to the local device or system for further use or storage. This approach can offload the computational work to the cloud, reducing the resource requirements on the local device but may introduce potential security risks due to the reliance on a third-party service.

Distributed Implementation

[0520] In a distributed implementation, the components of the aggregate signature generation and verification apparatus **100** are distributed across multiple devices or nodes in a network. Each node may store a portion of the common message and generate a portion of the user signatures. The user signatures are then transmitted to an aggregator node, which combines the user signatures to generate an aggregate signature. The aggregate signature is then transmitted to a verification node for verification. This approach can provide better fault tolerance and scalability but may require more complex coordination and communication among the participating nodes.

[0521] FIGS. **10** and **11** depict example computer systems useful for implementing various embodiments described in the present disclosure. Various embodiments may be implemented, for example, using one or more computer systems, such as computer system **500** shown in FIG. **10**. One or more computer system(s) **500** may be used, for example, to implement any of the embodiments discussed herein, as well as combinations and sub-combinations thereof.

[0522] Computer system **500** may include one or more processors (also called central processing units, processing devices, or CPUs), such as a processor **504**. Processor **504** may be connected to a communication infrastructure **506** (e.g., such as a bus).

[0523] Computer system **500** may also include user input/output device(s) **503**, such as monitors, keyboards, pointing devices, etc., which may communicate with communication infrastructure **506** through user input/output interface(s) **502**. One or more of processors **504** may be a graphics processing unit (GPU). In an embodiment, a GPU may be a processor that is a specialized electronic circuit designed to process mathematically intensive applications. The GPU may have a parallel structure that is efficient for parallel processing of large blocks of data, such as mathematically intensive data common to computer graphics applications, images, videos, etc.

[0524] Computer system **500** may also include a main memory **508**, such as random-access memory (RAM). Main memory **508** may include one or more levels of cache. Main memory **508** may have stored therein control logic (i.e., computer software, instructions, etc.) and/or data. Computer system **500** may also include one or more secondary storage devices or secondary memory **510**. Secondary memory **510** may include, for example, a hard disk drive **512** and/or a removable storage device or removable storage drive **514**. Removable storage drive **514** may interact with a removable storage unit **518**. Removable storage unit **518** may include a computer-usable or readable storage device having stored thereon computer software (control logic) and/or data. Removable storage drive **514** may read from and/or write to removable storage unit **518**.

[0525] Secondary memory **510** may include other means, devices, components, instrumentalities, or other approaches

for allowing computer programs and/or other instructions and/or data to be accessed by computer system **500**. Such means, devices, components, instrumentalities, or other approaches may include, for example, a removable storage unit **522** and an interface **520**. Examples of the removable storage unit **522** and the interface **520** may include a program cartridge and cartridge interface, a removable memory chip (such as an EPROM or PROM) and associated socket, a memory stick and USB port, a memory card and associated memory card slot, and/or any other removable storage unit and associated interface.

[0526] Computer system **500** may further include communications interface **524** (e.g., network interface). Communications interface **524** may enable computer system **500** to communicate and interact with any combination of external devices, external networks, external entities, etc. (individually and collectively referenced as remote device(s), network(s), entity(ies) **528**). For example, communications interface **524** may allow computer system **500** to communicate with external or remote device(s), network(s), entity(ies) **528** over communications path **526**, which may be wired and/or wireless (or a combination thereof), and which may include any combination of LANs, WANs, the Internet, etc. Control logic and/or data may be transmitted to and from computer system **500** via communications path **526**.

[0527] Computer system **500** may also be any of a personal digital assistant (PDA), desktop workstation, laptop or notebook computer, netbook, tablet, smartphone, smart-watch or other wearable devices, appliance, part of the Internet-of-Things, and/or embedded system, to name a few non-limiting examples, or any combination thereof.

[0528] Computer system **500** may be a client or server computing device, accessing or hosting any applications and/or data through any delivery paradigm, including but not limited to remote or distributed cloud computing solutions; local or on-premises software (“on-premise” cloud-based solutions); “as a service” models (e.g., content as a service (CaaS), digital content as a service (DCaaS), software as a service (SaaS), managed software as a service (MSaaS), platform as a service (PaaS), desktop as a service (DaaS), framework as a service (FaaS), backend as a service (BaaS), mobile backend as a service (MBaaS), infrastructure as a service (IaaS), etc.); and/or a hybrid model including any combination of the foregoing examples or other services or delivery paradigms.

[0529] FIG. **11** illustrates an example machine of a computer system **900** within which a set of instructions, for causing the machine to perform any one or more of the operations discussed herein, may be executed. In alternative implementations, the machine may be connected (e.g., networked) to other machines in a LAN, an intranet, an extranet, and/or the Internet. The machine may operate in the capacity of a server or a client machine in a client-server network environment, as a peer machine in a peer-to-peer (or distributed) network environment, or as a server or a client machine in a cloud computing infrastructure or environment.

[0530] The machine may be a personal computer (PC), a tablet PC, a set-top box (STB), a Personal Digital Assistant (PDA), a cellular telephone, a web appliance, a server, a network router, a switch or bridge, a specialized application or network security appliance or device, or any machine capable of executing a set of instructions (sequential or otherwise) that specify actions to be taken by that machine.

Further, while a single machine is illustrated, the term “machine” shall also be taken to include any collection of machines that individually or jointly execute a set (or multiple sets) of instructions to perform any one or more of the methodologies discussed herein.

[0531] The example computer system 900 includes a processing device 902, a main memory 904 (e.g., read-only memory (ROM), flash memory, dynamic random-access memory (DRAM) such as synchronous DRAM (SDRAM), etc.), a static memory 906 (e.g., flash memory, static random-access memory (SRAM), etc.), and a data storage device 918, which communicate with each other via a bus 930.

[0532] Processing device 902 represents one or more processing devices such as a microprocessor, a central processing unit, or the like. More particularly, the processing device may be complex instruction set computing (CISC) microprocessor, reduced instruction set computing (RISC) microprocessor, very long instruction word (VLIW) microprocessor, or processor implementing other instruction sets, or processors implementing a combination of instruction sets. Processing device 902 may also be one or more special-purpose processing devices such as an application-specific integrated circuit (ASIC), a field-programmable gate array (FPGA), a digital signal processor (DSP), network processor, or the like. The processing device 902 is configured to execute instructions 926 for performing the operations and steps discussed herein.

[0533] The computer system 900 may further include a network interface device 908 to communicate over the network 920. The computer system 900 also may include a video display unit 910, an alphanumeric input device 912 (e.g., a keyboard), a cursor control device 914 (e.g., a mouse), a graphics processing unit 922, a signal generation device 916 (e.g., a speaker), graphics processing unit 922, video processing unit 928, and audio processing unit 932.

[0534] The data storage device 918 may include a machine-readable medium 924 (also known as a computer-readable storage medium) on which is stored one or more sets of instructions 926 (e.g., software instructions) embodying any one or more of the operations described herein. The instructions 926 may also reside, completely or at least partially, within the main memory 904 and/or within the processing device 902 during execution thereof by the computer system 900, where the main memory 904 and the processing device 902 also constitute machine-readable storage media.

[0535] In an example, the instructions 926 include instructions to implement operations and functionality corresponding to the disclosed subject matter. While the machine-readable storage medium 924 is shown in an example implementation to be a single medium, the term “machine-readable storage medium” should be taken to include a single medium or multiple media (e.g., a centralized or distributed database, and/or associated caches and servers) that store the one or more sets of instructions 926. The term “machine-readable storage medium” shall also be taken to include any medium that is capable of storing or encoding a set of instructions 926 for execution by the machine and that cause the machine to perform any one or more of the operations of the present disclosure. The term “machine-readable storage medium” shall accordingly be taken to include, but not be limited to, solid-state memories, optical media, and magnetic media.

[0536] Some portions of the detailed description have been presented in terms of algorithms and symbolic representations of operations on data bits within a computer memory. These algorithmic descriptions and representations are the ways used by those skilled in the data processing arts to most effectively convey the substance of their work to others skilled in the art. An algorithm is here, and generally, conceived to be a self-consistent sequence of operations leading to a desired result. The operations are those requiring physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical or magnetic signals capable of being stored, combined, compared, and otherwise manipulated. It has proven convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like.

[0537] It should be borne in mind, however, that all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. Unless specifically stated otherwise as apparent from the above discussion, it is appreciated that throughout the description, discussions utilizing terms such as “identifying” or “determining” or “executing” or “performing” or “collecting” or “creating” or “sending” or the like, refer to the action and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system’s registers and memories into other data similarly represented as physical quantities within the computer system memories or registers or other such information storage devices.

[0538] The present disclosure also relates to an apparatus for performing the operations herein. This apparatus may be specially constructed for the intended purposes, or it may comprise a computer selectively activated or reconfigured by a computer program stored in the computer. Such a computer program may be stored in a computer-readable storage medium, such as but not limited to, any type of disk including floppy disks, optical disks, CD-ROMs, and magnetic-optical disks, read-only memories (ROMs), random access memories (RAMs), EPROMs, EEPROMs, magnetic or optical cards, or any type of media suitable for storing electronic instructions, each coupled to a computer system bus.

[0539] The operations and illustrations presented herein are not inherently related to any particular computer or other apparatus. Various types of systems may be used with programs in accordance with the teachings herein, or it may prove convenient to construct a more specialized apparatus to perform the operations. The structure for a variety of these systems will appear as set forth in the description herein. In addition, the present disclosure is not described with reference to any particular programming language. It will be appreciated that a variety of programming languages may be used to implement the teachings of the disclosure as described herein.

[0540] The present disclosure may be provided as a computer program product, or software, that may include a machine-readable medium having stored thereon instructions, which may be used to program a computer system (or other electronic devices) to perform a process according to the present disclosure. A machine-readable medium includes any mechanism for storing information in a form readable by a machine (e.g., a computer). For example, a machine-

readable (e.g., computer-readable) medium includes a machine (e.g., a computer) readable storage medium such as read-only memory (“ROM”), random access memory (“RAM”), magnetic disk storage media, optical storage media, flash memory devices, etc.

[0541] In some embodiments, a tangible, non-transitory apparatus or article of manufacture comprising a tangible, non-transitory computer useable or readable medium having control logic (software) stored thereon may also be referred to herein as a computer program product or program storage device. This includes, but is not limited to, computer system **500**, main memory **508**, secondary memory **510**, and removable storage units **518** and **522**, as well as tangible articles of manufacture embodying any combination of the foregoing. Such control logic, when executed by one or more data processing devices (such as computer system **500**), may cause such data processing devices to operate as described herein.

[0542] Based on the teachings contained in this disclosure, it will be apparent to persons skilled in the relevant art(s) how to make and use embodiments of this disclosure using data processing devices, computer systems, and/or computer architectures other than that shown in FIGS. **10** and **11**. In particular, embodiments can operate with software, hardware, and/or operating system implementations other than those described herein.

[0543] It is to be appreciated that the Detailed Description section, and not any other section, is intended to be used to interpret the claims. Other sections can set forth one or more but not all exemplary embodiments as contemplated by the inventor(s), and thus, are not intended to limit this disclosure or the appended claims in any way.

[0544] While this disclosure describes exemplary embodiments for exemplary fields and applications, it should be understood that the disclosure is not limited thereto. Other embodiments and modifications thereto are possible and are within the scope and spirit of this disclosure. For example, and without limiting the generality of this paragraph, embodiments are not limited to the software, hardware, firmware, and/or entities illustrated in the figures described herein. Further, embodiments (whether or not explicitly described herein) have significant utility to fields and applications beyond the examples described herein.

[0545] Embodiments have been described herein with the aid of functional building blocks illustrating the implementation of specified functions and relationships thereof. The boundaries of these functional building blocks have been arbitrarily defined herein for the convenience of the description. Alternate boundaries can be defined as long as the specified functions and relationships (or equivalents thereof) are appropriately performed. Also, alternative embodiments can perform functional blocks, steps, operations, methods, etc. using orderings different than those described herein.

[0546] References herein to “one embodiment,” “an embodiment,” “an example embodiment,” or similar phrases, indicate that the embodiment described can include a particular feature, structure, or characteristic, but every embodiment may not necessarily include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it would be within the knowledge of persons skilled in the relevant art(s) to incorporate such feature, structure, or characteristic into

other embodiments whether or not explicitly mentioned or described herein. Additionally, some embodiments can be described using the expression “coupled” and “connected” along with their derivatives. These terms are not necessarily intended as synonyms for each other. For example, some embodiments can be described using the terms “connected” and/or “coupled” to indicate that two or more elements are in direct physical or electrical contact with each other. The term “coupled,” however, can also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

[0547] The breadth and scope of this disclosure should not be limited by any of the above-described exemplary embodiments but should be defined only in accordance with the following claims and their equivalents. In the foregoing specification, implementations of the disclosure have been described with reference to specific example implementations thereof. It will be evident that various modifications may be made thereto without departing from the broader spirit and scope of implementations of the disclosure as set forth in the following claims. The specification and drawings are, accordingly, to be regarded in an illustrative sense rather than a restrictive sense.

1. A method for generating an aggregate signature of a message, the method comprising:

- at a local storage device, storing a common message;
- at a local key generation module, generating a secret cryptographic signing key and a public cryptographic verification key pair for a user using a collision-resistant hash function, and transmitting the public cryptographic verification key to a hash generation module;

- at a local signature generation module, generating a user signature for the common message using the secret cryptographic signing key for the user;

- transmitting the user signature to an aggregator module;
- at the aggregator module, combining the user signature for the common message with other user signatures for the common message received from other users to generate an aggregate signature using non-interactive batch arguments;

- transmitting the aggregate signature to a verification module; wherein

- the aggregate signature is constructed from the non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies, wherein:

- a size of the aggregate signature is sublinear in a number of signers and is based on a security parameter λ ;

- the non-interactive batch arguments are constructed from verifiable private information retrieval; and

- the aggregate signature is capable of being verified in sublinear time $O(\log k)$ when all parties sign the common message, where k is the number of signers.

2. The method of claim **1**, wherein the non-interactive batch arguments that support adaptive subset extraction for monotone policies are constructed from Verifiable Private Information Retrieval.

3. The method of claim **1**, wherein an untrusted aggregator party combines the signatures of multiple parties into an aggregate signature $\hat{\sigma}$ to certify expressive monotone policies over the signatures.

4. The method of claim **3**, wherein at least t out of k parties signed the common message.

5. The method of claim 3, further comprising weighted thresholds where each party has a different weight and a signing policy is defined with respect to a sum of the party weights.

6. The method of claim 1, wherein: the method relies only on standard cryptographic assumptions; and constructions are in a common reference string (CRS) model where a size of the CRS grows only polylogarithmically in the number of signers.

7. The method of claim 1, wherein:

the size of the aggregate signature is sublinear in k and t and only depends on the security parameter λ ; and
if all parties sign the common message, the verification time is sublinear in t .

8. The method of claim 1, wherein:

if a vector of messages has a succinct representation, the verification time is sublinear in t ; and
aggregation time grows with a number of signatures t and not the number of signers k .

9. A method for verifying an aggregate signature of a message, the method comprising:

at a local storage device, storing a common message;
at a local key generation module, generating a secret key and a verification key pair for a user, and transmitting the verification key to a hash generation module;
at a local signature generation module, generating a user signature for the common message using the secret key for the user;

transmitting the user signature to an aggregator module;
at the aggregator module, combining the user signature for the common message with other user signatures for the common message received from other users to generate an aggregate signature;

transmitting the aggregate signature to a verification module;

at the verification module, receiving the common message that has been signed by multiple users;
receiving the aggregate signature for the signed common message;

receiving verification keys from one or more of the multiple users that signed the common message;
generating a hash digest of the received verification keys;
verifying the aggregate signature for the signed common message based on the hash digest of the received verification keys, the common message, and the aggregate signature for the signed common message; and
storing a result of the verification of the aggregate signature;

wherein the aggregate signature is constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies.

10. The method of claim 9, wherein the verifying is performed without knowing an identity of the multiple users that signed the common message.

11. An apparatus for generating an aggregate signature of a message, the apparatus comprising:

a local storage device configured to store a common message;

a local key generation module configured to generate a secret key and a verification key pair for a user, and transmit the verification key to a hash generation module;

a local signature generation module configured to generate a user signature for the common message using the secret key for the user;

a transmission module configured to transmit the user signature to an aggregator module;

the aggregator module configured to combine the user signature for the common message with other user signatures for the common message received from other users to generate an aggregate signature;

the transmission module further configured to transmit the aggregate signature to a verification module; and

wherein:

the aggregate signature is constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies;

a size of the aggregate signature is sublinear in a number of signers and is based on a security parameter λ ;

the non-interactive batch arguments are constructed from verifiable private information retrieval; and

the aggregate signature is capable of being verified in sublinear time $O(\log k)$ when all parties sign the common message, where k is the number of signers.

12. The apparatus of claim 11, wherein the non-interactive batch arguments that support adaptive subset extraction for monotone policies are constructed from Verifiable Private Information Retrieval.

13. The apparatus of claim 11, wherein an untrusted aggregator party combines the signatures of multiple parties into an aggregate signature to certify expressive monotone policies over the signatures.

14. The apparatus of claim 13, wherein:

at least a threshold number of parties signed the common message; and

further comprising a weighted threshold module where each party has a different weight and a signing policy is defined with respect to a sum of the party weights; and

wherein the apparatus relies on standard cryptographic assumptions.

15. The apparatus of claim 11, wherein the non-interactive batch arguments are in a common reference string (CRS) model where a size of the CRS grows polylogarithmically with the number of signers.

16. The apparatus of claim 11, wherein the size of the aggregate signature is sublinear in the number of signers and a threshold number of signers and depends on the security parameter λ .

17. The apparatus of claim 11, wherein:

if all parties sign the common message, a verification time is sublinear in a threshold number of signers; and

if a vector of messages has a succinct representation, the verification time is sublinear in the threshold number of signers.

18. The apparatus of claim 11, wherein an aggregation time grows with a number of signatures and not the number of signers.

19. An apparatus for verifying an aggregate signature of a message, the apparatus comprising:

a local storage device configured to receive a common message that has been signed by multiple users;

a first reception module configured to receive an aggregate signature for the signed common message;

a second reception module configured to receive verification cryptographic keys from one or more of the multiple users that signed the common message;
a hash generation module configured to generate a hash digest of the received verification cryptographic keys;
a verification module configured to verify the aggregate signature for the signed common message based on the hash digest of the received verification cryptographic keys, the common message, and the aggregate signature for the signed common message;
a storage module configured to store a result of the aggregate signature verification; and
wherein the aggregate signature is constructed from non-interactive batch arguments that support adaptive subset extraction for expressive monotone policies.

20. The apparatus of claim **19**, wherein the verification module is configured to verify the aggregate signature without knowing an identity of the multiple users that signed the common message.

* * * * *